

PITANJA ZA USMENI ISPIT IZ PREDMETA OPERATIVNI SISTEMI 2020

Napomena: Pitanja su numerisana kao kod profesora na c2 (ima par gresaka u oblasti 2)

1.1. Šta je to proces, razlika programa i procesa?

Proces predstavlja izvršavanje nekog programa nad datim skupom podataka. Pojmovi program i proces su slični, ali ipak različiti jer je program pasivni, a proces aktivni entitet koji uključuje i stanje memorije i procesora u toku izvršenja.

1.2. Fundamentalni memorijski dijelovi procesa.

Znamo da procesor može izvršavati samo procese koji su u internoj memoriji. Kako je operativna memorija relativno malog kapaciteta, dolazimo do sljedećih problema:

-Proces je veći od interne memorije (problem se dešava i kod monoprogramiranja i kod multiprogramiranja)

-Nemamo dovoljno memorije da učitamo sve spremne procese (problem se dešava kod multiprogramiranja).

Virtualna memorija predstavlja posebnu memoriju i razlikuje se od fizičke memorije. Operativni sistem drži dijelove programa koji su u tom trenutku potrebni u glavnoj memoriji a ostatak na disku. Virtualna memorija funkcionise i kod multiprogramiranja, pri tome dijelovi različitih programa mogu istovremeno biti u memoriji. Osnovna ideja je da programi mogu imati proizvoljnu veličinu a OS treba voditi računa o tome da u memoriji budu samo oni dijelovi procesa (i kod i podaci) koji su u datom trenutku potrebni, a ostatak neka čeka na disku. Znači jedan dio procesa je u internoj memoriji a drugi dio je u eksternoj memoriji, a sam proces ne zna ništa o tome – njemu se čini da ima dovoljno operativne memorije i da se čitav proces nalazi u internoj memoriji. Drugim riječima: ako nema dovoljno memorije, uzećemo malo od diska – procesi neće znati za to, možda korisnik primijeti usporavanje rada programa.

1.3.Šta čini process control block?

PCB sadrži važne informacije o određenom procesu, direktno ili indirektno:

- Trenutno stanje procesa: ready, running, waiting, i dr.
- Jedinstvenu identifikaciju procesa (ime ili jedinstveni identifikator procesa PID)
- Pokazivač na proces roditelj
- Pokazivač na child proces (ako postoji)
- Prioritet procesa (dio koji se odnosi na raspoređivanje CPU)
- Pokazivač koji pokazuje na memoriju procesa
- Područje registara
- Informacije o memoriji procesa
- Lista otvorenih datoteka
- Status zauzetih ulazno-izlaznih resursa
- Procesor na kome radi (ako ima više procesora)

1.4.Koji događaji pokreću proces?

Postoje četiri događaja koje uzrokuju pokretanje jednog procesa:

- inicijalizacija sistema
- sistemski poziv za pokretanjem procesa kroz neki drugi proces
- korisnički zahtjev za stvaranjem procesa
- inicijalizacija batch poslova

1.5.Koji događaji završavaju proces?

Prije ili kasnije proces završava rad iz jednog od sljedećih razloga:

- normalni izlaz (dobrovoljno)
- izlaz sa greškom (dobrovoljno)
- fatalna greška (nije dobrovoljno)
- ubijen (killed) od strane drugog procesa (nije dobrovoljno)

1.6.U kojim stanjima može biti proces?

Svaki proces može biti u jednom od sljedećih stanja:

- New- novi: proces je upravo kreiran
- Running – aktivan (izvršava se): instrukcije se izvršavaju
- Waiting (blocked) – čeka (blokiran): proces čeka na pojavu nekog događaja kao što je neka U/I operacija
- Ready – spreman: proces čeka da bude dodijeljen procesoru
- Terminated – završen: proces je završio izvršenje

1.7. Koji događaji prebacuju proces sa izvršenja u neke od redova čekanja?

- Vrijeme za proces je isteklo
- Pokrenut drugi proces te se čeka njegovo izvršavanje.
- Hardverski prekid, prelazi se na izvršavanje u jezgru.
- Čekanje događaja
- Zahtjev ka periferijskom događaju

1.8. Šta radi dispečer?

Kada proces treba da pređe iz drugog stanja u stanje RUNNING ili iz stanja RUNNING u drugo stanje, dešava se izmjena konteksta. Izmjenu konteksta obavlja dispečer, koji se izvršava u jezgru i prepušta procesor procesu koga je za taj trenutak odabrao raspoređivač. Funkcije dispečera su izmjena konteksta, prelazak u korisnički režim rada i skok na odgovarajuću lokaciju u korisničkom programu.

1.9. Šta se dešava pri izmjeni konteksta?

Tokom izmjene konteksta se sačuva stanje procesora ili niti koja se prethodno izvršavala i učitana ranije sačuvano stanje za izvršavanje nove niti. Izmjena konteksta podrazumijeva, najjednostavnije rečeno, pomjeranje pokazivača na trenutni skup registara.

1.10. Kako je postignuto na i386 da jednom instrukcijom skoka možemo promijeniti kontekst procesa?

1.11. Koji događaji završavaju proces?

Prije ili kasnije proces završava rad iz jednog od sljedećih razloga:

- normalni izlaz (dobrovoljno)
- izlaz sa greškom (dobrovoljno)
- fatalna greška (nije dobrovoljno)
- ubijen (killed) od strane drugog procesa (nije dobrovoljno)

1.12. Kojom Windows funkcijom se kreira proces, koliko ima parametara i koji su najvažniji

Korištenjem funkcije CreateProcess se kreira novi proces. Ova funkcija ima deset složenih parametara, ali većini njih se može dodijeliti podrazumijevana prazna vrijednost. Najvažniji parametri su lpApplicationName, i lpCommandLine koji pokazuju na niz znakova sa imenom izvršnog programa procesa i njegovim parametrima iz komandne linije. Parametar lpEnvironment pokazuje na niz znakova koji sadrže konfiguracijske vrijednosti sistema oblika IME=VRIJEDNOST. Parametar lpStartupInfo je pokazivač na strukturu od 18 polja koji govore o veličini i položaju prozora aplikacije, boji tekstualnih aplikacija, preusmjerenju ulaza i drugo. Polje cb ove strukture mora biti inicijalizirano na njenu veličinu. Informacije o procesu se dobivaju preko posljednjeg parametra lpProcessInformation koji je pokazivač na strukturu od 4 polja u kojoj se nalaze identifikatori i rukovatelji procesa i njegove glavne niti.

1.13. Kojom POSIX funkcijom se kreira proces i kako ona radi?

Sistemska poziv fork() kreira novi proces. Kada proces pozove ovaj poziv, pokrenuti proces postaje "dijete" procesa koji ga je pokrenuo. Dijete proces dobiva kopije područja instrukcija i područja podataka od roditeljskog procesa. Pošto se područje instrukcija normalno ne mijenja, ne radi se fizičko kopiranje, nego se dio memorije dijeli između oba procesa i tako štedi prostor i vrijeme

1.14. Kada nastaje Zombie proces?

Ukoliko prilikom wait() poziva, dijete proces završi sa radom, a roditelj proces ga ne čeka sa wait(), tada se dijete proces ne briše odmah. Dijete-proces neće biti obrisan jer bi se u tom slučaju izgubio izlazni kôd. Tada dijete-proces postaje zombi-proces. Proces više nije u memoriju, ali se podaci o njemu još čuvaju. Ukoliko roditelj-proces završi bez wait poziva, tada će init proces (PID=1) preuzeti hijerarhiju nad dijete-procesom i obrisati zombi procese koje je preuzeo.

1.15. Šta su programske niti i koje su im prednosti nad procesima?

Programska nit je osnovna jedinica korištenja CPU. Programske niti se ponekad nazivaju laki (lightweight) proces LWP. Nit ima svoj programski brojač, skup registara i stek prostora. Svaka nit dijeli sa svojom partnerskom niti kodnu sekciju, sekciju podataka, resurse operativnog sistema. Nit i proces su različiti koncepti i moraju se tretirati odvojeno. Procesi grupišu resurse, niti su osnovni entiteti koji se izvršavaju na CPU.

1.16. Kako je realizovano raspoređivanje niti u korisničkom prostoru?

Ovaj metod je da se paket programske niti postavi potpuno u korisnički prostor. Jezgro ne zna ništa o njemu. Što se jezgra tiče, to je običan jednonitni proces. Prva i najočiglednija prednost je da programske niti na korisničkom nivou (userlevel thread) mogu biti implementirane na operativnom sistemu koji ne podržava programske niti. Većina operativnih sistema su bili u ovoj kategoriji a neki su još uvijek. Sve ovakve implementacije imaju istu generalnu strukturu koja je ilustrirana na slici. Međutim, ova implementacija ima velike nedostatke. Prvi je implementacija blokirajućeg sistemskog poziva, gdje je nedopustivo dozvoliti niti da pravi sistemski poziv. Drugi problem kod ove implementacije je taj da ako jedna programska nit počne rad, nijedna druga nit u tom procesu neće raditi osim ako prva nit ne prepusti CPU dobrovoljno. Treći problem je najteži, a to je da programeri žele niti upravo u aplikacijama koje često blokiraju, kao što je višenitni Web server.

1.17. Kako je realizovano raspoređivanje niti u jezgri?

Razmatrajući situaciju u kojoj jezgro zna za niti i upravlja njima dolazimo na to da nije potreban runtime sistem. Nema tabele niti u svakom procesu. Umjesto toga jezgro ima tabelu programskih niti koja prati sve niti u sistemu. Kad nit želi da kreira ili da uništi postojeću nit, pravi poziv u jezgro, koje onda kreira ili uništava niti tako što obnavlja njihovu tabelu.

1.18. Kako se u Windows-u kreiraju niti?

Niti se u Windows-u kreiraju funkcijom `CreateThread()`. Prvi parametar je `NULL` ako nije potrebno prosljeđivati rukovatelje na objekte u kreirane procese. Drugi parametar određuje veličinu područja u stack sekciji za navedenu nit. Kao treći parametar funkcije `CreateThread` prosljeđuje se funkcija, koja će se izvršavati kada započne izvršavanje niti. Ova funkcija mora biti pisana da ima jedan parametar i to pokazivač na tip `void`. Iako to izgleda ozbiljno ograničenje, složeniji parametri se mogu proslijediti prema niti pravljenjem strukture i zatim prosljeđivanjem pokazivača na nju. Parametar funkcije koja se izvršava na nit se prosljeđuje kao četvrti parametar funkcije `CreateThread`. Peti parametar su kreacijske zastavice. Ako je on 0 nit se starta odmah, a ako je `CREATE_SUSPENDED` nit je inicijalno suspendovana. Posljednji parametar predstavlja pointer na mjesto gdje će se smjestiti identifikator niti. Ova funkcija vraća rukovatelj niti koga koriste druge funkcije za rad s nitima

1.19. Kako se POSIX API-jem kreiraju niti?

Kreiranje niti se obavlja funkcijom `pthread_create`. Njen prvi parametar je pointer na varijablu koja će sadržati identifikator niti nakon njenog kreiranja. Drugi parametar je pointer na strukturu atributa niti kojim se može uticati na ponašanje niti, u većini slučajeva može se staviti `NULL`. Treći parametar pokazuje na funkciju u kojoj se nit izvršava, a četvrti je argument koji se prosljeđuje toj funkciji. Pozivalac nakon kreiranja niti može pozvati `pthread_join` funkciju da sačeka završetak date niti.

1.20. Objasnite kriterije algoritama za raspoređivanje: poštenost, skalabilnost, transparentnost.

Poštenost znači da svaki proces treba dobiti šansu da se izvršava i da procesi iste kategorije dobivaju približno jednako procesorsko vrijeme.

Skalabilnost se odnosi na prilagođenost algoritma različitim stepenima opterećenja, na primjer povećanje broja procesa da ne usporava drastično raspoređivač.

Transparentnost se odnosi na višeprocorske i distribuirane sisteme. Ovaj kriterij znači da za korisnika ne treba biti velike razlike između načina raspoređivanja na jednom ili drugom procesoru ili lokalnom i udaljenom računaru.

1.21. Šta su algoritmi sa istiskivanjem, a šta bez istiskivanja?

Kod algoritama raspoređivanja bez istiskivanja proces se izabere za izvršavanje i pusti se da se izvršava dok samog sebe ne blokira (zbog U/I zahtjeva ili zbog čekanja na drugi proces) ili dok on dobrovoljno oslobodi procesor

(sistemskim pozivom ili završetkom procesa). Takav način raspoređivanja je bio npr. u Windows 3.1 i u mnogim sistemima u realnom vremenu. Ovakvi raspoređivači su jednostavniji, ali loše napisani programi, na primjer oni koji su se zaglavili u petlji, mogu srušiti cijeli sistem.

Kod algoritma za raspoređivanje sa istiskivanjem, izabrani proces se izvršava fiksno vrijeme, ukoliko se u međuvremenu nije blokirao ili završio. Na kraju tog vremenskog intervala, proces se skida sa procesora i raspoređivač bira drugi proces za izvršenje, ako je raspoloživ. Linux i Windows NT imaju ovakve raspoređivače.

1.22. Objasnite algoritam raspoređivanja procesa FCFS.

Vjerovatno najjednostavniji od svih algoritama za raspoređivanje je nonpreemptive algoritam FCFS (engl. firstcome first-served). Kod ovog algoritma, procesi se pridružuju procesoru u redosljedu njihovih zahtjeva. U osnovi, postoji jedan red čekanja spremnih procesa. Kada se prvi proces kreira u sistemu, on odmah dobije procesor i dozvoljeno mu je da ga drži koliko hoće. Ako drugi procesi zatraže procesor, oni se smiještaju na kraj reda čekanja spremnih procesa. Kada se izvršni proces blokira, onda se na procesor dovodi prvi proces iz liste spremnih procesa. Kada blokirani proces postane spreman za izvršavanje onda se on smiješta u red čekanja spremnih procesa.

1.23. Šta je efekt konvoja i kada nastaje?

Efekt konvoja je nedostatak FCFS algoritma i koji negativno utiče na balansiranu upotrebu resursa. On se dešava ako se procesi različitih profila povremeno blokiraju

1.24. Objasnite algoritam raspoređivanja procesa SJF.

Vjerovatno najjednostavniji od svih algoritama za raspoređivanje je nonpreemptive algoritam FCFS (engl. firstcome first-served). Kod ovog algoritma, procesi se pridružuju procesoru u redosljedu njihovih zahtjeva. U osnovi, postoji jedan red čekanja spremnih procesa. Kada se prvi proces kreira u sistemu, on odmah dobije procesor i dozvoljeno mu je da ga drži koliko hoće. Ako drugi procesi zatraže procesor, oni se smiještaju na kraj reda čekanja spremnih procesa. Kada se izvršni proces blokira, onda se na procesor dovodi prvi proces iz liste spremnih procesa. Kada blokirani proces postane spreman za izvršavanje onda se on smiješta u red čekanja spremnih procesa.

1.25. Objasnite algoritam raspoređivanja procesa SRTN.

Preemptive verzija SJF algoritma predstavljena je algoritmom SRTN (engl. Shortest remaining time next). Kod ovog algoritma, raspoređivač uvijek izabira proces čije je preostalo vrijeme za izvršavanje najkraće. I ovdje, vrijeme izvršavanje treba biti poznato unaprijed. Kada novi proces dolazi u sistem, njegovo ukupno vrijeme se poredi sa vremenom preostalim za tekući proces. Ako novi proces treba manje vremena da završi izvođenje od tekućeg procesa na procesoru, tekući proces se skida sa procesora i novi proces započinje izvođenje.

1.26. Objasnite algoritam raspoređivanja procesa HRRN.

Algoritam Najbolji odnos odziva sljedeće (Highest response ratio next), HRRN raspoređivanje obavlja u isto vrijeme kada i SRTN algoritam, pri pokretanju, završetku procesa i blokiranju. Ali, ne bira se proces s najkraćim preostalim vremenom, nego proces s najvećim količnikom vremena provedenog u čekanju i preostalog vremena izvršenja. Jasno, ovaj algoritam nema najmanje srednje vrijeme čekanja, ali bolje ispunjava cilj poštenosti

1.27. Objasnite algoritam raspoređivanja procesa Round Robin.

Svakom procesu se pridružuje jedan vremenski interval, koji se naziva kvantum, u kome je dozvoljeno procesu da se izvršava. Ako se proces još izvršava na kraju kvantuma, proces se skida sa procesora i dovodi se novi proces na procesor. Ako se proces blokirao ili završio izvođenje prije nego što je kvantum istekao, vrši se izmjena konteksta na procesoru i novi proces može dobiti svoj kvantum.

1.28. Objasnite algoritam raspoređivanja procesa Kooperativno raspoređivanje.

Ako operativni sistem nije dizajniran da omogući prekidanje procesa usred njegovog izvršavanja, Round robin nije moguć. Otprogrami jezgra mogu pozivati ako se neki potprogram jezgra već izvršava. Da bi se omogućio višeprocetni rad, a zadržala podloga dizajnirana za jednoprocetni operativni sistem, definirana su pravila samim procesima da povremeno pozivaju neki sistemski poziv koji će obaviti izmjenu konteksta. Mana kooperativnih

raspoređivača je što se programi moraju pisati strogo uzimajući u obzir postojanje takvih raspoređivača. Nije npr. dopustivo da se proces izvršava bez povremenog poziva specijalnog sistemskog poziva koji obavlja raspoređivanje. Takav proces bi zaustavio izvršavanje svih ostalih procesa.

1.29. Koja je uloga kredita kod prioritetskog raspoređivanja sa vremenskim prilagođavanjem prioriteta?

U modifikovanoj verziji prioritetskog raspoređivanja procesima se dodjeljuje vrijednost koja se zove kredit. Kredit se inicijalno računa na bazi prioriteta procesa. Nakon svakog prekida sata kredit se smanjuje određenom formulom. Zavisno od dizajna algoritma može se proces izvršavati dok mu kredit ne dostigne vrijednost 0 ili dok ne padne ispod vrijednosti kredita koju ima neki drugi proces. U prvoj varijanti kredit ima više funkciju kvantuma, a u drugoj funkciju prioriteta. Kada svi krediti dođu do nule ponovo se računaju.

1.30. Objasnite algoritam raspoređivanja procesa Round Robin sa prioritetima.

Često je zgodno da se grupa procesa nalazi u klasi prioriteta i koristi prioritarno raspoređivanje među klasama, ali round-robin raspoređivanje u svakoj klasi. Prvo se uslužuje klasa sa najvećim prioritetom, zatim ona sa nižim itd. Prioriteti se moraju podešavati dinamički inače procesi nižeg prioriteta nikada neće doći na procesor.

1.31. Objasnite algoritam raspoređivanja Garantovano raspoređivanje.

Najprije se izračuna relativna količina vremena koja treba biti dodijeljena procesu. Na primjer, ako u sistemu ima n procesa, svaki proces bi mogao dobiti $\frac{1}{n}$ procesorskog vremena. Nazovimo vrijeme kada je izračunata ova relativna količina vremena za sve procese trenutkom obećanja. U svakom trenutku kada se poziva raspoređivač, izračuna se ukupno vrijeme proteklo od trenutka obećanja t_s , a za sve procese koliko je taj proces proveo u izvršavanju t_r . Za svaki proces se izračuna količnik $\frac{t_r}{t_s}$ i što je on manji, to je obećanje lošije ispunjeno. Stoga se za naredno izvršavanje bira proces s najmanjim ovakvim odnosom.

1.32. Objasnite algoritam raspoređivanja Ručno raspoređivanje.

Broj procesa koji postoje u ovakvim sistemima je mali i njihovo ponašanje je poznato unaprijed. Ako je vrijeme odziva poznato za sve procese, onda se mogu ručno napraviti statičke tabele koje kažu u koje vrijeme se pokreću koji procesi. Događaji na koje mora odgovoriti sistem u realnom vremenu mogu se podijeliti na periodične (koji se dešavaju u pravilnim intervalima) i neperiodične (koji se dešavaju nepredvidivo).

1.33. Objasnite algoritam raspoređivanja Lutrijsko raspoređivanje

Osnovna ideja je da se procesima daju tiketi za razne sistemske resurse. Kad god odluka o raspoređivanju treba da se napravi, lutrijski tiket se izabira slučajno i proces koji drži taj tiket dobija resurs.

1.34. Objasnite algoritam raspoređivanja najraniji rok prvo.

U ovom algoritmu, procesi koji se mogu rasporediti se raspoređuju tako da se prije izvršavaju oni procesi čiji rok završetka dolazi ranije. Algoritam omogućava da se izvrši veći broj procesa u odnosu na statički definirane prioritete. Ali pri velikom opterećenju, skup procesa čiji uslovi izvršenja neće biti zadovoljeni je teško predvidljiv, što za sisteme u realnom vremenu može biti kobno.

1.35. Objasnite algoritam raspoređivanja monotonom stopom.

Pri raspoređivanju se veći prioritet dodjeljuje onim procesima čija učestalost pojave je veća (tj. period je kraći). Npr., ako se događaji od senzora pozicije šalju svakih 10ms, a događaji od senzora temperature svakih 100 ms, uvijek će u raspoređivanju imati prednost obrada senzora pozicije.

1.36. Objasnite algoritme raspoređivanja Particionirano raspoređivanje i Globalno raspoređivanje.

U particioniranom raspoređivanju, proces zajedno sa svim nitima je trajno dodijeljen jednom procesoru od pokretanja do njegovog završetka. Namjenski kratkoročni red čekanja se održava za svaki procesor. Unutar reda čekanja za pojedini procesor koriste se algoritmi raspoređivanja namijenjeni jednom procesoru. Prednost ovog pristupa je da može biti manje posla oko raspoređivanja, jer je zadatak procesoru dodijeljen jednom i za sve vrijeme.

1.37. Koji algoritmi se koriste u Podjeli opterećenja?

FCFS. Svaki proces koji se izvršava kada dolazi u red čekanja u njega stavi sve svoje niti. Kako koji procesor postaje slobodan, dodjeljuje mu se naredna slobodna nit. Dodijeljena nit se nastavlja na datom procesoru do svog završetka.

Prvo najmanji broj niti: Prioritet se daje procesima koji imaju manji broj neraspoređenih niti kako bi se brže pokrenuli takvi procesi. Dodijeljena nit se nastavlja na datom procesoru do svog završetka.

Prvo najmanji broj niti sa istiskivanjem: Prioritet se daje procesima koji imaju manji broj neraspoređenih niti, ali se pri tome prekida neki proces koji je aktivirao više niti.

1.38. Objasnite algoritam raspoređivanja društveno raspoređivanje.

Društveno raspoređivanje (engl. gang scheduling) je način istovremenog raspoređivanja svih niti koje pripadaju istom procesu. Razlog za ovo je manje izmjena konteksta kada je potrebna saradnja između niti i što veći stepen paralelizma unutar pojedinačnog procesa. Pokušaće se rasporediti M procesa na N procesora, a svaki proces ima N ili manje niti.

1.39. Koje uslove treba zadovoljavati dobro rješenje kritične sekcije?

Dobro rješenje problema kritične sekcije treba da zadovoljava uslove:

- U kritičnoj sekciji se smije izvršavati najviše jedan proces ili nit.
- Osiguranje uzajamnog isključivanja treba da radi i za jednoprocesorske i višeprocesorske sisteme i ne smije se zasnivati na vremenu boravka u kritičnoj sekciji,
- Proces koji se izvršava izvan kritične sekcije ne smije da blokira druge procese
- Proces ne smije beskonačno čekati da bi ušao u kritičnu sekciju
- Proces se ne smije ni beskonačno zadržati u kritičnoj sekciji kako ne bi blokirao druge procese.

1.40. Koji su nedostaci rješenja kritične sekcije dijeljenom varijablom?

Prva ideja je da se uvede jedna zajednička varijabla koja je u početku ima vrijednost 0. Kada proces želi ući u kritičnu sekciju on testira tu varijablu. Ako je ona jednaka 0, proces je postavi na vrijednost 1 i nastavlja sa izvršavanjem kritične sekcije. Ako je dijeljena varijabla već imala vrijednost 1, proces mora da čeka sve dok ona ne postane jednaka 0. Efektivno, vrijednost 0 znači da nijedan proces nije u kritičnoj sekciji, a vrijednost 1 znači da ima proces u kritičnoj sekciji.

1.41. Koji su nedostaci rješenja kritične sekcije zabranom prekida?

Neka rješenja se zasnivaju na specifičnostima instrukcija koje centralni procesor pruža, pa se mogu realizovati samo na nekim računarskim sistemima. Glavni razlog stanja natjecanja je izmjena konteksta procesa ili niti pod uticajem prekida sata realnog vremena. Ako se taj prekid zabrani, dobije se jednostavno rješenje problema kritične sekcije. Na Intelovim procesorima, prekidi se zabranjuju naredbom CLI, a dopuštaju naredbom STI.

1.42. Šta je to instrukcija TSL i kako se njom rješava kritična sekcija?

Mnogi računari, posebno oni koji su projektovani sa višestrukim procesorima, imaju instrukciju tipa TSL RX, LOCK, a znači provjeri i postavi (engl. Test and Set Lock) koja radi na sljedeći način: čita sadržaj memorijske riječi LOCK u registar RX i onda smiješta vrijednost različitu od 0 u memorijsku lokaciju LOCK. Operacija čitanja riječi i pisanje u nju su nedjeljive, kažemo atomske, tako da drugi procesi ne mogu pristupiti varijabli sve dok se instrukcija ne završi. Procesor izvršava TSL instrukciju i zaključava memorijsku sabirnicu da bi zabranio drugim procesorima da pristupe memoriji dok se instrukcija ne završi.

Kako se rj k s

1.43. Koje operacije imaju semafori sa zaposlenim čekanjem?

Uzajamno isključivanje je specijalni slučaj limitiranja broja procesa koji mogu ući u kritičnu sekciju pa je E. W. Dijkstra (1965) uveo specijalne cjelobrojne varijable koje se zovu semafori i koje između ostalog mogu da čuvaju taj broj. Nad semaforima su pored inicijalizacije dopuštene samo dvije operacije: wait i signal.

Prva operacija (u literaturi poznata kao P, wait ili down) čeka u petlji ako semafor nema pozitivnu vrijednost, a zatim umanjuje vrijednost semafora za 1. Ona se poziva prije ulaska u kritičnu sekciju. Druga operacija (V, signal ili up) uvećava semafor za 1 i poziva se nakon kritične sekcije.

1.44. Šta su to semafori sa blokiranjem i deblokiranjem?

Za razliku od semafora sa zaposlenim čekanjem, semafori sa blokiranjem procesa mogu imati i negativne vrijednosti u polju value. Pozitivna vrijednost semafora znači da je semafor otvoren za još toliko poziva wait operacije. Vrijednost 0 znači da je semafor u zatvorenom stanju, ali ni jedan proces nije blokiran. Negativna vrijednost semafora $-x$ znači da je x procesa trenutno u blokiranom stanju.

1.45. Kako se kreiraju, inkrementiraju i oslobađaju semafori u Windows?

Semafori se kreiraju pomoću funkcije `CreateSemaphore(lpSemaphoreAttributes, lInitialCount, lMaximumCount, lpName)`, a funkcija `ReleaseSemaphore(hSemaphore, lReleaseCount, lpPreviousCount)`; inkrementira semafor.

1.46. Kako se kreiraju, inkrementiraju i oslobađaju semafori u Posix API ?

Funkcija `sem_open(pcName, oFlag, iNmode, iValue)` kreira novi semafor, a funkcija `sem_post(pSem)` inkrementira

1.47. Šta su to muteksi?

Kada semaforska mogućnost za brojanje nije potrebna, koristi se pojednostavljena verzija semafora i on se naziva mutex.

1.48. Šta su to monitori?

Monitor predstavlja skup procedura, promjenljivih i struktura podataka koje su sve grupisane zajedno u posebnu vrstu modula. Procesi mogu zvati procedure u monitoru kad god žele, ali ne mogu direktno pristupiti internim strukturama podataka iz procedura koje se nalaze izvan monitora.

1.49. Šta su to barijere?

Barijere predstavljaju mehanizam za sinhronizaciju grupe procesa. Neke aplikacije su podijeljene u faze i imaju pravilo da se nijedan proces ne smije izvršavati u sljedećoj fazi ako svi procesi nisu spremni da uđu u tu fazu, te se ovakvo ponašanje procesa unutar aplikacije rješava barijerama.

1.50. Problem proizvođača i potrošača.

Kao primjer kako se ove funkcije koriste u praksi, razmotrimo problem proizvođač – potrošač, poznat i pod nazivom problem ograničenog bafera. Dva procesa dijele zajednički bafer fiksne veličine. Jedan od njih, proizvođač, stavlja informacije u bafer, a drugi ih uzima (kažemo koristi). Problemi se javljaju kada proizvođač želi da stavi novi element u bafer a bafer je već pun. Za proizvođača je rješenje da se blokira, a da se deblokira kada potrošač ukloni iz bafera jedan ili više elemenata. Slično, ako potrošač želi da ukloni jedan element iz bafera i vidi da je bafer prazan, treba da se blokira sve dok proizvođač ne stavi element u bafer i deblokira ga.

1.51. Problem 5 filozofa.

Problem filozofa koji jedu je koristan za modeliranje procesa koji se natječu za ekskluzivan pristup ograničenom broju resursa, takvih kao što su U/I uređaji.

Pet filozofa sjede za okruglim stolom. Svaki filozof ima tanjir sa špagetama. Za jedenje špageta potrebno je koristiti dvije viljuške. Između dva tanjira se nalazi po jedna viljuška. Rješenje je dato u sljedećem dijelu i omogućuje maksimalni paralelizam među procesima. Koristi jedan niz, stanje da vodi evidenciju da li filozof jede, razmišlja ili je gladan (kada pokušava da dobije dvije viljuške). Filozof se može premjestiti u stanje "jede" samo ako nijedan susjed ne jede. Susjedi filozofa su definisani sa makro vrijednostima LEFT i RIGHT. Drugim riječima, ako je i jednak 2 onda je LEFT jednako 1 a RIGHT jednako 3.

1.52. Problem čitača i pisaca.

Problem filozofa koji jedu je koristan za modeliranje procesa koji se natječu za ekskluzivan pristup ograničenom broju resursa, takvih kao što su U/I uređaji. Drugi, poznati, problem predstavlja problem čitača i pisaca kojim se modelira pristup bazama podataka. Zamislamo, na primjer, jedan sistem za rezervaciju karata, sa mnogo procesa koji se natječu u želji da pristupe sistemu čitajući ili pišući svoje zahtjeve. Prihvatljivo je da imamo više procesa koji čitaju baze podataka u isto vrijeme, ali ako neki od procesa želi da ažurira (čitaj: piše) bazu podataka nijedan drugi proces ne smije imati pristup bazi podataka, čak ni "čitači" (proces koji čitaju bazu podataka). Kako da se izgradi

program za čitače i pisce u bazu podataka. U jednom rješenju, prvi čitač koji dobije pristup bazi podataka izvršava operaciju down() na semaforu db. Sljedeći čitači treba samo da povećaju brojač rc za jedan. Kada čitači napuštaju bazu podataka, oni smanjuju brojač za jedan i posljednji čitač izvršava up() proceduru koja omogućuje blokiranim štampačima, ako ih ima, da dobiju pristup bazi podataka.

1.53. Prednost i mana realizacije međuprocenke komunikacije dijeljenim datotekama.

U operativnim sistemima sa grafičkim korisničkim sučeljem koristi se međuprocenka komunikacija koja je inicirana od strane korisnika. Korisnik može radeći u jednoj aplikaciji npr. Microsoft Office odabrati pojedine dijelove teksta ili slika, pritisnuti taster Ctrl+C ili opcijom Copy iz menija, nakon čega dati podaci mogu biti dostupni drugim aplikacijama.

1.54. Šta su to cijevi i koje vrste cijevi postoje?

Cijevi (pipe) predstavljaju jednosmjerni komunikacioni kanal između dva procesa u UNIX sistemima. Cijev ima dva kraja: jedan za pisanje i jedan za čitanje. Koristeći cijevi, jedan proces može puniti bajte na kraju za pisanje, a neki drugi proces može preuzimati ovaj tok bajta na kraju za čitanje. Pisanje podataka u cijevi i njihovo čitanje se obavlja slično pisanju i čitanju datoteka.

UNIX poznaje dvije vrste cijevi, neimenovane i imenovane

1.55. Objasnite komunikaciju procesa porukama sa i bez mailboxa.

Poruka ostaje u redu čekanja dok ne bude pročitana, nakon čega se briše iz njega. Svaki primalac može imati svoj red poruka koje su mu namijenjene, a može postojati i jedan zajednički red poruka koji se zove mailbox. U slučaju komunikacije sa zajedničkim redom poruka u pozivima send, receive i tryreceive se umjesto primaoca ili pošiljaoca navodi oznaka mailbox-a, a stvarni primalac i pošiljalac su smješteni u tijelo poruke..

1.56. Šta su Clipboard i DDE u Windows?

Korisnik može radeći u jednoj aplikaciji npr. Microsoft Office odabrati pojedine dijelove teksta ili slika, pritisnuti taster Ctrl+C ili opcijom Copy iz menija, nakon čega dati podaci mogu biti dostupni drugim aplikacijama. Podaci se smještaju u posebno zajedničko područje memorije koje se zove odlagalište ili clipboard. Clipboard podržava više različitih formata podataka: bit-mapirane slike, upute za iscrtavanje likova (metadatoteke), tekst i Unicode tekst. DDE (Dynamic Data Exchange) predstavlja mehanizam među-procesne komunikacije slanjem onih tipova podataka koji se mogu naći unutar clipboard koristeći sistem Windows poruka. Poruke tipa WM_DDE_DATA koje se šalju između aplikacija u svom parametru mogu pokazivati na podatke koji se šalju pozivom funkcije SendMessage. Pored ovog tipa poruka, postoje i tipovi poruka za uspostavljanje DDE komunikacije, izvršavanje zahtjeva na serveru, prekid komunikacije i potvrdu prijema podataka.

1.57. Šta su to signali u Unix sistemima?

Signal je neka vrsta prekida programa u Unix sistemima. Oni predstavljaju vrlo rudimentarnu među-procesnu komunikaciju. Signali se dešavaju kada se pritisne CTRL+C za prekid programa i tada se šalje signal SIGINT ka programu. Uobičajena akcija pri prijemu signala je završetak izvršavanja programa ili snimanje slike memorije u trenutku završetka izvršavanja i završetak izvršavanja. Ove uobičajene akcije se mogu predefinisati.

2.1. Koji su koraci u upotrebi resursa u operativnim sistemima?

Tipično, pristup resursu sastoji se iz tri koraka:

1. Zahtjev za resursom se postiže odgovarajućim sistemskim pozivom koji zavisi od vrste resursa. Na primjer, zahtjev za memorijom u Windows sistemu se obavlja pozivom VirtualAlloc(), a zahtjev za datotekom u Unix-u se obavlja pozivom open(). Ako je resurs zauzet, proces koji ga zahtijeva mora čekati dok ne postane slobodan. Ovo čekanje može biti zaposleno čekanje, ako je sistemski poziv vratio grešku, a može se i uspavati proces dok resurs ne

postane slobodan. Ako to sam poziv ne radi, mogu se i eksplicitno pozvati semafori ili drugi mehanizmi zaključavanja resursa.

2. Korištenje resursa (npr. upis podataka u dodijeljenu memoriju) postaje moguće kada je zahtjev bio uspješan.

3. Oslobođanje resursa je potrebno pozvati kada se resurs više ne koristi.

2.2. Šta je to potpuni zastoј?

Potpuni zastoј se mođe definirati na sljedeći naćin: Skup procesa se nalazi u stanju potpunog zastoја ako svaki proces u skupu ćeka na neki događaj koji mođe proizvesti jedino neki proces iz tog skupa.

2.3. Uslovi nastajanja potpunog zastoја.

1. Uslov uzajamnog iskljućivanja znaći da svaki resurs ili je dodijeljen taćno jednom procesu ili je dostupan. Ako neki proces traži resurs koji je već dodijeljen drugom procesu, proces tražilac mora ćekati dok se resurs ne oslobodi.

2. Uslov neoduzivosti resursa znaći da resursi koji su prethodno dodijeljeni ne mogu biti oduzeti procesu ukoliko ih proces samostalno ne oslobodi.

3. Uslov stanja prisvajanja resursa i ćekanja na drugi znaći da proces, koji trenutno zauzima ranije dodijeljene resurse, mođe zatražiti i nove resurse.

4. Uslov kružnog ćekanja podrazumijeva da dva ili više procesa, ćekaju na neki resurs koji drži sljedeći ćlan lanca. Na primjer: proces P1 zauzeo štampać i želi skener. Skener je već dodijeljen procesu P2 a proces P2 traži slog u datoteci. Slog u datoteci je dodijeljen procesu P3 a proces P3 traži štampać.

Sva ćetiri uslova moraju biti prisutna da bi došlo do potpunog zastoја. Ako bar jedan nije ispunjen, to nije potpuni zastoј.

2.4. Kako se modelira potpuni zastoј grafom dodjele resursa?

Tri od spomenuta ćetiri uslova se mogu grafićki modelirati pomoću Holtovih grafova alokacije resursa. Ovi grafovi se sastoје od skupa ćvorova i skupa strelica. Postoje dvije vrste ćvorova: procesi predstavljeni krugovima i resursi predstavljeni pravugaonicima. Strelica od ćvora koji predstavlja resurs prema ćvoru koji predstavlja proces znaći da je proces dobio resurs koji je tražio. Strelica od procesa ka resursu znaći da proces ćeka da dobije zahtijevani resurs.

2.5. Nojev algoritam.

Najjednostavniji algoritam je nojev algoritam. Posmatrajući nojeve kako kopaju rupe u pijesku za jaja neki ljudi su zakljućili da noj prosto ignoriše opasnosti savane, zavlaućenjem glave u pijesak i praveći se da opasnost ne postoji. Tako se mođe gledati i na problem potpunog zastoја. Mišljenja o ovom "algoritmu" su podijeljena. Matematićari uglavnom smatraju da potpuni zastoјi trebaju biti sprijećeni bez obzira na cijenu, dok inženjeri porede koliko se ćesto pojavljuje potpuni zastoј a koliko ćesto sistem ne funkcioniše iz drugih razloga. Ako se potpuni zastoј uslijed zauzimanja resursa javi rijetko, a drugi hardverski ili softverski problemi se javljaju ćešće, ulaganja u otklanjanje ili sprjećavanje potpunog zastoја se ne isplate. Ćak se mogu smatrati i štetnim, jer stalne provjere da li je došlo ili bi

moglo doći do potpunog zastoја troše više hardverskih resursa. Stoga Unix i Windows do verzije XP, ignorišu ovaj problem, jer većini korisnika povremeno pojavljivanje potpunog zastoја neće smetati. Ali u sistemima realnog vremena sama pomisao da se softverski sistem mođe zagušiti je alarmantna. Jedna od varijanti je i ne brinuti o potpunom zastoју u normalnom radu, jer algoritmi njegove detekcije troše previše resursa, ali prilikom razvoja

operativnog sistema posvetiti pažnju i ovom problemu za kritične dijelove operativnog sistema. Tako na primjer, postoji program za verifikaciju drajvera za Windows koji se pokrene prilikom njihovih testiranja i upozorava na uzroke potpunog zastoja.

2.6. Algoritam za detekciju potpunog zastoja.

Umjesto ignorisanja, drugi pristup je prepoznavanje, a zatim otklanjanje potpunog zastoja. U ovom pristupu operativni sistem ne pokušava spriječiti pojavu potpunog zastoja. U ovom slučaju, potpuni zastoji su mogući, a sistem ih pokušava prepoznati i otkloniti. Stoga se mora osigurati algoritam detekcije (koji ispituje da li je došlo do zastoja) i algoritam za oporavak ako je došlo do potpunog zastoja.

Imamo različite vrste algoritama za detekciju:

1. Detekcija potpunog zastoja mjerenjem vremena:

Jedan od najjednostavnijih i najčešće korištenih načina detekcije potpunog zastoja je uočiti da su neki procesi previše u stanju čekanja. Na primjer, modifikovati rutinu za čekanje na semafor tako da nakon isteka izvjesnog vremena izađe sa greškom uz pretpostavku da je u pitanju potpuni zastoj. Tako na primjer, Windows API funkcija `WaitForSingleObject()` ima kao drugi parametar maksimalan broj milisekundi koje se provedu u čekanju. Kada to vrijeme istekne, funkcija vraća vrijednost `WAIT_TIMEOUT`. Mana ovog načina detekcije potpunog zastoja je što će i slučajevi koji nisu potpuni zastoj biti detektovani kao takvi, ukoliko je korištenje zauzetog resursa duže od vremena koje je navedeno kao maksimalno dopušteno vrijeme čekanja procesa na dati resurs. Ovaj pristup ne dokazuje koji skup procesa predstavlja potpuni zastoj.

2. Detekcija potpunog zastoja grafom alokacije resursa:

Ako graf alokacije resursa sadrži jednu ili više zatvorenih kontura, sistem se vjerovatno nalazi u stanju potpunog zastoja.

Na papiru je lako prepoznati zatvorenu konturu, ali kako to uraditi u jezgru operativnog sistema? Algoritam koji to radi kreće od jednog izabranog čvora. Graf se tada posmatra kao stablo sa korijenom u tom čvoru uzimajući u obzir samo izlazne veze iz čvora i primjenjuje način prolaska kroz stablo algoritmom prvo dubinska pretraga. Ovaj algoritam u pseudo kodu izgleda ovako:

procedure IspitajČvor(Graf G, Čvor v):

Označi Čvor v kao ispitan

Za svaku izlaznu vezu iz čvora v prema čvoru w

Ako čvor w nije označen kao ispitan Rekursivno zovi IspitajČvor(G, w)

Ako je čvor w označen kao ispitan, to predstavlja da postoji kontura

Ako se ova rutina pozvana za izabrani čvor vrati bez indikacije da postoji kontura, i ako ovaj test prođe za sve početne čvorove, to znači da sistem nije u potpunom zastoju.

3. Detekcija potpunog zastoja matricnom metodom:

Kada postoji više kopija jednog resursa, koristi se algoritam baziran na matricama za prepoznavanje potpunog zastoja. Neka postoje procesi P_1 do P_n . Postoji m vrsta resursa. U svakom trenutku su neki resursi zauzeti. Vektor resursa R sadrži za svaki resurs i broj slobodnih kopija. Matrica alokacije A je dimenzija $n \times m$ kao i matrica novih zahtjeva N . Element A_{ij} matrice A sadrži broj resursa klase j koje zauzima proces P_i . Analogno je N_{ij} broj resursa tipa j koje je proces i zahtijevao, ali mu još nisu dodijeljeni.

Definišimo poređenje vektora (relaciju $\leq$) tako da važi $A \leq B$, ako je svaki element vektora A , manji ili jednak od elementa vektora B na odgovarajućoj poziciji. Na početku algoritma su svi procesi neoznačeni. Kada se jedan proces označi, to znači da nije uključen u potpuni zastoj i može nastaviti s radom. Po završetku algoritma ostaju neoznačeni procesi koji učestvuju u nekom potpunom zastoju. Algoritam za detekciju potpunog zastoja glasi:

1. Označi sve procese P_i čiji je i -ti red matrice A jednak nula-vektoru, jer taj proces ne koristi resurse.
2. Nađi bar jedan neoznačeni proces P_i , kod koga važi za red matrice $N_i \leq R$.
3. Ako takav proces postoji, saberi red matrice A_i sa R , i rezultat dodijeli vektoru R . označi proces P_i , pa idi na korak 1.
4. Kada nema takvog procesa završi algoritam. Sistem nije u potpunom zastoju ako su svi procesi označeni.

2.7. Načini oporavka od potpunog zastoja.

Rješenje da se resurs oduzme od procesa i dodijeli drugom procesu je izvodivo u samo nekim rijetkim situacijama. Za primjer, uzeće se situacija da je proces 1 zauzeo jedinicu trake A , preuzeo njenu strukturu u internu memoriju, i zatražio jedinicu trake B . Jedinicu trake B već drži proces 2, a još želi jedinicu trake A . Oduzme li se jedinica trake A od procesa 1, preuzeta struktura sadržaja trake je nevažna, jer će je proces 2 promijeniti nakon toga. Stoga se ne može garantovati uspješan nastavak rada procesa 1. Ovaj pristup je pouzdan samo u slučajevima kada je tok izvođenja programa u potpunosti poznat (npr. batch sistemi čiji programi uvijek počinju alokacijom svih resursa koje će koristiti i završavaju njihovim oslobađanjem).

Puno jednostavnije rješenje je prekidanje procesa. Ovo prekidanje može raditi administrator sistema, kada on donosi odluku koji proces treba prekinuti, npr. Unix naredbom `kill`. Nakon ovoga zauzeti resursi procesa se oslobađaju i dodjeljuju drugim procesima koji su čekali na njih. Alternativno, odluku o izboru procesa koji će se prekinuti može donijeti i operativni sistem, na bazi kriterija kao što su prioritet procesa, vrsta zauzetog resursa, proteklo vrijeme itd. Mana pristupa je gubitak odrađenog posla prekinutog procesa. Ukoliko prekid jednog procesa nije doveo do oslobađanja njegovih resursa ili ako je nastalo više ciklusa čekanja na resurse, a oslobođen je samo jedan, treba prekinuti više procesa koji su u ciklusu čekanja na resurse ili čak sve njih.

Restart sistema ili podsistema se preporučuje kao rješenje potpunog zastoja u složenijim slučajevima. Primjeri su preporuke da se Java virtualna mašina ponovo pokrene ako višenitne aplikacije uđu u potpuni zastoj ili u distribuiranim sistemima da se zaglavljani čvorovi ponovo pokrenu. Slično, ako se potpuni zastoj desio između drajvera u jezgri, također je restart sistema jedino rješenje.

Problem gubitka dosadašnjeg rada koji se javlja u rješenjima prekidanja procesa ili ponovnog pokretanja sistema se može djelomično ublažiti metodom oporavka snimanjem kontrolnih tačaka. U pojedinim trenucima se čuva stanje cijelog sistema, na primjer snima se memorija svih procesa na disk. Kada se desi potpuni zastoj, procesi se vraćaju u stanje snimljeno na onoj kontrolnoj tački prije nego su resursi zauzeti i nastavljaju, pa je izgubljen samo dio izvršavanja procesa. Pristup se obično ne koristi u operativnim sistemima, jer je snimanje cijele memorije sporo, a puna kontrolna tačka treba snimiti ne samo sadržaj memorije, nego i sadržaj diska, jer se i on mijenja u toku rada

procesa. Ali u oblasti sistema za upravljanje bazama podataka, kod kojih se potpuni zastoji uslijed zaključavanja slogova često javljaju, ovaj pristup je vrlo popularan. U ovim sistemima se podaci baze podataka čuvaju u dva dijela: datotekama s podacima i transakcijskim logovima. Kontrolnu tačku predstavlja početak transakcije. Kada se zapisuju podaci, najprije se smještaju u transakcijske logove, i tek kada su sasvim potvrđeni prebacuju se u datoteke s podacima. Ako se desio zastoj, tada se gube samo podaci koji su u logu.

2.8. Pojam sigurnog stanja.

Stanje u kome postoji takav redoslijed dodjele resursa koji neće dovesti do potpunog zastoja se zove sigurno stanje. Ako takav redoslijed ne postoji stanje nije sigurno.

2.9. Bankarov algoritam za izbjegavanje potpunog zastoja za više resursa.

Bankarov algoritam (engl. Banker's Algorithm) predstavlja algoritam za izbjegavanje potpunog zastoja. Ovaj algoritam je sličan algoritmu za detekciju potpunog zastoja, ali se izvršava u različitom trenutku. Bankarov algoritam, u verziji kada postoji više vrsta resursa i više kopija jednog resursa, baziran je na matricama za prepoznavanje potencijalnog potpunog zastoja među procesima P_1 do P_n . Neka postoje procesi P_1 do P_n . Postoji m vrsta resursa. U svakom trenutku su neki resursi zauzeti. Vektor resursa R sadrži za svaki resurs i broj slobodnih kopija R_i . Matrica alokacije A je dimenzija $n \times m$ kao i matrica novih zahtjeva N . Element A_{ij} matrice A sadrži broj resursa tipa j koje zauzima proces P_i . Analogno je N_{ij} broj resursa tipa j koje je proces i zahtijevao, ali mu još nisu dodijeljeni.

Definišimo poređenje vektora (relaciju $\leq$) tako da važi $A \leq B$, ako je svaki element vektora A , manji ili jednak od elementa vektora B na odgovarajućoj poziciji. Na početku algoritma su svi procesi neoznačeni. Algoritam detekcije sigurnog stanja izgleda ovako:

1. Nađi bar jedan neoznačeni proces P_i , kod koga važi za red matrice $N_i \leq R$.
2. Ako takav proces postoji, saberi red matrice A_i sa R , i rezultat dodijeli vektoru R . označi proces P_i , pa idi na korak 1.
3. Kada nema takvog procesa završi algoritam.

Ako su na kraju svi procesi označeni, sistem je u sigurnom stanju i postoji bar jedan redoslijed izvršavanja procesa i dodjele resursa koji ne vodi potpunom zastoju. Ali ako ima neoznačenih procesa sistem nije u sigurnom stanju. Ovaj algoritam se koristi samo u veoma malom broju sistema jer je beskoristan u praksi za sisteme gdje broj procesa nije konstantan, npr. ako se oni naknadno pokreću. Osim toga rijetko se unaprijed zna koliko će resursa biti potrebno procesu.

2.10. Kako se mogu sprječiti potpuni zastoji izbjegavanjem uzajamnog isključivanja?

Mnogi operativni sistemi za pojedine resurse koriste specijalizovane procese koji čine da uslov uzajamnog isključivanja bude neispunjen. Ako je neki proces zauzeo resurs prije svih ostalih i zauzeo ga u cijelosti samo za sebe, a ne traži druge resurse koje bi neki procesi još tražili, taj proces neće biti uzrok potpunog zastoja. Najbolji primjer za to je printer spooler (lpd u Unix-u, spoolsv Windows-u). To je jedini program koji koristi printer i on brine o redoslijedu štampanja dokumenata. Nijedan drugi program ne može zauzeti printer i stoga printer neće uticati na potpuni zastoj. Slično ovome, sistemi za upravljanje bazama podataka su jedini programi koji kao resurs koriste datoteke u kojima se podaci čuvaju. Ali, specijalizovani program ne može se koristiti na primjer za dijeljene varijable između dva procesa ili niti.

2.11. Kako se mogu sprječiti potpuni zastoji izbjegavanjem prisvajanja i čekanja?

Ako možemo spriječiti da proces čeka na resurse dok druge resurse drži zauzetim možemo spriječiti potpuni zastoj, kroz sprječavanje stanja prisvajanja

2.12. Kako se mogu sprječiti potpuni zastoji izbjegavanjem kružnog čekanja?

Ako ima više kopija resursa iste vrste (i da je procesu svejedno koja će mu biti dodijeljena) tada pojava kružnog čekanja ne znači nužno potpuni zastoj.

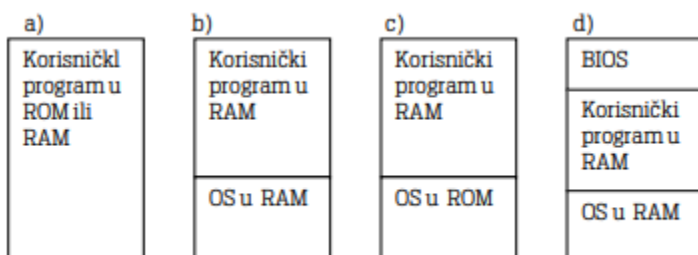
2.13. Šta su to primarna, sekundarna i tercijarna memorija?

Memorija kojoj se pristupa bez korištenja U/I kontrolera direktno preko adresiranja memorijskih lokacija ili podataka u samom procesoru se zove primarna memorija

Sekundarna memorija zahtijeva upotrebu U/I kontrolera, ali je i dalje dostupna bez dodatne ljudske ili mašinske intervencije. Tipični predstavnik ovakve memorije je tvrdi disk

Tercijarna memorija nije stalno spojena na računare. Riječ je o jedinicama trake od kojih svaka ima veliki kapacitet (npr. više terabajta) i čuvaju rijetko korištene podatke, npr. stare snimke video zapisa

2.12. Koje su četiri tipične konfiguracije memorije u jednoprocenim sistemima?



Slika 47 Konfiguracije memorije kod jednoprocenih sistema

2.13. Šta je to swapping?

U slučaju kada nemamo dovoljno interne memorije, može se koristiti tehnika kao što je swapping. Swapping (prebacivanje procesa) – ako nemamo dovoljno mjesta u operativnoj memoriji za smještanje svih spremnih procesa, neki se izbacuju na disk. Kada je potrebno, cijeli spremni procesi iz interne memorije se prebacuju na disk, odnosno spremni procesi sa diska se prebacuju u internu memoriju.

2.14. Koji su pristupi za određivanje listi čekanja kod multiprogramiranja s fiksnim particijama?

Kod ovog pristupa programi mogu biti unaprijed pripremljeni da se izvršavaju na adresama koje odgovaraju određenoj particiji i veličini date particije.

2.15. Kako je realizovana statička relokacija?

Slučaj višeprocenih sistema sa fiksnim particijama sa jedinstvenim redom čekanja otvara novo pitanje. Neka program sa diska treba učitati u memorijsku particiju koja mu je dodijeljena. Program je dizajniran da se izvršava u particiji koja počinje od adrese 1000h i na određenom mjestu pristupa lokaciji 123Ah npr. instrukcijom LD A,&123A na Z80 procesoru. Ali, kada je proces nakon čekanja u redu na particije došao da se izvršava, ispostavilo se da je njegova prirodna particija zauzeta, ali je slobodna particija koja počinje od adrese 3000h. Program punilac (loader) sada mora da ima tabelu sa svim mjestima u programu koje treba promijeniti da bi se program izvršavao u drugoj particiji. Sva takva mjesta treba sabrati sa razlikom između početnih adresa dodijeljene i preferirane particije, pa bi u datom primjeru instrukcija postala LD A,&323A. Ovaj postupak se zove statička relokacija, može se obaviti samo jednom, isključivo pri pokretanju procesa (ne i kada bi još bio potreban, npr. prilikom vraćanja procesa koji je swapping-om snimljen na disk) i razmjerno je spor, ali nekad jedini moguć.

2.16. Koja je uloga relokacionog i limit registra kod dinamičke relokacije?

Najjednostavniji način mapiranja između logičkog i fizičkog adresnog prostora je upotrebom relokacionog registra, koji određuje početnu fizičku adresu procesa.

Uz relokacijski registar može se dodati i limit registar koji predstavlja najveću dopuštenu fizičku adresu kojoj se može pristupiti. Na ovaj način se rješava i problem zaštite između procesa.

2.17. Relokacija s relokacionim registrom.

Mapiranje virtualnog adresnog prostora u fizički obavlja hardverski uređaj koji se zove MMU4 jedinica za upravljanje memorijom. Najjednostavnija šema mapiranja je šema koja koristi relokacioni registar. Relokacioni registar definiše adresu fizičkog početka programa. Svaka logička adresa koju generiše program sabira se sa vrijednošću relokacionog registra i tako dobijamo fizičke adrese. Pri tome korisnički program počinje od nulte adrese.

2.18. U čemu je razlika alokacije memorije s promjenjivim particijama u odnosu na fiksne?

Kod promjenljivih particija ne zna se unaprijed ni broj, ni veličina, ni lokacija particija, nego se ti parametri određuju dinamički prilikom pokretanja i izvršenja procesa. Particije se kreiraju i oslobađaju po potrebi.

2.19. Koje slučajeve razlikujemo kod zauzimanja i oslobađanja memorije s promjenjivim particijama ako se koriste ulančane liste?

6 slučajeva.

a) Zahtijevana veličina particije koja se zauzima je jednaka veličini particije koja je bila slobodna (u ovom primjeru 5000 bajta) i koja je određena da se dodijeli. U tom slučaju dovoljno je promijeniti polje zauzetosti odgovarajućeg sloga ulančane liste.

b) Zahtijevana veličina particije koja se zauzima je manja od veličine particije koja je bila slobodna (npr. 3000 bajta) i koja je određena da se dodijeli. Tada se stanje njenog sloga promijeni na zauzeto, dužina particije skрати na zahtijevanu dužinu i ubaci novi član lanca koji predstavlja slobodnu particiju dužine koja je razlika između ranije dužine bivše slobodne particije i zahtijevane dužine.

c) Oslobađa se particija koja je locirana između dvije zauzete particije. Tada je dovoljno njeno stanje zauzetosti u listi promijeniti.

- d) Oslobađa se particija koja je locirana iza jedne slobodne, a ispred jedne zauzete particije. Jedan element liste se briše i spaja nova slobodna particija sa dosadašnjom.
- e) Oslobađa se particija koja je locirana iza jedne zauzete, a ispred jedne slobodne particije. Jedan element liste se briše i spaja nova slobodna particija sa dosadašnjom.
- f) Oslobađa se particija koja je između dvije slobodne particije. Brišu se dva elementa ulančane liste i formira jedna slobodna particija od ovih particij

2.20. Šta su kontrolni blokovi memorije?

Ako se ne želi pripremati poseban prostor u memoriji za smještanje liste slobodnih blokova, može se koristiti i drugi, ekvivalentan pristup preko kontrolnih blokova memorije (MCB).

2.21. Šta su to bitmape za evidentiranje zauzetih blokova?

Kod upravljanja memorijom pomoću bit mape, memorija se dijeli na komade iste veličine, na primjer 256 bajta. Svakom komadu odgovara jedan bit u bit mapi pri čemu 0 znači da je komad slobodan a 1 da je komad zauzet (Slika 52). Tako dolazimo do niza nula i jedinica (bitova) koji se zove bit mapa memorije.

2.22. Objasnite algoritme first fit, best fit, next fit, worst fit.

First fit

Kod prvog uklapanja (first fit) alokator kreće od početka liste slobodnih blokova ili bitmape i traži prvu slobodnu particiju u listi, odnosno prvu sekvencu nula u bitmapi dovoljne dužine. Ta particija se dodjeljuje procesu, a ako je proces zahtijevao manje memorije nego što je veličina particije, formira se i odgovarajuća nezauzeta particija, šupljina. Ovo je dosta brz algoritam, ali vremenom na početku liste nastaje puno malih šupljina, pa pretraživanje liste postaje teže.

Next fit

Algoritam Sljedeće uklapanje (next fit) pretražuje listu slobodnih blokova na sličan način kao prvo uklapanje, samo što ne započinje svaki put pretraživanje od početka liste, već od mjesta gdje je posljednji put dodijelio particiju. Ovaj algoritam izbjegava problem pojave malih šupljina na početku liste, pa je pretraživanje brže, ali ima manu što brzo potroši slobodni prostor na samom kraju memorije.

Best fit

Algoritam Najbolje uklapanje (best fit) prolazi kroz cijelu listu i odabere šupljinu najmanje veličine u koju može stati zahtijevana količina memorije. Ovaj algoritam je sporiji od prvog i sljedećeg uklapanja jer mora svaki put preći cijelu listu. Ako su zahtjevi takvi da se nalaze šupljine prave veličine, algoritam dobro iskorištava memoriju. Ali u praksi mala je šansa da će šupljina biti u potpunosti iste veličine kao što je zahtijevano, pa dobivamo mnoštvo malih šupljina koja se ne mogu praktično iskoristiti bez spore kompakcije.

Worst fit

Algoritam Najgore uklapanje (worst fit) prolazi kroz cijelu listu i odabere šupljinu najveće veličine. Algoritam se zasniva na pretpostavci da ako se zauzme najveća raspoloživa šupljina, njen ostatak će biti

dovoljno velik da bude upotrebljiv za kasnije zahtjeve. Međutim, simulacije su pokazale da je ovo najlošiji algoritam, jer će brzo potrošiti velike šupljine.

2.23. Šta je to buddy sistem?

To je pristup za alokaciju memorije koji kombinuje neke osobine alociranja po fiksnim i promjenjivim particijama. Sistem drugova (buddy system) dopušta alokaciju particija samo one veličine koja je stepen broja 2.

Algoritam radi na sljedeći način.

1. Neka je bio zahtjev za alokacijom particije veličine s u memoriji veličine 2^m bajta koju posmatramo kao jedan blok.
2. Ako je tražena količina memorije s takva da je $2^{m-1} < s \leq 2^m$, zauzmi čitav blok
3. U suprotnom blok se dijeli na dva jednaka dijela, umani m za 1, i idi na 2
4. Dobiven je najmanji blok koji je veći ili jednak zahtijevanoj veličini s .
5. Nađeni blok se ubacuje u ulančanu listu blokova određene veličine.

2.24. Kako se alocira memorija u Windows API a kako u Posix API?

Windows API

Memorija pod Windows sistemom se alocira koristeći poznate C funkcije `malloc` i `free`. Windows ima veći broj funkcija za alociranje globalne i privatne memorije:

`VirtualAlloc(lpAddress, dwSize, dwAllocationType, dwProtect)`

`HeapAlloc(hHeap, dwFlags, dwBytes)`

Posix API

Za razliku od oblasti procesa, Unix kompatibilni sistemi ne pružaju mnogo sistemskih poziva za upravljanje memorijom. Sve što je dostupno su funkcije koje povećavaju krajnju adresu sekcije podataka trenutnog procesa.

`int brk(void *last) ->` Postavi kraj područja memorije za proces na datu lokaciju

`void *sbrk(int increment) ->` Uveća kraj područja memorije za proces za dati broj bajta

Aplikacije trebaju koristiti C funkcije `calloc()`, `malloc()`, `free()` i slično. Ove funkcije nisu realizovane u jezgri operativnog sistema, nego izvršni C sistem mora da ima algoritme za alokaciju memorije.

2.25. Šta su dinamički dijeljene biblioteke?

Potprograme koje koriste skoro svi programi (npr. svi potprogrami iz standardne biblioteke jezika C, potprogrami za grafičko okruženje) je neracionalno ugraditi u svaki program. Dovoljno bi bilo imati samo

jednu kopiju ove funkcije u memoriji dostupnu svim procesima, čime se postižu uštede u memoriji i na disku. Više potprograma koji se mogu pozvati iz različitih procesa grupišu se u **dinamičke dijeljene biblioteke**.

2.26. U čemu je razlika između implicitno i eksplicitno vezanih dinamički dijeljenih biblioteka?

U **implicitnom** načinu se unutar izvršne verzije programa nalazi spisak dinamičkih biblioteka i njihovih potprograma koje program koristi. Pri startu programa punilac poveže sve dinamičke biblioteke. Programer u jeziku C treba da koristi samo odgovarajuće datoteke sa zaglavlјima i pravilno konfiguriran kompajler, pa i ne osjeti razliku u pozivu između potprograma u dinamičkim bibliotekama i običnih potprograma.

Eksplicitni način se koristi ako se unaprijed ne zna da li je dinamička biblioteka instalirana u sistemu i za proširivanje mogućnosti programa u budućnosti pri čemu se unaprijed ne mora znati ni ime biblioteke ni ime funkcije.

2.27. Kako je organizovan program ako se koristi overlay?

Program je podijeljen u podprogramme.

2.28. Šta je Harvard arhitektura?

Harvard arhitektura se smatra alternativom Von Neumanovoj. U Harvard arhitekturi program i podaci su potpuno odvojeni.

2.29. Kada se koriste memorijske banke?

Nekad je dizajner računarskog sistema ograničen osobinama procesora koji je izabran, ali ima uticaj na izbor memorije i razvoj operativnog sistema. Ako je primoran da koristi znatno veću memoriju onda se koriste memorijske banke. Banka je fiksna memorijska particija koja se bira iz skupa veće fizičke memorije i ubacuje u adresni prostor.

2.30. Šta radi memory management unit?

Može se reći da MMU(**memory management unit**) obavlja transformaciju virtualne adrese u fizičku adresu odgovarajućom funkcijom: $FizičkaAdresa = f(VirtualnaAdresa)$

2.31. Kako se računa fizička adresa sa segmentacijom s više relokacionih registara?

Jedan pristup, je sličan kontinualnoj alokaciji s relokacionim registrom, samo što postoje **tri ili više relokacionih registara** namijenjenih različitim područjima memorije. Tako npr. Intel 8086 ima četiri takva registra koji se zovu segmentni registri: CS koji u memoriji pokazuje početak koda, DS koji pokazuje početak podataka, SS koji pokazuje početak područja za stack i ES koji se koristi za pristup području podataka kada se ne želi mijenjati DS. Na taj način instrukcije JMP 1000 (bezuslovni skok na memorijsku lokaciju 1000) i MOV AX,[1000] (čitanje vrijednosti memorijske lokacije 1000 i prebacivanje u registra AX) pristupaju različitim fizičkim lokacijama unutar istog procesa. Na ovom procesoru zadržane su 16 bitne logičke adrese u programima (što čini programe kraćim jer instrukcije zauzimaju manje prostora u memoriji, nego kada bi bile 32 bitne), a fizički adresni prostor je realizovan kroz 20 bitne adrese pa dopušta do 1 megabajt fizičke memorije. Segment ovdje predstavlja kontinualno područje

od 64 kilobajta unutar fizičke memorije. Transformacija logičke u fizičku adresu je jednostavna i brza: vrijednost segmentnog registra relevantnog za instrukciju pomnožena sa 16 (što se postiže ožičenjem) se sabere sa logičkom adresom. Pristup ima dosta mana. Adresa početka segmenta je direktno vezana za relokacioni registar. Korisnički proces mora imati pravo da postavi adresu početka segmenta ako se njegovi podaci protežu u više segmenata. U tom slučaju ništa ga ne sprječava od pristupa području memorije koje je dodijeljeno drugom procesu ili jezgri i oštećenja drugih procesa ili jezgra OS. Dalje, segmenti su fiksne veličine. Stoga se ne mogu npr. koristiti nizovi i slične strukture čija veličina prelazi veličinu segmenta.

2.32. Kako se računa fizička adresa sa tabelom segmenata?

U drugom pristupu segmentaciji (korišten u Windows 3.1, u standardnom režimu), uvodi se **tabela segmenata** koja ima onoliko elemenata koliko je moguće imati segmenata u sistemu. Elementi tabele se zovu deskriptori. Deskriptor sadrži za svaki segment, kome se pristupa preko rednog broja njegovu početnu adresu, veličinu, vrstu, prava pristupa i podatak da li je segment prisutan u memoriji. Različiti segmenti mogu biti različitih veličina. Pošto svaki segment ima svoj adresni prostor (linearan), različiti segmenti ne utiču jedan na drugi. Transformacija logičke adrese u fizičku se obavlja tako što se za dati redni broj segmenta iz odgovarajućeg deskriptora preuzme adresa početka segmenta i sa tom adresom sabere adresa unutar segmenta. Segmenti su stoga relokabilni, a korisnički program ne operiše s fizičkim adresama. Tabeli segmenata pristupa jezgro i raspoređuje logičke segmente po fizičkoj memoriji. Kako segmenti imaju svoju definisanu dužinu u jezgri, postignuta je i zaštita segmenata.

2.33.Šta je deskriptorski keš?

Kako se početna adresa svakog segmenta nalazi u memoriji, segmentacija sa tabelom segmenata na prvi pogled usporava pristup memoriji, jer bi se pri svakom pristupu memoriji moralo pročitati gdje segment počinje, pa tek onda pristupiti podatku. Da bi se to izbjeglo, u procesor se ugrađuje **deskriptorski keš**, koji pamti početne adrese zadnje pristupanih segmenata.

2.34.Šta je to tabela stranica?

Iz virtualne adrese se direktno dobiva broj virtualne (logičke) stranice. Virtualna adresa se dijeli na broj virtualne stranice (biti na višim pozicijama) i poziciju unutar stranice (biti na nižim pozicijama). Iz broja stranice se dobiva fizička adresa korištenjem **tabele stranica** (engl. page table) koja preslikava virtualnu stranicu u fizičku stranicu. Ova tabela se dodjeljuje svakom procesu i mijenja se u toku izvršavanja procesa, kao i prilikom izmjene konteksta procesa. Broj virtualne stranice predstavlja indeks u tabeli stranica. U toj tabeli se nalazi broj odgovarajućeg okvira stranice, i informacija da li je stranica u fizičkoj memoriji. Broj okvira stranice i pozicija unutar stranice daju fizičku adresu koja se proslijeđuje memorijskim integrisanim kolima.

2.35. Čemu služe bit prisutnosti, modifikacije, referenciranja i zaštite?

Potreban je jedan bit koji označava da li je navedena virtualna stranica u fizičkoj memoriji. Taj bit se zove **bit prisutnosti** (present bit) i ima npr. vrijednost 1 ako je stranica u fizičkoj memoriji, a 0 ako nije.

Da bi se znalo koji okvir je manje korišten, u tablicu stranica se pored rednog broja fizičkog okvira i bita prisutnosti dodaju **bit modifikacije M** i **bit referenciranja R**. Svaki put kada se čita ili piše podatak iz memorijskog područja date stranice, u tabeli stranica se bit R postavi na vrijednost 1. Ako se podatak upisuje u tom području, pored bita R, na vrijednost 1 se postavlja i bit modifikacije M. U tablici stranica se mogu nalaziti i **biti zaštite**, npr. biti koji određuju da se u toj stranici nalaze podaci koji se mogu samo čitati, ili da stranici smije pristupati samo jezgro.

2.36. Čemu služi TLB?

Skup specijalnih registara koji predstavljaju kopije najčešće korištenih elemenata tabele stranica zove se **Translation lookaside bafer (TLB)**. On se nalazi unutar MMU ali se sastoji od relativno malog broja elemenata. Kada se jedna virtualna adresa pošalje MMU, on prvo provjerava da li će odgovarajući broj stranice pronaći u TLB. Ako se pronađe odgovarajući broj stranice, tada se broj okvira stranice dobije iz TLB. Ako se virtualna stranica ne nalazi u TLB, MMU tada normalno pristupa tabeli stranica, ali za buduće pristupe upisuje element tabele u TLB pri čemu izbacuje neki stari element tablice.

2.37. Kada se koristi hijerarhijsko straničenje?

Praktičniji način za smanjenje veličine tabele stranica je straničenje na dva ili više nivoa. Za veće adresne prostore, može se ići i na više nivoa tablica stranica. Štoviše, samo tablica najvišeg nivoa mora stalno biti u memoriji. Tablice na nižim nivoima se mogu čuvati i izvan centralne memorije i ubacivati u memoriju tek kada se zaista pristupi memorijskoj lokaciji na koju trebaju pokazivati.

2.38. U čemu je razlika između metode jednake raspodjele i metode proporcionalne raspodjele?

Metoda **jednake raspodjele** je najjednostavnija metoda. Ako ima m okvira i n procesa, prema metodi jednake raspodjele, svakom se procesu dodjeljuje jednaka količina okvira m/n .

Prema metodi **proporcionalne raspodjele**, procesima koji su prijavili veći adresni prostor, dodjeljuje se i relativno više fizičkih okvira. Ako je m ukupan broj okvira fizičke memorije, n broj procesa, a S_i -veličina virtualnog memorijskog prostora procesa p_i , tada se svakom procesu i dodjeljuje $a_i = \frac{S_i}{\sum_{i=1}^n S_i}$ fizičkih okvira.

2.38. U čemu je razlika između lokalne i globalne strategija straničenja.

U slučaju da broj dodijeljenih fizičkih okvira procesu ostaje isti, govorimo o **lokalnoj strategiji straničenja**. Ako dođe do greške stranice, moramo izbaciti stranicu u skupu dodijeljenih okvira. Prednost pristupa je u bržem izboru te stranice, jer se ne moraju čitati tablice stranica drugih procesa. Očigledni su nedostaci ove strategije: nekada je proces u stanju kada ima nedovoljno fizičkih okvira, a nekada u stanju kada ih ima više nego što treba. U prvom slučaju se dešavaju česte greške stranica, a u drugom slučaju drugi procesi kojima u tom trenutku treba više fizičke memorije ne mogu iskoristiti one fizičke okvire koji su dodijeljeni procesu koji ih drži a ne koristi.

Ako proces mijenja skup fizičkih okvira na račun drugih procesa, tada je riječ o **globalnoj strategiji straničenja**. Ako proces A generiše grešku stranice, stranicu koja će biti izbačena iz RAM se ne traži samo u skupu okvira pridruženih tom procesu, već u skupu svih okvira. Stoga se može desiti, da se izbací neka stranica iz okvira koji je bio dodijeljen procesu B, a na to mjesto ubaci tražena stranica procesa A. Ova strategija bolje koristi memoriju ali zahtijeva više pretraživanja, jer obično svaki proces ima svoju tabelu stranica, pa treba pri određivanju fizičkog okvira pretražiti sve tabele stranica (ili imati dodatnu tabelu indeksiranu po fizičkim okvirima).

2.39. U čemu je razlika između učitavanja po potrebi i učitavanja s predviđanjem.

Kod **učitavanja po potrebi** (engl. demand paging) stranica se ubacuje sa diska u memoriju, samo onda kada je MMU traži. Ovaj pristup štedi radnu memoriju, jer se u njoj čuvaju samo stvarno potrebne stranice. Loša strana je sporo pokretanje programa, jer će u ranoj fazi program zahtijevati više memorijskih lokacija, a ovim algoritmom će se puniti jedan po jedan memorijski okvir bez iskorištenja optimalnog trenutka rotacije diska.

Kod **učitavanja sa predviđanjem** (engl. anticipatory paging) pokušava se pogoditi koje će stranice biti potrebne za rad procesa. Jezgro ili pozadinski program ubaci u memoriju te stranice, obično u vrijeme kada procesor nije zauzet izvršavanjem aplikacija. Ako je pretpostavka koje stranice treba učitati bila tačna, smanjiće se kasniji broj grešaka stranice i izvršavanje procesa dobiva na brzini. Ako je pretpostavka bila loša, od učitavanja nema nikakve koristi. Kako cijena kapacitet interne memorije raste, ovaj pristup je sve popularniji.

2.40. Algoritam određivanja zamjene stranica: Slučajni izbor.

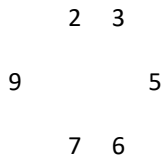
Najjednostavnije rješenje je izbaciti bilo koju stranicu koja je trenutno u radnoj memoriji. Osnovna prednost ovog algoritma je što nije potrebno dodatno vrijeme za pretraživanje stranica u tabelama radi donošenje odluke o tome koja stranica će biti izbačena, niti su potrebne dodatne memorijske strukture osim same tabele stranica. Algoritam ne vodi ni na koji način računa o korištenosti i značaju neke memorijske stranice, pa može rezultovati lošim performansama. Algoritam je korišten u OS/390 iz 1996. kao rezervni algoritam kada drugi korišteni algoritmi pokazuju loše performance

2.41. Algoritam određivanja zamjene stranica: FIFO.

Naredna logična ideja je izbaciti najstariju stranicu. Pored tabele stranica, održava se i lista svih stranica koje su trenutno u memoriji. Lista je poredana po vremenu prvog pristupa stranici, tako da se stranica koja je najstarija nalazi na početku liste, a na njenom kraju je ona koja je zadnja ubačena u memoriju. Kod greške stranice, stranica sa početka liste se uklanja a na kraj se dodaje nova stranica. Održavanje i pretraživanje ove liste je relativno brzo, a algoritam se može primijeniti i u jednostavnim sistemima straničenja koji nemaju informaciju o korištenju stranice. Pretpostavka algoritma je da stranice koje su davno učitane sadrže instrukcije koje su davno izvršene ili podatke koji više nisu potrebni. To ne mora biti tačno. Može se zbog toga izbaciti važna stranica iz memorije. Primjer predstavlja stranica koja pripada glavnoj petlji nekog programa, učitana je među prvim i najčešće se koristi. FIFO algoritam će je izbaciti i zatim ponovo vratiti. U nešto izmijenjenoj verziji, ovaj algoritam je korišten na operativnom sistemu VAX/VMS.

2.42. Algoritam određivanja zamjene stranica: LDF .

Iako je decenijama izgledalo da se na polju jednostavnih algoritama za zamjenu stranica koji ne uzimaju u obzir korištenost stranice nema šta više istraživati, Gyanendra Kumar je 2017. objavio rad o algoritmu LDF. Ideja algoritma je da se većina programa izvršava sekvencijalno, pa je pri izvršavanju programskog koda ili čitanju podataka iz logičke stranice k , vjerovatno ubrzo potrebna stranica $k+1$. Udaljenost između stranica se računa tako da sve logičke stranice kojima proces pristupa poredaju u krug u rastućem redoslijedu logičkog broja stranice, Na primjer logičke stranice 3,5,2,6,7,9 poredane u krug izgledaju ovako.



Udaljenost između stranica 9 i 6 je ovdje 2, putem preko stranice 7, a udaljenost između stranica 2 i 6 je 3, putem preko stranica 3, 5 i 6. Kada neka stranica nije u memoriji, a pristupa joj se, izbacuje se ona stranica koja je najdalja od nje. Ako postoje u memoriji dvije stranice koje su jednako udaljene od stranice koja je izazvala grešku, izbacuje se ona sa manjim rednim brojem. Algoritam se ponaša dobro za stranice s programskim kodom, ali za podatke koji se naizmjenično čitaju iz raznih krajeva memorije (npr. neki algoritmi za sortiranje) može imati loše performanse.

2.43. Algoritam određivanja zamjene stranica: NRU .

Slučajni izbor stranice ne vodi računa o tome da li je stranici pristupano ili da li je mijenjana u međuvremenu. Znatno unapređenje ovog algoritma može se postići ako tabele stranica imaju informacije o tome. Mnogi sistemi za upravljanje memorijom podržavaju u tablici stranica dva statusna bita koji označavaju da li je sadržaju te stranice pristupano. Bit R (engl. Referenced) označava da li se pristupalo tom okviru. Bit M (engl. modified) znači da li je mijenjan sadržaj tog okvira. Bit R se postavlja automatski na 1 svaki put kad se čita ili piše bajt koji pripada toj stranici. Bit M se postavlja na 1 kada se piše podatak u bilo koji bajt koji pripada toj stranici. Kad je neki od ova dva bita postavljen na 1, on zadržava tu vrijednost sve dok operativni sistem ne promijeni njihovu vrijednost u tabeli stranica. Pri pravljenju tabele stranica, tokom pokretanja procesa, operativni sistem postavlja oba bita za sve stranice u tabeli na 0. U algoritmu NRU se periodično, na svaki prekid sata, R bit briše, ali ne i M bit. Tako je R bit postavljen samo kod stranica koje su u zadnje vrijeme korištene. Kombinacijom R i M bita stranice se dijele u četiri klase:

Kl.	R	M	Situacija
0	0	0	Ova stranica nije ni čitana ni mijenjana u zadnje vrijeme
1	0	1	Stranica je mijenjana u prošlosti, ali od zadnjeg čišćenja R bita nije joj u međuvremenu pristupano
2	1	0	Stranica je čitana od zadnjeg čišćenja R bita
3	1	1	Stranica je mijenjana od zadnjeg čišćenja R bita

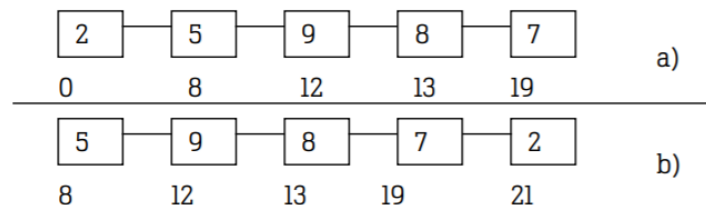
Kada se pojavi greška stranice algoritam NRU (Not Recently Used) pregleda sve stranice i odabere za brisanje iz memorije bilo koju stranicu iz najniže klase. Time se smanjuje potreba za upisivanjem izbačenih stranica na disk i izbacivanjem nedavno korištenih stranica.

2.44. Algoritam određivanja zamjene stranica: Optimalni algoritam.

László Bélády je 1966. predstavio optimalni algoritam. U ovom algoritmu treba izbaciti onu stranicu za koju se smatra da će najkasnije biti potrebna u budućnosti ili još bolje ako neće uopće više biti potrebna. Ako će stranica broj 1 biti potrebna nakon 3 sekunde rada, a stranica broj 2 nakon 30 sekundi, bolje je izbaciti stranicu broj 2. Dokazano je da ovakva strategija rezultuje najmanjim brojem grešaka stranice, ali ima jedan ogroman nedostatak. Ovaj algoritam nije moguće realizovati, jer se ne može predvidjeti ponašanje programa u budućnosti. Upotrebna vrijednost ovog algoritma je jedino za ocjenu drugih algoritama, nakon što je prethodno evidentirano kojim je stranicama pristupano i uočeno koliko grešaka stranica su ti algoritmi napravili. Ali, ako se budućnost ne može predvidjeti, prošlost se može pratiti, pa naredni algoritmi analiziraju dosadašnje pristupanje stranicama kako bi se približili optimalnom algoritmu.

2.45. Algoritam određivanja zamjene stranica: Druga šansa.

Glavna mana algoritma FIFO je što može izbaciti vrlo često korištenu stranicu jer je rano ubačena u operativnu memoriju. Algoritam je moguće znatno poboljšati uzimajući u obzir pored starosti stranice i stanje njenog bita pristupa, čime nastaje algoritam Druga šansa. Ovaj algoritam sprječava odbacivanje stranica koje se često koriste ispitujući R bit najstarije stranice. Ako je $R=0$, to znači da je stranica dugo u memoriji a da joj nije pristupano. Prema tome ona nije potrebna, može se izbaciti i u njen stranični okvir staviti nova. Ako je $R=1$, to znači da je stranica dugo u memoriji, ali je nedavno korištena. Tada će se staviti $R=0$ a stranica prebaciti na početak liste sortirane po vremena dolaska (tj. njeno vrijeme dolaska postavljamo na trenutno) i analizirati sljedeću stranicu. Znači, ako je stranica korištena dobiva još jednu šansu. Rad ovog algoritma je prikazan na slici Slika 66a. Tu se vide stranice čuvane u ulančanoj listi i sortirane prema vremenu kad su stigle u memoriju.

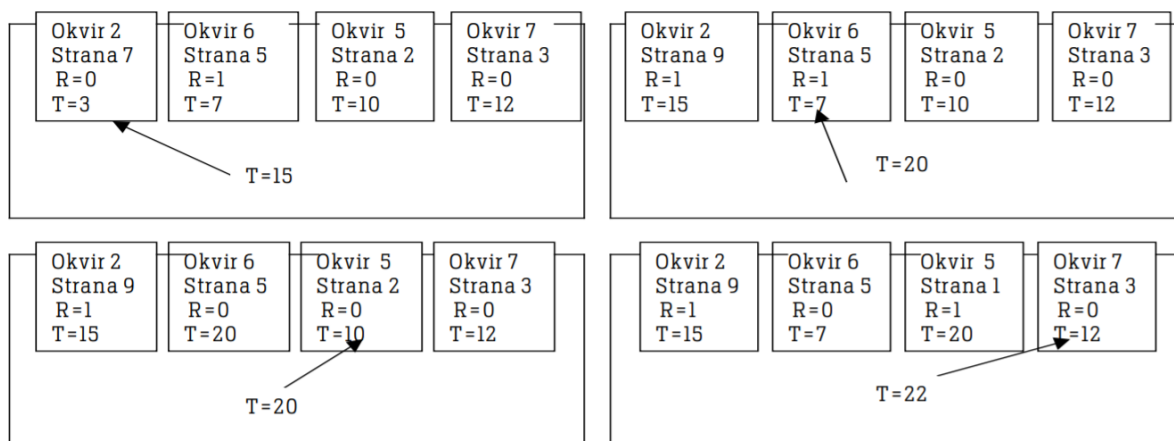


Neka se nakon situacije (a) desila greška stranice u trenutku 21. Najstarija stranica je 2, koja je stigla u trenutku 0 (pokretanja procesa). Ako stranica 2 ima $R=0$, stranica se uklanja iz memorije. S druge strane ako je R bit postavljen ($R=1$), stranica 2 se stavlja na drugi kraj liste (Slika 66b) i njeno vrijeme ulaska u memoriju se postavlja na trenutno vrijeme (21). Za ovu stranicu R bit se postavlja na 0. Potraga za odgovarajućom stranicom nastavlja sa stranicom 5, koja je sljedeća po vremenu dolaska.

2.46. Algoritam određivanja zamjene stranica: Satni (Clock page) algoritam.

Algoritam Druga šansa se može ubrzati, ako se okviri stranica čuvaju u kružnoj listi, kao što je prikazano na slici Slika 67. Jedan pokazivač pokazuje na stranicu koja je najranije učitana. Kada dođe do greške stranice, provjeri se R bit stranice na koju pokazuje pokazivač. Ako je $R=0$ stranica se izbacuje a nova se ubacuje u okvir koji je zauzimala izbačena stranica i pokazivač se pomjera za jedno mjesto. Ako je $R=1$, tada se promijeni R da postane 0 i pokazivač se pomjera na sljedeću stranicu, čime se nastavlja traženje. Ako se pri tome obiđe cijeli krug, svi R biti su postali 0,

pa će se stranica obrisati. Satni algoritam ima isti broj grešaka stranica kao algoritam druga šansa, i bira potpuno iste stranice, ali je brži u realizaciji jer se pomiče samo jedan pokazivač.



2.47. Algoritam određivanja zamjene stranica: LRU (Least Recently Used) .

Budućnost se ne može predvidjeti, ali inverzijom ideje optimalnog algoritma može se ovaj algoritam aproksimirati. Tako umjesto odbacivanja stranice koja će najkasnije u budućnosti biti korištena, algoritam LRU (Least Recently Used) radi na principu da se odbacuje stranica koja se najranije zadnji put koristila u prošlosti. Ova inverzija je zasnovana na pretpostavci da stranice koje su se puno koristile u zadnjih nekoliko instrukcija će biti vjerovatno korištene u nekoliko budućih instrukcija. Ovaj algoritam je dobar, ali teško ostvarljiv. Osnovni problem sa ovim algoritmom je što zahtijeva posebnu hardversku podršku. Tablica stranica treba za svaku stranicu, pored broja fizičkog okvira, bita prisustva i bitova R i M, da ažurira i polje o vremenu zadnjeg pristupa. Vrijeme se može evidentirati posebnim brojačem koji se uvećava nakon svake procesorske instrukcije ili memorijskog ciklusa i pri svakom čitanju ili pisanju u memorijsku stranicu treba prepisati vrijednost tog brojača u odgovarajući TLB element tablice stranica. Kada se desi greška stranica, pregledaju se brojači svih stranica i odabere među njima ona sa najmanjom vrijednosti brojača. Ako tablica stranica nema odgovarajuće polje za vrijednost brojača, realizacija ovog algoritma je softverska i nedopustivo je spora. Tablica stranica bi u tom slučaju bila napravljena tako da svaki pristup memoriji izaziva grešku stranice. U potprogramu koji se poziva prilikom greške stranice, upisivalo bi se u dodatnu tabelu zadnje vrijeme pristupa odgovarajućoj stranici. To bi značilo usporenje rada računara više desetina puta

2.48. Algoritam određivanja zamjene stranica: LFU (Least Frequently Used) .

Ako nemamo potrebnu hardversku podršku sa brojem pristupa u tabeli stranica, GClock se ne može efikasno realizovati, ali može se probati njegova aproksimacija. Za svaku stranicu dodjeljuje se po jedan softverski brojač izvan tabele stranica koji se na početku postavlja na 0. Kod svakog prekida sata operativni sistem provjeri tablicu stranica u memoriji i sabere sa svakim brojačem odgovarajuće stranice njen R bit. Nakon toga se svi R biti postave na 0. Kada je potrebno neku stranicu izbaciti iz memorije, izbacuje se ona sa najmanjom vrijednosti brojača, jer se njoj najmanje pristupalo. Takav algoritam se zove LFU (Least Frequentntly Used) ili NFU (Not Frequently Used). Ovakav algoritam na prvi pogled predstavlja dobru aproksimaciju LRU algoritma. Međutim, nekada se dešavaju situacije da je pojedina stranica u prošlosti mnogo korištena, a onda dugo nije korištena, kao što je stranica u kojoj

se nalaze instrukcije za inicijalizaciju programa. Kako ovaj brojač nije očišćen u međuvremenu, često korištene kasnije ubačene stranice neće moći dostići vrijednosti brojača onih stranica koje su ostale u memoriji “zbog stare slave”. Posljedica je da će nove i često trenutno potrebne stranice biti izbačene iz memorije. Alternativa je algoritam koji bi izbacivao stranice sa najvećom vrijednošću brojača MFU (Most Frequently Used). Logika algoritma je da stranica sa malim brojačem je tek počela da se koristi, pa je relativno nova, ali ovaj algoritam dovodi do puno veće štete: dešavaće se da se stranice izbacuju upravo u trenutku kada se najviše koriste.

2.49. Algoritam određivanja zamjene stranica: Algoritam starenja .

Nedostatak LFU algoritma (akumulirano pamćenje pristupa stranica) može se riješiti ako se sabiranje brojača sa vrijednošću R zamijeni operacijom pomicanja u desno. Ovaj algoritam je poznat pod imenom starenje (aging). Pri svakom prekidu sata pomjeramo brojače tako da se bit R stranice za koju je vezan brojač stavi na mjesto najznačajnijeg bita brojača, dosadašnji najznačajniji bit prelazi na drugu poziciju s lijeve strane, i svi ostali biti se tako pomjeraju u desno, pri čemu se najniži bit gubi. Efektivno n-bitni brojač se ažurira po formuli:

$$brojač^n = \frac{brojač^{n-1}}{2} + 2^{n-1} R$$

Primjer ovog algoritma pokazuje Slika 68. U prvom intervalu R biti imaju vrijednost 1, 0, 0, ..., 1. Nakon pomjeranja brojača za jedan bit u desno i dodavanja R bita krajnjem lijevom bitu imao situaciju sa druge kolone slike Slika 68. Ostale kolone pokazuju šta se dešava nakon sljedeća 2 intervala. Kada je stranica dugo neaktivna, njen brojač će težiti nuli. Nema, međutim, načina da se ustanovi koliko dugo je stranica ostala neaktivna nakon što je brojač postao nula. Ravnopravni kandidati za brisanje su i stranica koja je zadnji put korištena prije 10 prekida sata i stranica koja je zadnji put korištena prije 10000 prekida sata, jer brojač prikazuje samo događaje u zadnjih 8 prekida sata.

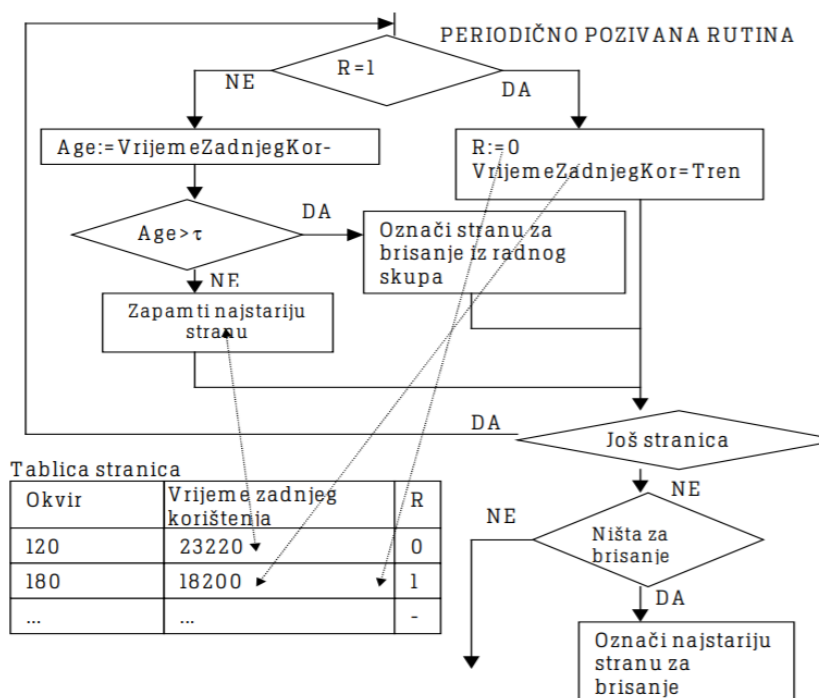
	Vremenski prekid 0		Vremenski prekid 1		Vremenski prekid 2		Vremenski prekid 3	
Strana 0	1	01000000	0	10100000	0	01010000	1	00101000
Strana 1	0	10000000	0	01000000	0	00100000	0	00010000
Strana 2	0	00000001	1	00000000	1	10000000	0	11000000
...	...	...	...	...	...	...	...	...
Strana 4096	1	00000100	1	10000010	1	11000001	0	11100000

R bit
Nedavna aktivnost

2.50. Šta je to radni skup stranica?

Do sada opisani algoritmi su odabirali jednu stranicu za brisanje iz radne memorije i pozivali su se prilikom greške stranice ili periodično. Pri tome su procesi zadržavali mnoge nepotrebne stranice u memoriji, dok se proces izbora stranice morao ponavljati svaki put kada se desi greška stranice. Sada će biti opisani algoritmi koji izbacuju više stranica koje ne zadovoljavaju kriterij čuvanja u memoriji, kako bi ostalim procesima pripalo više slobodne

memorije. Stranice koje proces u jednom određenom trenutku koristi zovu se radni skup (engl. working set). Njegova veličina varira tokom rada procesa. U početku je on prazan, (ako se koristi straničenje na zahtjev) i sistem generiše mnogo grešaka stranica dok se one ne ubace u radnu memoriju. Nakon ubacivanja potrebnih stranica u radnu memoriju, broj grešaka stranica opada. Ali ako proces počne zahtijevati dodatnu memoriju, ili mu je oduzeta memorija kako bi se dodijelile drugom procesu, proces ponovo proizvodi mnogo grešaka stranice i radi sporo. Ovakvo ponašanje naziva se thrashing. Zaključak je da radni skup ne smije biti mali (npr. samo nekoliko stranica) jer će greške stranica biti prečeste. Ali radni skup ne treba biti ni prevelik, da npr. zauzima cijeli virtualni adresni prostor procesa. Pošto se u jednom trenutku pristupa manjem broju stranica dovoljno je da radni skup ima takvu veličinu da greške stranica budu rijetke. Algoritmi zamjene stranica koji koriste radni skup, pored izbora stranice koja će biti obrisana, održavaju i radni skup na potrebnoj veličini. Sada će biti opisan jedan takav algoritam, baziran na LRU.



Rutina prikazana dijagramom toka na slici Slika 69 poziva se periodično, npr. svakih 50 ms. Uz tabelu stranica (sa bitima pristupa R), pamti se približno vrijeme zadnjeg pristupa stranici. Za razliku od LRU ovo vrijeme se ne mora evidentirati pri svakom pristupu memoriji, nego pri (daleko rjeđem) periodičnom pozivu rutine. Kod svakog elementa tablice se prvo ispituje R bit. Ako je ovaj bit postavljen na 1, taj bit se mijenja na 0, a u polje za vrijeme zadnjeg pristupa se unosi trenutno vrijeme. To znači da je stranici pristupano od zadnjeg izvršavanja rutine, pa stranica pripada radnom skupu i ne treba je izbaciti. Ukoliko je R=0 ta stranica je mogući kandidat za izbacivanje zato što joj nije pristupano u zadnjem vremenskom intervalu. Da se proračuna da li stranica pripada u radni skup uzima se u obzir njena starost, razlika trenutnog vremena i vremena zadnjeg pristupa. Parametar τ reguliše veličinu radnog skupa i označava koliko se dugo od zadnjeg pristupa stranice drže u memoriji. Ukoliko je starost stranice veća od τ , ova stranica ne spada više u radni skup i može biti izbačena. Ako su sve stranice korištene u intervalu koji se smatra periodom čuvanja radnog skupa, izbacuje se najstarija među njima.

2.51 Šta je to Beladejeva anomalija?

Intuitivno možemo zaključiti da sistem sa više fizičkih okvira stranice proizvodi manje grešaka stranice.

Laszlo Bélády je 1969. otkrio da FIFO (First in first out) u sistemu sa četiri okvira stranice ponekad proizvodi više grešaka stranice nego u sistemu sa tri okvira stranica. Ova situacija se naziva **Bélády anomalija**.

Godine 2010. je pokazano da se može konstruirati redoslijed pristupa stranicama u kome memorijski sistem sa većim brojem okvira generiše proizvoljan broj puta više grešaka stranice nego sistem sa manjim brojem okvira.

2.52. Kako izgleda segmentacija sa straničenjem?

Segmentacija je metoda upravljanja memorijom koja podržava logički korisnički pogled na memoriju.

U kombinovanom sistemu straničenje/segmentacija, korisnički adresni prostora je razbijen u niz segmenata po nahođenju programera. Svaki segment je dalje razbijen u niz stranica fiksne (i identične) veličine koje su jednaki dužini okvira glavne memorije. Ako je segment manji od dužine stranice, segment zauzima samo jednu stranicu.

U tablici stranica se mogu nalaziti i biti zaštite, npr. biti koji određuju da se u toj stranici nalaze podaci koji se mogu samo čitati, ili da stranici smije pristupati samo jezgro.

2.53. Koja je Uloga LDTR i GDTR registara na i386?

Srce virtualne memorije sadrži se od dvije tabele, LDT (Local Descriptor Table) i GDT (Global Descriptor Table). Svaki program ima svoju vlastitu LDT, dok postoji samo jedna GDT, koja se dijeli između svih programa na računaru. LDT opisuje segmente koji su lokalni svakom programu, uključujući njegov kod, podatke, stack, itd., dok GDT opisuje sistemske segmente, uključujući sam operativni sistem

2.54. Kako izgleda deskriptor na i386?

Elementi tabele segmenata se zovu deskriptori. Deskriptor se sastoji od 8 bajtova, uključujući adresu početka segmenta, veličinu i druge informacije kao što su da li je segment u memoriji.

2.55. Kako se linearna adresa transformiše u fizičku na i386?

Ako je straničenje onemogućeno, linearna adresa se kao fizička šalje u memoriju za čitanje ili upis. Sa druge strane, ako je straničenje omogućeno, linearna adresa se interpretira kao virtualna i mapira u fizičku adresu korištenjem tabele stranica

2.56. Koje prijetnje postoje za operativne sisteme?

Prijetnje koje sprječavaju osnovnu funkciju operativnih sistema da obradjuju podatke možemo podijeliti u više kategorija: **prirodne katastrofe, hardverske greške, ljudski faktor, softverske greške i zlonamjerni softver.**

Protiv **prirodnih katastrofa** ne možemo učiniti mnogo, ali možemo praviti redovan backup podataka na udaljeni cloud server i tako ih čuvati.

Ljudi su najveća prijetnja sigurnosti računarskog sistema, nenamjerno (pogrešno otkucan kod) ili namjerno (intruders, pristup nekim skrivenim podacima).

2.57. Koje su vrste zlonamjernog softvera?

Zlonamjerni program predstavlja program, napisan da bi uzrokovao štetu i navođen je ciljano na sistem. Ima ih više vrsta :

Virus - predstavlja dio koda koji se može reprodukovati, pišući svoj kôd kao dodatak nekoj datoteci (što liči na sličan proces reprodukcije bioloških virusa). Virusi mogu raditi i druge stvari (puštanje muzike, brisanje datoteka i sl).

Crv - predstavlja neovisan program. Crvi se šire korištenjem mreže za prijenos vlastitih kopija na druge računare, ali mogu da obavljaju kasnije slične operacija kao i virusi.

Trojanac - je program koji izgleda kao normalni program (igra, korisnički program) ali izvršava i neke druge funkcije, možda se pokreće crv ili virus ili izvršava neku od zlonamjernih akcija. Efekti trojanskog konja treba da budu tihi i nečujni.

Logička bomba - tajna opcija nekog programa koja se pokreće pod određenim uslovima, na primjer knjigovodstveni programi koji prestaju da rade ako u bazi podataka ne nađu da je na žiro-račun autora uplaćen određeni iznos ili ako otkriju da je instaliran konkurentski program.

Spyware - softver koji šalje informacije o korisničkom računaru, na primjer koje web stranice se najčešće posjećuju sa tog računara.

2.58. Šta je matrica kontrole pristupa i koji su načini pogleda na nju?

Sve kombinacije prava korisnika nad resursima u sistemu opisuju se **velikom matricom kontrole pristupa**, (access control matrix), skraćeno **ACM**. Redovi ove matrice su korisnici, a kolone resursi kojima se upravlja. Svaki član ove matrice predstavlja konkretna prava koja navedeni korisnik ima nad određenim resursom.

Pošto je ova matrica vecinski prazna, smještanjem nje u memoriju gubimo mnogo prostora, stoga imamo 2 alternative (**lista kontrole pristupa – ACL** , te **lista mogucnosti – C lista**)

2.59. Kako se postiže izolacija između procesa?

Izolacija između procesa je skup različitih hardverskih i softverskih tehnika koje štite svaki proces od drugih procesa u operativnom sistemu. Ona sprječava da proces P1 upisuje ili čita podatke koji pripadaju procesu P2. Procesna

izolacija se postiže virtualnim adresnim prostorom, gdje je adresni prostor procesa P1 različit od adresnog prostora procesa P2. Postojanje izolacije procesa smanjuje mogućnosti djelovanja zlonamjernog softvera i zlonamjernih korisnika

2.60. Šta je to tehnika pješčanika ?

Tehnike pješčanika (sandboxing) su načini izolacije nedovoljno pouzdanog softvera od ostatka sistema. One se koriste za izvršavanje netestiranog softvera ili softvera iz nepovjerljivih izvora. Oni često rade na principu virtualnog operativnog sistema koji simulira unutar posebne datoteke ponašanje računarskog sistema. Ako testirani softver sadrži virus, on se neće moći širiti dalje od područja pješčanika i zaraziće samo to područje.

Druga tehnika pješčanika je pokretanje procesa sa pravima koja dopuštaju minimalan broj sistemskih poziva.

2.61. U koje grupe možemo podijeliti kriptografske algoritme?

Kriptografske algoritme možemo podijeliti u sljedeće grupe:

- Tajni algoritmi: sigurnost se zasniva na tajnosti algoritma.
- Algoritmi zasnovani na ključu: algoritam je javno poznat, a ključ se čuva u tajnosti. Dije se na tri osnovna tipa
 - Simetrični - koristi se jedinstven ključ i za šifrovanje i za dešifrovanje.
- Asimetrični - postoje dva ključa (ključ za dešifrovanje se razlikuje od ključa kojim je sadržaj šifrovan).
 - Hibridni - kombinacija su prethodna dva.
- Algoritmi za jednosmjerno šifrovanje preko hash funkcija: šifrovani podaci se ne mogu vratiti u čitljiv oblik

2.62. Koje tri klase korisnika i tri vrste prava su u Unix sistemima?

Unix sistemi poznaju tri klase korisnika. Jednoj pripada vlasnik datoteke, drugoj članovi grupe korisnika vezanih uz datoteku, a trećoj ostali korisnici. Za svaku vrstu korisnika se navode po tri bita, za pravo čitanja, pravo pisanja i pravo izvršavanja.

Unix bazirani sistemi poznaju tri vrste prava. Prava pristupa se definišu sa devet bita ili 12 bita. Da se za svaku datoteku ne bi evidentirala prava pristupa za svakog korisnika pojedinačno, korisno je sve korisnike razvrstati u klase i za svaku od njih vezati spomenuta prava pristupa.

2.63. Šta su SID i pristupni token u Windows sistemima?

Svaki subjekt u sistemu, a to su korisnici, grupe i kompjuteri u mreži imaju sigurnosne identifikatore (SID). SID je broj koji ima 128 bita i predstavlja se odvojenim grupama, tako neki od subjekata može imati oznaku poput S-1-5-21-2025429065-492874223-1748137768500. Kada se korisnik prijavi, pokupe se SID-ovi korisnika i svih grupa kojima on pripada i sastavi lista koja se pridruži procesu. Ta lista zajedno s još nekim podacima se pakuje u pristupni token procesa. Ako proces kreira nove procese, njegovi sigurnosni tokeni se inicijalno nasljeđuju. No, niti koje kreira proces mogu imati i sasvim druge sigurnosne tokene, ukoliko je potrebno da se izvršavaju pod pravima udaljenih korisnika za lokalne servise.

2.64. Koja je razlika u načinu prijave na radnu grupu, domenu i aktivni direktorij u Windows sistemima?

Prema načinu provjere prijave, računarske mreže se organizuju u tri oblika. Prvi su radne grupe. **Kod radne grupe, svi računari u mreži su ravnopravni i na svakom od njih se nalazi spisak korisnika i lokalnih grupa koje se mogu na njega prijaviti.** Korisnici mogu dijeliti diskove, ali je broj korisnika koji mogu istovremeno pristupiti ograničen. Postavljanje radne grupe je jednostavno, ali je održavanje otežano, jer korisnik mora imati svoja imena na svim

računarima na kojim namjerava raditi. U slučaju promjene lozinke, ona se mora obaviti na svim računarima. **U drugom obliku, NT domena, u lokalnoj mreži postoji jedan računar koji obavlja poslove prijave i on se zove kontroler domene. Domenu čine svi računari čiji se korisnici prijavljuju preko istog kontrolera.** Nakon što se korisnik prijavi sa radne stanice, kontroler domene provjeri kojoj grupi on pripada i vrati odgovarajuće liste pristupa. Ovdje se korisnička imena i lozinke čuvaju centralizovano, pa se promjena lozinke i dodavanje novih korisnika rade na samo jednom mjestu. Kod većih mreža domenski kontroler postaje usko grlo, jer se svi korisnici prijavljuju preko njega. Često i organizaciona struktura firme ima hijerarhiju koja se ne može uklopiti u linearnu strukturu jednog domenskog kontrolera i mnogo računara povezanog na njega. Stoga je od Windows 2000 uveden **aktivni direktorij, imenski servis koji čuva informacije o objektima. Ti objekti su korisnici, računari, štampači i domene. Podaci u njima se čuvaju kroz hijerarhijski prostor imena, u odvojenim serverima za pojedine dijelove mreže, ali koji i dalje mogu da sarađuju po potrebi.** Od ove verzije Windows unaprijeđeni su i kriptografski protokoli prijave, Kerberos i TLS (Transport Layer Security).

3.1. Klasifikujte uređaje po namjeni, smijeru i količini podataka.

Prema namjeni uređaji se smještaju u slijedeće opšte kategorije: - uređaji za dugotrajno smještanje podataka (engl. storage devices), - uređaji za prenos podataka (engl. transmission devices), - uređaji koji obezbjeđuju interfejs ka korisniku (engl. human interface devices). Prema smijeru prenosa uređaje dijelimo na ulazne (npr. miš, skener), izlazne (npr. printer) i ulaznoizlazne (mrežna kartica). Prema jediničnoj količini prenesenih podataka uređaji se dijele na blokovske i znakovne uređaje. Izdvajamo mrežne uređaje kao specijalnu klasu. Ovakva podjela je uslovljena razlikama između ove dvije vrste ulaznih i izlaznih uređaja u pogledu: jedinice pristupa (blok za blokovske i znak za znakovne), načina pristupa (blokovski imaju direktan, a znakovni sekvencijalan pristup) i upravljanja. Ova podjela ipak ne obuhvata sve uređaje.

3.2. Koje vrste registara U/I uređaja postoje?

Registri ulazno-izlaznog uređaja dijele se u četiri grupe

- Kontrolni registri: Upis u ove registre postavlja režim rada perifernog uređaja. Na primjer, ako je uređaj zvučna kartica, upis u jedan od njenih kontrolnih registara postavlja jačinu zvuka.
- Statusni registri: Ovi registri se isključivo čitaju i opisuju status uređaja. Primjer predstavlja registar štampača koji indicira da nema papira.
- Ulazni registri podataka: Procesor ove registre isključivo može čitati i oni omogućavaju čitanje podataka sa perifernog uređaja. Na primjer, ako je uređaj disk, i nakon što su kontrolnim registrima izabrani sektori diska koji se žele čitati, uzastopnim čitanjem ulaznog registra podataka se čitaju bajtovi koji su stigli sa diska
- Izlazni registri podataka: Procesor u ove registre isključivo može pisati i oni omogućavaju slanje podataka perifernom uređaju. Na primjer, slanje narednog slova štampaču obavlja se upisom u njegov izlazni registar.

3.3. Koja je uloga kontrolera prekida?

Kontroler prekida šalje signal prekida procesoru preko prekidne linije (engl. interrupt request line, IRQ). Desiće se sljedeći događaji:

1. U/I kontroler šalje signal kontroleru prekida
2. Kontroler prekida šalje IRQ signal procesoru
3. Procesor završava izvršenje tekuće instrukcije.
4. Procesor potvrđuje prekid.
5. Procesor spašava svoje trenutno stanje (programski brojač i još neke registre)

6. Programski brojač se postavi na sekvencu instrukcija koje se bave prekidom.

Ukoliko se više prekida desi istovremeno, kontroler prekida donosi odluku čiji prekid obraditi na bazi prioriteta prekida.

3.4. Šta je to DMA kontroler?

U/I vođen prekidima i programirani U/I obavljaju transfer podataka kroz CPU, na čemu se troši mnogo vremena. Za razliku od njih, DMA (direktan pristup memoriji) je strategija koja koristi dodatni hardver za prijenos podataka – DMA kontroler, koji može preuzeti sistemsku sabirnicu i prenijeti podatke između U/I kontrolera i memorije bez uključivanja CPU u sam proces. Kada CPU želi prenijeti podatke, on to također obavlja uz pomoć DMA kontrolera.

3.5. U kojim režimima radi DMA kontroler?

- **Burst režim:** Čitav blok podataka se prenosi kroz jednu kontinualnu sekvencu. Kada je DMA kontroler dobio pravo na sistemsku sabirnicu, on prenosi sve podatke u bloku podataka (npr. 512 bajta) blokirajući procesor. Ovaj prijenos je najbrži, ali zadržava procesor prilično dug period vremena.
- **Režim krađe ciklusa:** U ovom režimu DMA blokira procesor prije prijenosa svakog bajta i odblokira ga nakon što je bajt prebačen. Time procesor nije mnogo neaktivan, ali je prijenos podataka manje brz nego u burst režimu rada.
- **Transparentni režim** zahtijeva najviše vremena za prijenos podataka, ali ne zaustavlja procesor, nego prenosi podatke isključivo kada procesor ne pristupa sabirnici. Realizacija ovog režima može biti dosta kompleksna.

3.6. Razlika između memorijski mapiranog U/I i U/I s odvojenim adresnim prostorom.

Sa memorijski mapiranim pristupom, CPU vidi U/I kontroler kao običnu skupinu lokacija u memoriji. Da bi poslao podatke U/I kontroleru, CPU piše ili čita podatke iz ove lokacije u memoriji. Ovo će smanjiti dostupan adresni prostor memorije. Sa U/I mapiranim pristupom, čini se da U/I kontroler okupira svoj vlastiti adresni prostor i koriste se specijalne instrukcije za komunikaciju sa njim. Ovo daje više adresnog prostora i za memoriju i za U/I, ali će povećati ukupni broj instrukcija. Takođe može smanjiti fleksibilnost sa kojim CPU adresira U/I kontrolere. Mnogi računari koriste hibridnu šemu, jer je neke uređaje praktičnije programirati kada su memorijski mapirani (obično ekran), a neke kada su U/I mapirani (npr. serijski port).

3.7. Kako se rješava problem u obrađivačima interapta: cilja da bude što brži i uradi što više?

Rješava se razdvajanjem obrađivača u dva dijela: gornja polovina i donja polovina obrađivača. U toku gornje polovine obrađivača obavljaju se poslovi koji su vremenski kritični: potvrda prijema prekida ili promjena stanja nekih registara kontrolera perifernog uređaja. Nakon ove faze novi prekidi se dozvoljavaju, a jezgro ili korisnički program mogu nastaviti od mjesta gdje su prekinuti. Ovo može biti i mjesto kada se poziva raspoređivač procesa. Donja polovina se izvršava kasnije, u pogodnijem trenutku, sa uključenim prekidima, kada se tipično obrađuju podaci stigili od perifernog uređaja.

3.8. Koja je glavna razlika u funkcijama drajvera blokovskih i znakovnih uređaja?

Glavna razlika između drajvera blokovskih uređaja i drajvera znakovnih uređaja je u operacijama čitanja i pisanja. Dok se znakovnim uređajima podaci šalju ili primaju od njih u toku poziva *read* ili *write*, zahtjevi za prijem podataka od blokovskih uređaja (ili slanje prema njima) se stavljaju u red čekanja za asinhroni pristup.

3.9. Koja su tri osnovna pristupa u realizaciji čitanja i pisanja kod drajvera znakovnih uređaja i po čemu su karakteristični?

Kod drajvera znakovnih uređaja, operacije čitanja i pisanja moguće je realizovati na tri osnovna načina.

- Programirani ulaz/izlaz **komunikacija prozivanjem** je najjednostavnija metoda za komunikaciju između procesora i perifernog uređaja. U ovom pristupu, procesor je odgovoran za svu komunikaciju sa uređajem, koristeći čitanje i upis u registre uređaja.
- Ako uređaj podržava prekide, komunikacija sa njim postaje znatno efikasnija. **U/I vođen prekidima** ne zahtijeva procesorsko prozivanje da li je uređaj spreman, nego uređaj to sam javlja. Rutina za slanje podatka uređaju bi se sada sastojala iz dva dijela, sinhronog i asinhronog. Sinhroni dio je u *driver_write* rutini drajvera i poziva se kada je korisnički proces pozvao rutinu *write*. Ova rutina prekopira podatke i uključuje prekide koje obrađuje drajver. Nakon toga pozove se raspoređivač kako bi se program pozivalac blokirao dok svi podaci ne budu poslani. Pretpostavka je da je signal spremnosti uređaja povezan na sistem prekida. Stoga, svaki put kada je uređaj spreman za prijem novog znaka, generirao bi se prekid. U rutini za obradu prekida bi se poslao novi znak, a kada je poslan i posljednji znak, deblokira se korisnički proces.
- Kada se koristi **DMA bazirana komunikacija**, drajver se također dijeli na sinhronu i asinhronu dijelove, kao i kod prekidima bazirane komunikacije. U ovom slučaju se prekidna rutina mnogo rjeđe poziva, u suštini samo pri završetku prijenosa. Poziv upisa podataka prema uređaju će postaviti DMA kontroler (početnu adresu bafera, koliko bajta se prenosi, I/O port kojim se obavlja transfer i pravac prijenosa) i reći mu da počne sa prijenosom podataka. Nakon toga se program pozivalac može uspavati. Kada je prijenos gotov, generiše se prekid koji vraća kontrolu program pozivaocu.

3.10. Šta je to bafer?

Bafer je dio memorije a funkcioniše na principu proizvođač-potrošač i služi za čuvanje privremenih podataka prilikom prijenosa podataka između dva uređaja ili između uređaja i aplikacije. Mali baferi koji se nalaze u kontrolerima uređaja obično nisu dovoljni ako je razlika u brzini između proizvođača (npr. aplikativnog programa) i potrošača (npr. drajvera koji šalje podatke uređaju) znatna. Uvođenjem dodatne memorije podaci se čuvaju kako ih proizvođač generiše i šalju potrošaču kada je on spreman bez prevelikog usporenja proizvođača.

3.11. Šta je to spuler?

Spooler je pozadinski proces najčešće vezan za štampače i crtače. On omogućava istovremeni pristup nedjeljivim uređajima tako što korisnički proces upisuje podatke namijenjene uređaju (npr. komande štampaču) u privremene datoteke na disku. Spooler gleda redoslijed dolaska tih privremenih datoteka i njihov sadržaj šalje redom jednu po jednu prema štampaču. Korištenjem spoolera, proces može brzo postaviti zahtjev u bafer, a nakon toga nastaviti sa drugim aktivnostima.

3.12. Kako se imenuju uređaji u korisničkom prostoru u Linuxu?

Simbolička imena uređaja kreiraju se alatom *mknod* koji poziva istoimeni sistemski poziv. Interni brojevi koji identifikuju drajvere uređaja, koji se definišu unutar drajvera uređaja radi njegovog prepoznavanja, se na ovaj način pridružuju simboličkom imenu.

3.13. Kako se imenuju uređaji u korisničkom prostoru u Windowsu?

Windows NT koristi manje poznat sistem za simboličko imenovanje uređaja. Njegov dio jezgra koji se zove ObjectManager imenuje uređaje interno. Tako na primjer prvi CD ROM uređaj se interno zove `\Device\CdRom0`, ali to ime nije vidljivo korisničkim programima i ne pripada datotečnom sistemu. Uređaji čija imena su dostupna korisničkim programima imaju globalno definirana alternativna imena. Ta imena počinju besmislenim imenom direktorija `\\.\`, kako ne bi greškom tražena u datotečnom sistemu.

3.14. Kako se konvertuje scan kod u ASCII kod?

Drajver tastature očitava *scan* kodove za znakove koji su se sakupili u bufferu. Treba ih konvertovati u razumljive znakove. Na primjer, kada se pritisne taster A, *scan* kod (30) se stavi u registar. Drajver treba da odredi, na bazi ranijeg pritiska ili otpuštanja tastera da li je to a, A, CTRL-A, ALT-A,... U tu svrhu se koristi ASCII tabela, prilagođena različitim nacionalnim jezicima koja se može eksterno učitati.

3.15. Kako promjena podatka u video memoriji utiče na sliku na ekranu?

Slika ekrana je pohranjena u video RAM-u u znakovnom režimu ili bit mapi. U znakovnom režimu, svaki bajt (ili 2 bajta) video RAM-a sadrži jedan znak (obično ASCII) koji se prikazuje. U režimu bit mape, svaki *pixel* (tačkica) na ekranu se predstavlja posebno u video RAM, sa 1 bit po *pixel*-u za najjednostavniju crnobijelu sliku do 24 bita po *pixel*-u za visoku kvalitetu prikaza boja.

3.16. Šta su to ANSI sekvence u ispisu na terminalu?

Neki drajveri za tekstualni ekran podržavaju dodatne mogućnosti koje nisu sastavni dio ASCII skupa znakova. To su ANSI *escape* sekvence i prije svega su namijenjene za precizno podešavanje kursora na ekranu. Njih prihvata terminalski drajver, i imaju posebno ponašanje nakon ESC znaka. To je bajt s ASCII kodom 27.

3.17. Šta je to paleta?

U režimu bit mape se svaki pixel na ekranu individualno kontroliše i predstavlja jednim ili više bitova u video RAM. Koliko bitova je potrebno, zavisi od broja boja koji se želi predstaviti. Sa jednim bitom po pixel-u mogu se imati samo dvije boje na ekranu (npr. crna i bijela) dok se sa n bita po pixel-u može imati 2^n boja. Skup boja se može izabrati i taj skup se zove paleta. Za $n=24$ može se imati oko 16 miliona različitih boja. Svaki pixel tada je predstavljen sa tri bajta koji predstavljaju crvenu, zelenu i plavu komponentu boje. Preko ovih komponenti se na crnoj pozadini ekrana mogu dobiti sve čovjekovom oku vidljive boje.

3.18. Šta radi funkcija WriteConsole u Windows?

Ispisuje tekst na konzolni ekran.

Sintaksa: WriteConsole(hConsoleOutput, lpBuffer, nNumberOfCharsToWrite, lpNumberOfCharsWritten, lpReserved)

3.19. Koja su četiri osnovna dijela Windows API grafičkog programa?

Program se sastoji od četiri osnovna dijela: registracija prozorske klase (WinMain), kreiranje i prikaz prozora (CreateWindow), petlje koja prosljeđuje poruke (WndProc) i prozorske procedure koja ih obrađuje.

3.20. Čemu služi terminal i šta je njegova moderna verzija?

Ranije su mnogo korišteni RS-232 terminali za komunikaciju s mainframe sistemima, bit po bit, preko serijske linije, između kojih se koristi eventualni telefonski modem. Terminal je uređaj vizualno sličan računarskom sistemu, kombinacija monitora, tastature, mikroprocesora i serijske komunikacije, ali njegov mikroprocesor ne izvršava operativni sistem niti korisnički program. Izvršavanje OS i programa obavlja centralni kompjuter, dok terminal samo brine o prijemu, slanju i prikazu podataka. Kompjuter i terminal su potpuno odvojeni, ali terminal ima sposobnost prikaza teksta koji mu se pošalje. Pojeftinjenje personalnih računara je učinilo da se i oni mogu koristiti kao terminali. To je najprije išlo također preko RS232 interfejsa, a nakon toga su računari opremljeni mrežnim karticama i protokolima, što je komunikaciju učinilo daleko bržom.

3.21. Koje su vrste udarnih a koj beskontaktnih štampača?

Udarni: matricni, lepezni i linijski Tehnologija iza matričnih štampača je vrlo jednostavna. U radu se pritisne bubanj (gumeni cilindar) i povremeno povuče prema naprijed kako ispis napreduje. U elektro-magnetskom pogonu glava štampača se kreće preko papira i udara iglicama vrpce štampača koja se nalazi između papira i glave štampača. Glava štampača na traci tiska tačkice tinte na papiru koje čine ljudski - čitljive znakove.

Štampači s lepezom su slični pisaćim mašinama. Glava štampača sastoji se od metalnog ili plastičnog točka isječenog po laticama. Svaka latica ima određen broj slova (velikih i malih slova), brojeva ili interpunkcijskih znakova na njoj. Kada latica pogodi vrpce štampača ovo rezultira ispisom tinte na papir.

Linijski štampači imaju mehanizam koji omogućava da se istovremeno štampa više znakova na istoj liniji. Mehanizam može koristiti veliki rotirajući bubanj ili papir u lancu petlje. Kada se bubanj ili lanac rotira preko površini papira, elektromehanički čekići iza papira gurnu papir (zajedno sa trakom) na površinu bubnja ili lanca, iscrtavajući lik na površini bubnja ili lanca.

Beskontaktni: termalni, inkjet, laserski, sa sublimacijom boje, termalni vosak i čvrsta tinta.

Termalni štampači koriste specijalni papir osjetljiv na temperaturu ili električni naboj. Glava štampača na pojedinoj tački papira zagrije to mjesto grijačem ili električnom varnicom, nakon čega papir na tom mjestu blago izgori i pojave se tamne tačke.

Inkjet štampač koristi jednu od najpopularnijih štampe tehnologije danas. Relativno niske cijene i višenamjenski ispis čine da su inkjet štampači dobar izbor za male firme i kućne urede. Inkjet štampači koriste boje na bazi vode koje se brzo suše i glavu štampača s nizom malih mlaznica koja rasprskava tintu na površini papira. Inkjet štampači mogu imati jednu mlaznicu (crno-bijeli ispis) ili četiri mlaznice koje predstavljaju boje cijan, magenta, žuta i crna.

Laserski štampači su poznati po velikim mogućnostima izlaza i niskim troškovima po stranici. Laserski štampači dijele mnogo istih tehnologija kao fotokopirni aparat. Valjci povuku list papira iz ladice za papir i kroz valjak, što daje papiru elektrostatički naboj. U isto vrijeme ispisni bubanj ima suprotno naelektrisanje. Površina bubnja se zatim skenira laserom, isprazni površina bubnja i ostave samo one tačke koje odgovaraju željenom tekstu i slikama. Ovaj naboj se zatim koristi da prisili toner da se zadrži na površini bubnja. Papir i bubanj se zatim dovode u kontakt.

Štampači termalnog voska imaju pojas za pogon CMYK trake veličine lista papira i specijalno - premazani papir ili folije. Glava štampača sadrži grijače kojim se topi vosak boja na papir, dok on ide kroz štampač. Štampači s sublimacijom boje su slični štampačima termalnog voska, osim što koriste film boje difuzne plastike umjesto obojenog voska. Glava štampača zagrijava film u boji i isparava sliku na posebno premazani papir. Štampači čvrste boje su cijenjeni zbog svoje sposobnosti za ispis na raznim vrstama papira, što se koristi za izradu prototipova ambalaža. Ovi štampači koriste tvrdi tintu koja se istopi i prska kroz male mlaznice na glavama štampača. Papir se zatim šalje prema valjku grijaču što dodatno tjera tintu na papir.

3.22. Navedite primjere jezika za opis stranica.

ESC/P. Riječ je o ASCII ispisu teksta uz pomoćne kodove za definiranje vlastitih slova, ispis kosim slovima, podebljano ili ispis grafike.

U firmi Adobe, je razvijen PDL zvani PostScript, jezik za opisivanje i oblikovanje teksta i slika koje može obraditi štampač. Neobična osobina ovog jezika je što se za svaku njegovu naredbu prvo navode njeni parametri, pa onda sama naredba

Hewlett-Packard je razvio Printer Command Language (ili PCL) za upotrebu u svojim laserskim i inkjet štampačima. Ovaj jezik, kao i ESC/P umjesto komandi koristi specijalne ESC kodove.

3.23. Na koji način Windows priprema stranicu za štampu?

Na Windows gotovo sve aplikacije koriste GDI, koji se može koristiti i za štampače, tako da je generiranje izgleda stranice na štampaču jednostavno. Kontekst uređaja štampača se dobije API pozivom StartDoc i koriste se pozivi za crtanje, poput TextOut i BitBlt. Zapamćene GDI komande se čuvaju u metadatoteci. Nakon funkcije EndDoc, privremena metadatoteka se dalje konvertuje u jezik štampača. Dobiveni podaci u jeziku štampača se pošalju programu spooler-u (spoolsv.exe).

3.24. Šta je CUPS?

CUPS (Common Unix Printing System). CUPS je više od spool programa. To je kompletan sistem za upravljanje štampačima. Poznaje novi Internet Printing Protocol (IPP). Može se konfigurisati web, GUI i komandnim sučeljem. Opremljen je programima filtrima koji prevode različite tekstualne i grafičke formate u PostScript jezik, koji se može direktno slati takvim štampačima. Ako štampač nije PostScript tipa, programima CUPS-raster se konvertuje PostScript u format prilagođen štampaču.

3.25. Koji principi dodirnih ekrana postoje?

Kod rezistivnog dodirnog ekrana, ekran je napravljen od dva sloja provodnog materijala. Jedan sloj ima vertikalne, drugi horizontalne linije. Kada se

pritisne gornji sloj on dođe u kontakt s drugim koji dopusti tok struji. Odgovarajuće horizontalne i vertikalne linije određuju poziciju na ekranu koja je dodirnuta.

Kapacitivni dodirni ekran je napravljen od laminata preko cijelog staklenog ekrana. Laminat provodi struju u svim pravcima, a vrlo mala struja ide iz sva četiri ugla. Kada se ekran dodirne, struja teče u prst ili olovku. Lokacija dodira se odredi poređenjem koliko je jak tok elektriciteta iz svakog ugla

Infracrveni dodirni ekran je ekran s unakrsnim horizontalnim i vertikalnim zracima IC svjetla. Senzori suprotnih strana ekrana detektuju zraku. Kada korisnik prekine zraku dodirom ekrana, može se odrediti lokacija prekida. Ekran sa akustičkim površinskim valom (SAW) emituje visoko frekventne zvučne talase horizontalno i vertikalno. Kada prst dodirne površinu, senzor prepozna prekid i odredi lokaciju dodira.

3.26. Koji tri vrste naredbi poznaju interpreteri komandi?

Naredbe su obično jedna od tri klase. Interne naredbe prepoznaje i obrađuje sam interpreter komandne linije i ne ovise o bilo kojoj vanjskoj izvršnoj datoteci. Uključene naredbe su izdvojene izvršne datoteke koja se općenito smatraju dijelom operativnog sistema i uvijek su uključene u OS. Vanjske naredbe su izvršne datoteke koje nisu dio osnovnog OS, ali su dodane naknadno od strane korisnika.

3.37. Šta je zapis u formi crijeva?

LEKCIJA 11.1 – UREĐAJI SEKUNDARNE MEMORIJE

Odgovor: Veže se za hard diskove. Od 2014. godine gustoća zapisa(staza) na hard disku se počinje praviti u obliku(formi) crijeva. Kod forme crijeva se upis u jednu stazu preklapa dijelom i na dvije susjedne staze. Inače ovo bi trebalo usporavati sam upis podataka, jer se pri ažuriranju staze moraju pročitati i obje susjedne staze, a nakon

toga upisati podatke na sve tri staze. Ipak, pošto moderni Hard Diskovi imaju veliku količinu RAM memorije u kojoj će se nalaziti nedavno čitani podaci i specijalni softver, ovo usporenje se neće primjetiti.

3.38. Koja je Formula prosječnog vremena pristupa diska?

LEKCIJA 11.1 – UREĐAJI SEKUNDARNE MEMORIJE

Vrijeme prijenosa podataka na disk i sa diska zavisi od rotacione brzine i računa se po sljedećoj formuli:

$$T = \frac{b}{rN}$$

gdje je T - vrijeme prijenosa, b - broj bajta koji trebaju biti preneseni, N - broj bajta na stazi, r - rotaciona brzina. Kod tehnologije zona broj bajta po stazi je varijabilan a to komplikuje računanje.

Finalno je : Ukupno prosječno vrijeme pristupa može biti izraženo sa:

$$T_a = T_s + \frac{1}{2r} + \frac{b}{rN}$$

gdje je Ts - prosječno vrijeme traženja.

3.39. Šta je formatiranje na niskom nivou?

Formatiranje na niskom nivou je proces kod koga se na magnetni medij upisuju oznake koje predstavljaju granice staza i sektora. Na nekim vrlo starim diskovima (MFM, raniji SCSI) radio ga je kupac, ali se danas isključivo radi fabrički.

Opširniji odgovor:

U toku ovog formatiranja pripreme se mjesta za sektore, staze i međuprostor između njih. Sektor se sastoji od zaglavlja za njegovo prepoznavanje (Preamble), samog sadržaja i podataka za korekciju grešaka sektora (ECC) (Slika 122). Ovu strukturu vidi samo kontroler diska, prema računaru se proslijeđuje isključivo blok sa podacima.

Naredni zadatak je dijeljenje diska na jednu ili više particija koje se ponašaju kao zasebni diskovi. **Više particija se kreira kada se želi instalirati više operativnih sistema ili ako se žele razdvojiti sistemski podaci od korisničkih. Lista svih particija nalazi se na samom početku diska (sektor 0, staza 0, površina 0). Ovaj sektor se naziva MBR, glavni startni zapis (engl. Master Boot Record). Pri pokretanju računara, BIOS nakon inicijalizacije memorije pronađe MBR na disku. Unutar MBR se nalazi kratki program koji pročita tabelu particija. Jedna među njima se zove aktivna. BIOS pročita MBR sa diska i pokrene izvršavanje u njemu programa, koji pročita prvi sektor aktivne particije.** Taj sektor se zove startni sektor (engl. boot sector). U njemu se takođe nalazi mali program čijim pokretanjem započinje bootstrap rutina odnosno učitavanje operativnog sistema u memoriju. Na kraju MBR se nalaze tabela particija i signatura MBR. Tabela particija sadrži početnu i krajnju adresu svake particije, na PC računarima ima četiri elementa po 16 bajta. Treća faza pripreme diska je formatiranje particija. U ovoj fazi se sektori koji pripadaju pojedinim particijama pune početnim vrijednostima koje su potrebne za realizaciju tabela za datotečne sisteme, ili particije za realizaciju virtualne memorije. Na primjer odredi se koji sektori u particiji će čuvati podatke o slobodnom prostoru, u kojima će biti sadržaj datoteka, a koji će govoriti o tome gdje se nalazi pojedina datoteka. Nekada se upiše i startni sektor.

3.40. Koja je uloga Master boot record?

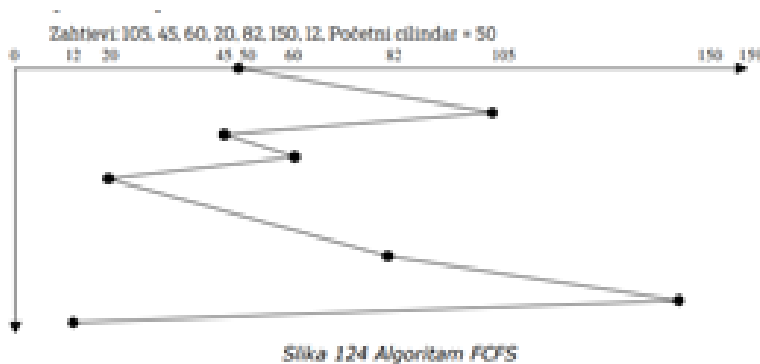
Podebljani dio teksta u prethodnom pitanju ujedno je odgovor na ovo pitanje.

3.41. Kada se koriste preplitanje i zakretanje?

Kada se čita sektor po sektor, računar ima vremena da obradi primljeni sektor dok je glava diska u međuprostoru između sektora. Disk se u međuvremenu još uvijek okreće i na sporijim računarima može se desiti da je glava diska došla na naredni sektor prije nego je završen prijenos prethodnog sektora u bafer. Kada dođe zahtjev za čitanje narednog sektora, morala bi se sačekati cijela rotacija diska. Da bi se to izbjeglo, koristi se prepletanje (Slika 123b). U ovoj tehnici logički sektori se ne redaju sekvencijalno u fizičke, nego se preskače po jedan ili dva sektora.

3.42. Algoritam za raspoređivanje diska FCFS.

Razmotrimo sljedeći primjer. Pretpostavimo da disk ima 160 cilindara i jednu glavu, i da imamo sekvencu zahtjeva za čitanjem cilindara 105, 45, 60, 20, 82, 150, 12. Pretpostavimo da je glava za čitanje/pisanje inicijalno postavljena na stazi 50. Šta bi se desilo da drajver diska opslužuje zahtjeve upravo tim redoslijedom, kao što je u najjednostavnijem algoritmu FCFS (first come, first served)? Ovaj algoritam, iako ravnopravno tretira sve zahtjeve, daje slabije performanse. Performanse zavise od pokretanja glave za čitanje i pisanje, a performanse će opadati ukoliko glava mora preći veći put.



Slika 124 prikazuje dijagram kretanja glave diska, gdje horizontalna osa predstavlja cilindar, a vertikalna vrijeme. U ovom primjeru glava za čitanje/pisanje će preći 438 cilindara. Zdravorazumski i poštenu algoritam očito ima loše performanse.

3.43. Algoritam SSTF.

Ako se između pristiglih zahtjeva uvijek bira cilindar koji je najbliži trenutnoj poziciji glave, dobiva se SSTF algoritam (shortest seek time first). Postižu se bolji rezultati, za isti primjer kao kod FCFS, pređeni put je 248 cilindara (Slika 125). Pa ipak, lokalno najbliži cilindar ne znači najkraći ukupno pređeni put. Na slici se vidi da je glava diska preko cilindra 45 prešla dva puta. Drugo, ovaj algoritam, ima problem izgladnjivanja (engl. starvation). Ako stalno dolaze

novi zahtjevi u red čekanja, glava se može zadržati dugo u jednoj zoni opslužujući zahtjeve za koje je potrebno malo kretanja, tako da se može desiti dugo čekanje na opsluživanje zahtjeva prema udaljenim cilindrima.

3.44. Algoritmi SCAN, LOOK i S-LOOK.

U velikim poslovnim zgradama i tržnim centrima u liftove ulaze novi ljudi na među-spratovima. Ako se lift kreće prema gore i ima još putnika u liftu u tom pravcu, nepošteno bi bilo da pridošlice promijene pravac lifta, nego oni moraju sačekati da lift završi sa penjanjem, pa tek onda početi spuštanje. SCAN algoritam zbog sličnosti sa ovim ponašanjem lifta se često naziva i elevator algoritam

Glava diska se naizmjenično pomiče od početnog cilindra na ploči ka krajnjem, pa onda od krajnjem ka početnom. Kada prelazi iznad pojedinih cilindara opslužuje zahtjeve prema njima. Ovaj algoritam rješava problem izgladnjivanja. Za spomenuti primjer, pređeni put je 256 cilindara (Slika 126). Unapređenje SCAN algoritma je LOOK algoritam. Kod ovog algoritma se promjena pravca ne obavlja na krajevima ploče, nego kada su opsluženi svi zahtjevi za cilindrima u datom smjeru.

U osnovnom LOOK algoritmu, početni smjer kretanja glave se preuzima od smjera u kome se glava diska kretala pri obradi ranijih zahtjeva. Varijanta S-LOOK određuje početni smjer da krene prema najbližem cilindru. Dalje se nastavlja kretanje u istom smjeru do najdaljeg zahtjeva, nakon čega se smjer mijenja.

3.45. Algoritam C-SCAN i C-LOOK.

Kod klasičnog SCAN algoritma, zahtjevi za cilindrima koji su u sredini ploče će biti češće opsluživani nego zahtjevi prema cilindrima na krajevima diska. Kružni SCAN algoritam (C-SCAN), smanjuje favorizovanje centralnih cilindara. Kod ovog algoritma se zahtjevi opslužuju samo u jednom smjeru. Uzmimo za primjer da je taj smjer prema većim brojevima cilindra. Kad glava dođe do posljednjeg cilindra na ploči, vrati se na nulti, pri čemu tokom povratka ne opslužuje zahtjeve.

Neki diskovi imaju mehanizam za brzo vraćanje glave. Uz raniji primjer, ako se povratni put ne računa, (na nekim diskovima se može povratak realizovati dosta brzo), pređeno je 154 cilindra (Slika 128). C-LOOK je varijanta algoritma LOOK koja opslužuje zahtjeve u jednom smjeru do posljednjeg zahtjeva u tom smjeru. Tada nastavlja od najdaljeg zahtjeva na drugoj strani ploče i ponovo opslužuje zahtjeve u istom smjeru. Ako se povratni put ne računa, CLOOK algoritam ima najkraći pređeni put (133 cilindra).

3.46. Koji je razlog keširanja diskova?

Razlog keširanja diskove je taj što je keširanje najefikasnija metoda za ubrzavanje pristupa disku ukoliko računar raspolaže dovoljnim RAM-om. U ovom kontekstu, keš predstavlja skup blokova koji logički pripadaju disku ali se drže u RAM, jer je RAM brži i od mehaničkih i od SSD diskova.

3.47. Koji su načini zamjene lošeg sektora rezervnim?

Dva su načina mapiranja lošeg sektora u rezervni. Prvi je razrješenje premošćenjem, uz preskakanje defektnog sektora. Ovaj način zahtijeva kopiranje svih sektora u stazi iza defektnog za jedan sektor naprijed ali je kasnije čitanje staze ostalo brzo. Drugi način je smještanje sektora u rezervni sektor, no ubuduće se ta staza neće moći čitati u toku jedne rotacije diska. Loše sektore može zamijeniti disk kontroler ili OS. U slučaju da to radi kontroler,

on mijenja preambule sektora na stazi, dok operativni sistem, pravi drugačije tablice, često definisan datotečnim sistemom.

3.47. Koji su načini zamjene lošeg sektora rezervnim?

Greške se prepoznaju razlikom upisane i pročitane vrijednosti I tada se sektor proglašava neispravnim I izbacuje iz upotrebe.

Posao se ponavlja aktiviranjem rezervnog sektora.

Prije formatiranja particije, nekoliko sektora se ostavlja za rezervu.

Dva postupka:

- Premosćenje uz preskakanje defektnog sektora (kopiranje svih sektora iza defektnog za jedan sektor naprijed, citanje staze ostaje brzo)
- Smještanje u rezervni sektor ()

Lose sektore mijenja disk kontroler (mijenja preambule) ili OS (pravi drugacije tablice)

3.48. RAID 0.

RAID 0 nema dodatne informacije za obnovu podataka u slučaju pada. Koristi sekada je potrebno povećati kapacitet i brzinu rada diskova.

Podaci se raspoređuju na različite diskove. Svaki fizički disk

ima svoj red čekanja na zahtjeve, koji mogu biti obrađeni nezavisno, pa ima bolje performanse. Disk je podijeljen na odsječke, koji predstavljaju skupove disk sektora.

Odsječci se redom postavljaju na svaki od fizičkih diskova. Npr., ako je RAID 0 sastavljen od četiri fizička diska, logički odsječci 0, 4, 8, 12... će se smjestiti na jedan disk, odsječci 1, 5, 9, 13... na drugi, odsječci 2, 6, 10, 14... na treći a odsječci 3, 7, 11, 13,... na četvrti fizički disk.

Transformacija logičkog u fizički sektor dobija dodatnu koordinatu: sektor se transformiše u četvorku (disk, cilindar, glava, sektor).

Nema redundancije, otkaz bilo kog diska znači gubitak svih podataka

3.49. RAID 1.

RAID 1 ili ogledalo svi diskovi imaju identičan sadržaj.

Svaki upis sektora na jednom reflektuje se na drugi disk. Upis se obavlja disk kontrolerom paralelno, pa nema gubitka vremena.

RAID 1 ubrzava čitanje jer se zahtjev može proslijediti bilo kom diskova koji sadrže iste podatke. Pouzdanost vrlo visoka (od n može opasti n-1 diskova). Mana je cijena I stvarna iskoristenost (2 diska 50%, za 4 je 25%).

3.50. RAID 4 i 5.

RAID 4

Koristi posvećeni disk pariteta kao i RAID 3, ali podatke po ne raspoređuje po pojedinačnim bajtovima, nego po odsječcima veličine sektora. Lako uključivanje klasičnih diskova u RAID konfiguraciju. Za N odsječaka, na N diskova, dovoljan je jedan blok pariteta veličine odsječka. Čitanje ima dobre performanse, pisanje sporije nego RAID 1 ili 0, jer svaki ciklus upisa učestvuje i disk pariteta.

RAID 5

Unapređenje RAID 4. Razlika je što ne čuva sve odsječke sa paritetima na istom disku, nego se neki

odsječci čuvaju na jednom, a neki na drugim diskovima. Koristi se cirkularna šema alokacije. Uz RAID 1, RAID 5 je najpopularniji jer ima dobar odnos cijene, brzine, pouzdanosti i paralelizma.

3.51. Kako su organizovani SSD diskovi?

Za zapis informacija se koriste NAND flash i DRAM memorija

NAND flash je MOSFET s 2 gate, od kojih je jedan izolovan pa čuva podatke godinama.

SSD pohranjuju podatke u flash memorijske ćelije koje su raspoređene u nizove po 32-128 bita. Nizovi se smještaju u stranicama od 4 do 16 K.

Stranice se grupišu u blokove obično 128 do 512 stranica. Blokovi u pakete.

NAND memorijske ćelije mogu se direktno upisati samo kada su prazne, u suprotnom sadržaj se briše prije pisanja. Pisanje podataka na prazne stranice je brzo, ali usporeno ako je stranica popunjena. Vrijeme života se mjeri se brojem ciklusa brisanja.

3.52. Kako je organizovana staza na CD-ovima?

CD dizajniran za čuvanje digitalizovane muzike i čuvanje podataka koji se ne mijenjaju. Sadržaj CD-a je podijeljen na sektore, koji su poredani sekvencijalno u jednu spiralu, njihova veličina je 2352 bajta. CD se pravi busenjem rupa laserom na master disku od specijalnog stakla.

Kalup je napravljen sa zaprekama sa rupama u koje se izliva smola i ima isti uzorak rupe kao stakleni disk. Na smolu se stavi aluminijum. Rupice i ravnine su raspoređeni u spiralama. Laser ih čita i prepoznaje kao (0 i 1).

3.53. Šta su to datotečni atributi?

Datotečni atributi su dodatni podaci o datoteci (ime, sadržaj, velicina, način pristupa sadržaju)

Lista atributa zavisi od sistema i dokumentacije.

NTFS (sadrži sadržaj datoteke i slovodni prostor na disku)

FAT (samo šest bitova pristupa)

Grupe atributa su:

- Imenovanje (ime, ekstenzija)
- Lociranje (početni blok)
- Prava (pravo pristupa, lozinka, kreator. . .)
- Pristup (read only, skrivena, arhivirana. . .)
- Struktura (ASCII/binarna, kompresovana, kriptografska . . .)
- Slog struktura (dužina sloga, pozicija ključa)
- Vremena (vrijeme kreiranja, izmjena . . .)
- Veličina (trenutna, maksimalna veličina)

3.54. Koji su načini realizacije fizičkog sadržaja datoteke?

Niz bajta

- OS ne zna ili ne brine o sadržaju u datoteci,
- značenje bajtima daje korisnički program,

- koristi se za OS opce namjene

Niz slogova

- datoteka organizovana da je najmanja jedinica podatka veca od 1 bajta,
- niz slogova koji svaki ima internu strukturu,
- ideja da citanje vraca jedna sloga, a pisanje upisuje novi,
- koristi se kod Michigan Terminal System, Pick Operating System)

Razliciti tokovi

- koriste moderni OS,
- zasnovano na smjestanju razlicitih tokova nezavisnih bajtova u jednu datoteku,
- u izvrsnim datotekama su dvije kopije,
- svaka predstavlja sekvencu bajtova

3.55. U čemu je razlika između ASCII i binarnih datoteka?

ASCII datoteke se sastoje od linija teksta na kraju kojih se nalazi bajt sa kodom CR. Linije ne moraju biti iste veličine. Datoteke se prilikom prikaza na ekranu razumiju kao čitljivi tekst.

Binarne datoteke su niz bajta i nisu pogodne za štampanje. Pokušaj njihovog ispisa na ekranu obično rezultuje besmislenim sadržajem na ekranu. Njihov sadržaj je smislen, ali nije namijenjen da se čita od strane čovjeka nego od strane aplikativnih programa i operativnog sistema.

3.56. Koji su načini pristupa logičkim slogovima datoteka?

Sekvencijalni pristup

Neke datoteke je moguće čitati samo sekvencijalnim pristupom. Zahtijeva da se čitaju svi bajtovi ili slogovi fiksni redoslijedom od početka. Pristup je uobičajen za ASCII i datoteke na magnetnim trakama. Nije dozvoljeno čitanje od proizvoljne pozicije niti preskakanje određenog broja slogova. Jedino se može postaviti na početak datoteke.

Direktni pristup

Datoteke, čiji se bajti ili slogovi mogu čitati u bilo kojem redoslijedu su datoteke sa direktnim pristupom. Navodi se ključ, iz kojeg se nekom formulom izračuna pozicija u datoteci.

Indeksna organizacija

Zasniva se na posebnom području (indeks) u kome se čuvaju pokazivači na dijelove datoteke prema navedenom ključu. U indeksnom području se nalaze uređeni parovi (vrijednost ključa, pozicija, sloga). Koriste se u bazama podataka zbog brze pretrage. Indeksi mogu sadržati više različitih vrsta ključeva, a organizovani su kao niz uređenih parova ili u obliku stabla.

3.57. Razlozi za mapiranje datoteke u memoriji.

Jedan razlog je u brzini samog sistemskog poziva, jer je manje zahtjevno proslijediti jedan cijeli broj nego, npr. 30 znakova imena. Drugi razlog je što je mnogim programima potrebno otvoriti istu datoteku više puta istovremeno, na primjer prilikom njenog sortiranja praktično je kretati se kroz datoteku s dva različita pokazivača. ?

3.58. Jednonivovski, dvonivovski i hijerarhijski direktorij.

- Jednonivovski: Najjednostavniji oblik direktorija je jedan direktorij koji sadrži sve datoteke u datotečnom sistemu. Kod ranih personalnih računara to je bila uobičajena praksa, jer je postojao samo jedan korisnik, a

kapacitet disketa kao osnovnog medija za čuvanje podataka i nije dopuštao mnogo datoteka. Problem sa jednim direktorijom u sistemu sa više korisnika je što različiti korisnici mogu slučajno dovesti do brisanja datoteka. Na primjer, kada korisnik A kreira datoteku mailbox, a kasnije korisnik B kreira istu datoteku, doći će do uništenja datoteke korisnika A.

- Dvonivovski: Nije potrebno mnogo promijeniti datotečni sistem da se uvede određeno grupiranje datoteka. Može se npr. svakom korisniku dodijeliti direktorij u okviru koga može kreirati svoje datoteke, a u spisku datoteka dodati jedan atribut koji kaže kojem direktoriju datoteka pripada. Na ovaj način ime datoteke koje je izabrao jedan korisnik ne utiče na istoimene datoteke drugih korisnika.
- Hijerarhijski: Često jedan korisnik ima veliki broj datoteka, koje nije moguće lako organizovati unutar jednog direktorija. Pojavljuje se potreba grupisanja datoteka u manje grupe, recimo, direktor želi da izvještaje o prodaji grupiše u jedan direktorij, a putne naloge da čuva u drugom direktoriju. Za ovakav pristup je potrebno imati hijerarhijsku strukturu direktorija, poput stabla. Kod ovog pristupa, svaki korisnik može imati direktorija koliko mu treba tako da datoteke može grupisati na prirodni način.

3.59. Šta se postiže organizacijom direktorija u formi acikličnog grafa?

Neki datotečni sistemi dopuštaju da se datoteka istovremeno nalazi u više direktorija. Ovakav model pruža najviše mogućnosti, jer korisnici mogu da u svoji direktorijima imaju sve datoteke koje su im potrebne, bez trošenja prostora na višestruke kopije iste datoteke koja je potrebna za više korisnika. Datotečni sistemi koji imaju strukturu acikličnog grafa direktorija omogućavaju dijeljene datoteke između direktorija.

3.60. Unix pozivi creat, open, read write.

- Datoteka se otvara pozivom *open*, čiji prvi parametar predstavlja ime datoteke, drugi predstavlja način pristupa i može biti *O_RDONLY* (samo za čitanje), *O_WRONLY* (samo za pisanje), *O_RDWR* (čitanje i pisanje), dok treći predstavlja prava koja se dodjeljuju datoteci (oktalni broj koji se koristi uz *chmod* komandu).
- Nova datoteka se kreira pozivom *creat*, čiji su parametri ime datoteke i prava. Ove dvije funkcije vraćaju datotečni deskriptor koji je parametar pozivima za čitanje, pisanje i pozicioniranje.
- Čita se pozivom *read*, kome su parametri datotečni deskriptor, adresa u memoriji gdje se podaci prebacuju i broj bajta koji se prebacuje.
- U datoteku se upisuje se pozivom *write*, parametri su mu datotečni deskriptor, adresa u memoriji odakle podaci smještaju u datoteku i broj bajta koji se prebacuje.

3.61. Kako je organizovana tipična particija?

Svaka particija je podijeljena u specijalizovana područja čiji broj, veličina i sadržaj se razlikuju za svaki datotečni sistem, ali se ipak mogu grupirati po namjeni.

- Boot područja se obično nalaze na početku particije. U njima se nalazi kratki program koji učitava jezgro operativnog sistema ili, ako je particija s podacima bez operativnog sistema, prijavljuje grešku.
- Područje opisa particije sadrži podatke o datotečnom sistemu, kao što su veličina i pozicija ostalih područja, broju sektora koji čine blok, maksimalnom broju datoteka koje sistem može imati.
- Područje sadržaja datoteka je najvažnije područje i u njemu se nalaze podaci koje pojedine datoteke imaju.
- Područja za praćenje alociranog prostora datoteka opisuju gdje se na disku nalazi sadržaj datoteka.
- Područja za praćenje slobodnog prostora opisuju koji blokovi na disku nisu dodijeljeni datotekama.
- Područja direktorija opisuju sadržaj direktorija u slučaju da oni nisu u području sadržaja datoteka.

3.62. Kontinualna alokacija datoteka.

Kontinualna alokacija ne zahtijeva posebno područje za evidentiranje blokova dodijeljenih datoteci. Evidencija se svodi na zapisivanje samo dva broja: adresa na disku prvog bloka datoteke i broj blokova u datoteci. Iz pozicije prvog bloka u datoteci, jednostavnim operacijama se može doći do željenog bloka. Performanse kod čitanja su izuzetno dobre, jer cijela datoteka može biti pročitana u samo jednoj operaciji čitanja, pošto disk kontroler poznaje čitanje više sektora odjednom. Jednako brzo je i pisanje koje ne mijenja dužinu datoteke. Međutim, kontinualna alokacija se ponaša loše kada podatke treba često mijenjati. U početku, ova tehnika alokacije ne predstavlja problem, jer se datoteke uvijek pišu na početku praznog prostora za disk. Ako treba datoteku produžiti, a iz nje se odmah nalazi druga datoteka, moramo tada datoteku koju produžujemo prebaciti na drugo slobodno mjesto.

3.63. Alokacija datoteka ulančanim listama sa pokazivačima u blokovima.

Kod ove alokacione tehnike, svaki blok sa podacima pored sadržaja dijela datoteke ima na svom početku ili kraju i broj bloka u kome se datoteka nastavlja. Sa ovom alokacijom nema problema sa traženjem dovoljno velikog kontinualnog bloka, jer bilo koji slobodan blok može biti iskorišten za produživanje datoteke. Dovoljno je da se u elementu direktorija smjesti samo pozicija početnog bloka, a da se ostalim blokovima može prići koristeći listu. Pristup datotekama se obavlja sekvencijalno, a u slučaju da pristupamo datoteci slučajnim pristupom, pristup je izuzetno spor, jer je potrebno pročitati sve blokove prije traženog bloka da bi se došlo do njega.

3.64. Alokacija datoteka ulančanim listama sa pokazivačima u tabeli.

Alokacija sa ulančanim listama ima priličnu sporost u pristupu proizvoljnom bloku datoteke, jer se moraju pročitati svi prethodni blokovi, jer su pokazivači na nastavak lanca blokova smješteni u samim blokovima. Ako se pokazivači u lancu izdvoje u posebno područje particije, tada će prolazak kroz lanac biti znatno brži jer će biti moguće učitati više pokazivača (po mogućnosti sve) u RAM, a čitanje RAM-a je oko 10 000 puta brže od čitanja sa diska. Tabela u kojoj su smješteni ovi pokazivači zove se FAT /File allocation table). Kao u prethodnoj alokacionoj tehnici, dovoljno je imati pokazivač na prvi blok datoteke koji se drži u elementu direktorija bez obzira na veličinu datoteke. Osnovna mana, kod ove alokacione tehnike, je što cijela tabela mora biti u memoriji cijelo vrijeme, ako se želi postići direktni pristup ili ubrzati sekvencijalni.

3.65. Datotečni sistem s indeksnim čvorovima.

Indeksna alokacija na više nivoa primijenjena je u datotečnim sistemima kod svih operativnih sistema iz porodice Unix sistema. Jedno područje diska sadrži indeksne čvorove, relativno male veličine, po jedan čvor za svaku datoteku. Unutar indeksnog čvora se ne čuva ime datoteke, ali se čuvaju podaci o njenoj veličini, vremenima kreiranja i pokazivačima na blokove. Zavisno od podvarijanti datotečnog sistema, zaglavlje datoteke sadrži više pokazivača na blokove podataka (npr. 10), te pokazivače na blokove pokazivača, koji mogu biti jednostruko indirektni, dvostruko indirektni i eventualno trostruko indirektni pokazivači

3.66. Ekstentna alokacije blokova na disku.

Neki datotečni sistemi kombinuju indeksnu alokaciju na jednom nivou i kontinualnu alokaciju. Umjesto da pokazivači indeksnog čvora pokazuju na blokove fiksne veličine, oni pokazuju na kontinualne nizove blokova promjenjive veličine, koji se zovu ekstenti. Tako se u području direktorija čuva za svaki eksten početni blok i koliko blokova on zauzima. Postojanje velikih kontinualnih blokova predstavlja malu fragmentaciju u smislu fizičke pozicije bloka. Mana ove alokacije je potencijalna eksterna fragmentacija kada disk postane skoro pun, pa se ne može pronaći dovoljno veliki kontinualni blok. Pored toga, potrebno je rezervirati više prostora u svakom elementu direktorijske strukture, nego kod drugih načina alokacije.

3.67. Navesti po primjer alokacija kontinualne, spregnute, ekstentne, indeksne na jednom nivou, indeksne na više nivoa.

Kontinualna alokacija - ISO 9960 FS

Spregnuta alokacija - u operativnom sistemu KERNAL za Commodore 64

Indeksna na jednom nivou - CP/M datotečni system

Indeksna na više nivoa – Unix datotečni system

Ekstetna - NTFS (engl. New Technology File System) standardni microsoftov datotečni system

3.68. Kako Windows 98 FAT pamti duga imena datoteka?

Da bi se dobilo dugo ime (npr. "Istraživanje proizvodnje nafte.doc") bilo je potrebno zapisati ga u posebne elemente direktorija, upisane ispred osnovnog imena u obrnutom redoslijedu . Pri tome se ime datoteke kodira koristeći Unicode, kako bi se omogućila i nacionalna slova u imenima. Polje atributa svakog elementa sa dugim imenom sadrži vrijednost 0x0f, koja je za prethodne sisteme nečitljiva i biće ignorirana ako se čita sa starim sistemima (na disketi na primjer). Najviši bit u polju rednog broja imena kaže sistemu koji je posljednji komad imena.

3.69. Koja je uloga atributa u NTFS?

U Unix-u, datoteka se predstavlja nizom bajta, dok je kod NTFS-a, datoteka skup atributa i svaki atribut predstavlja niz bajta. Aplikativni programi, kroz API pozive, datoteke i dalje vide kao nizove bajtova.

Svaki atribut ima dalje zaglavlje i sadržaj. Zaglavlje atributa određuje njegov tip (numerički identifikator), veličinu i ime (Unicode UTF 16), flegove da li je kompresovan i šifrovan. Neki važniji atributi s navedenim identifikatorom su: \$FILE_NAME (48) za ime datoteke, \$BITMAP (176) za slobodne blokove, \$DATA (128) sa sadržajem, \$INDEX_ROOT (144) za opis direktorijske strukture, \$INDEX_ALLOCATION (160) za elemente direktorijske strukture. Tijelo atributa se može smjestiti u samu MFT datoteku ili biti eksterni atribut (pokazuje na blok). Tijelo atributa može biti i sam sadržaj datoteke. Za kratke datoteke (do 700 bajta) podaci se mogu staviti i u sam MFT. Za duže datoteke, u zaglavlju \$DATA (tip 128) atributa polje runlist pokazuje na listu ekstenta. Kako se u opisu datoteke može nalaziti više \$DATA atributa, NTFS datoteka.

3.70. Koji su načini evidencije slobodnih blokova na disku?

Pet metoda se koristi u ovu svrhu. Prva se sastoji od korištenja ulančane liste diskovnih blokova, pri čemu svaki blok drži brojeve slobodnih diskovnih blokova. Sa 1 KB blokom i 32 bitnim brojem diskovnog bloka, svaki blok može držati brojeve 255 slobodnih blokova, pri čemu je jedno mjesto potrebno za pokazivač na sljedeći blok. Jedan disk veličine 256 GB treba listu slobodnih blokova od maksimalno 1,052,689 blokova da bi držao svih 228 brojeva diskovnih blokova. Često se slobodni blokovi koriste da bi držali listu slobodnih blokova. Kada se koristi metoda sa ulančanim listama, samo jedan blok pokazivača se drži u glavnoj memoriji. Kada se datoteka kreira, potrebni blokovi se uzimaju iz liste blokova sa pokazivačima. Kada se završi sa korištenjem datoteke, čitaju se novi blokovi pokazivača u memoriju sa diska. Slično, kada se obriše datoteka, blokovi se oslobađaju i dodaju u blok pokazivača na slobodne blokove. Kada se ovaj blok popuni, piše se na disk. Druga tehnika za upravljanje slobodnim prostorom na disku je bit mapa. Jedan disk sa n blokova zahtjeva mapu sa n bita. Slobodni blokovi su predstavljeni sa 1 u mapama, a alocirani blokovi sa 0 (ili obrnuto). Jedan 256 GB disk ima 228 blokova od 1 KB i zahtjeva 228 bita za mapu, koja zahtjeva 32,768 blokova. Nije iznenađenje da bit mapa zahtjeva manje prostora pošto koristi 1 bit po bloku prema 32 bita kod modela sa ulančanim listama. Samo ako je disk gotovo pun (tj. ima nekoliko slobodnih blokova) će model sa ulančanim listama zahtijevati manje blokova nego bit mapa. Treća metoda je uvezana lista indeksnih blokova. Koriste se specijalni indeksni čvorovi koji pokazuju na slobodne blokove. Svaki indeksni blok sadrži adrese slobodnih blokova i pokazivač na sljedeći indeksni blok. Veliki broj slobodnih blokova se može naći veoma brzo. Četvrta metoda je korištenje podataka iz druge strukture za opis datoteka da se zaključi koji su blokovi slobodni. Na primjer, u FAT datotečnom sistemu FAT tablica navodi za svaki blok (blok) njegovog sljedbenika u lancu datoteke. No, ista tablica se može koristiti i za informaciju da li je neki blok slobodan ili neispravan, npr. ukoliko se neki blokovi ne mogu pojaviti u lancu, njihovi brojevi mogu se koristiti u FAT tabeli za oznaku slobodnog bloka. U FAT 16 sistemu vrijednosti 0, 0xFFF7 i 0xFFFF u FAT tabeli ne predstavljaju brojeve blokova gdje se lanac

nastavlja, nego redom predstavljaju slobodni blok, neispravan blok i blok na kraju FAT lanca. Peta metoda je vezana za ekstentnu i kontinualnu alokaciju. Neki datotečni sistemi, kao što je NTFS, rezervišu slobodni kontinualni prostor za konkretnu datoteku određene veličine (npr. 64K za NTFS), čak i ako je ona kraća. Prilikom širenja datoteke, prvo se gleda da li ona ima slobodnog prostora ranije dodijeljenog za nju, a tek ako nema, gledaju se slobodni blokovi predstavljeni bit mapama.

3.71. Kako se provjerava konzistentnost datotečnog sistema?

Konzistentnost datotečnog sistema se povremeno provjerava specijalnim alatima kao što su Unix fsck ili Windows chkdsk ili scandisk u nekim verzijama. Ovi alati se mogu pokrenuti kad god je potrebno, posebno kod pada računara. Alati rade na sljedećem principu. Pripreme se dvije tabele blokova u memoriji: tabela zauzetih i tabela slobodnih blokova. Tabela slobodnih blokova se direktno dobije iz strukture za upravljanje slobodnim blokovima (ulančane liste slobodnih blokova ili bit mape). Tabela zauzetih blokova se dobiva analizirajući indeksne čvorove ili lance alokacije za svaku datoteku. Svaki put kada se zaključi da blok pripada datoteci povećava se odgovarajući element tabele zauzetih blokova za 1. Kada su pregledane sve strukture koje prate zauzete i slobodne blokove (npr. indeksni čvorovi i bit mape) mogu se usporediti ove dvije tabele u memoriji. Ako su svi blokovi ili zauzeti ili slobodni, ovaj dio datotečnog sistema je konzistentan. Ali, ako se pokazalo da je neki blok istovremeno zauzet ili slobodan, ili da nije ni zauzet ni slobodan, ili (najgore) da je zauzet sa tako da dvije datoteke pokazuju na njega, datotečni sistem je nekonzistentan i alat za popravku datotečnog sistema treba da sredi potrebne tabele.

3.72. Navedite neke tehnike povećanja performansi datotečnih sistema.

Takve tehnike su keširanje datotečnog sistema, čitanje unaprijed i fizičko smještanje dijelova datotečnog sistema.

4.1. Šta ulazi u operativni sistem po tradicionalnom i savremenom shvatanju?

Operativni sistem predstavlja sistemski softver koji upravlja hardverskim i softverskim resursima i pruža zajedničke usluge računarskim programima. Gotovo svi računarski programi zahtijevaju operativni sistem za rad, uz rijetke izuzetke poput firmware, programa koji je ugrađen u ROM računara ili drugog programabilnog uređaja.

U modernijem shvaćanju operativnih sistema, oni pružaju i usluge ne samo drugim programima nego i korisnicima kroz aplikacije koje se isporučuju sa njim, pa se postavlja pitanje koliko široku oblast softvera operativni sistem zauzima. Po najužem shvaćanju u operativni sistem ulaze samo skup sistemskih poziva i apstrakcija hardvera, tj. ono što se inače naziva jezgrom.

Najčešće shvaćanje uključuje u opseg operativnog sistema: jezgro, školjku i skup osnovnih sistemskih alata. Školjka može biti tekstualna ili grafička. Po ovom shvaćanju, u OS ulazi onaj minimum potreban za pokretanje aplikativnih programa.

Po modernom shvaćanju, u operativni sistem ulazi sve što je njegov proizvođač uključio. Ovo shvaćanje je, međutim, previše široko iz ugla proučavanja operativnih sistema, jer bi moglo obuhvatiti gotovo sve teme računarskih nauka, s obzirom na sve aplikacije koje se isporučuju u jednom većem sistemu.

4.2. Opišite batch sisteme.

Sistemi sa grupnom obradom (engl. batch) kojima se predaju poslovi na izvršavanje pomoću ulaznih jedinica. Obrada se obavlja sekvencijalno bez komunikacije između korisnika i posla.

Sistemi s grupnom obradom (engl. batch) su najranija vrsta operativnih sistema iz 1960-ih i 70-ih. Primjer ovakvog sistema je IBM 7090/94 IBSYS, sa ulaznim uređajima poput čitača kartica, izlaznim uređajima kao što je linijski štampač, i ulazno/izlaznim uređajima poput jedinice magnetne trake.

Korisnik pripremi posao (engl. job) za obradu. Posao sadrži program i podatke. Podaci su obično uneseni na bušene kartice i zatim snimljeni na traku. Operater to postavi na sistem, i pokrene izvršavanje. Samo OS je stalno u memoriji. Zadatak ovog primitivnog operativnog sistema je da redom učitava poslove koji se nalaze u redu čekanja jedan po jedan. Svaki posao se učitava, pokrene i sačeka njegovo izvršenje. Korisnik u vrijeme izvršavanja ne sarađuje sa svojim programom. Po završetku rada, program šalje rezultate na izlaznu traku ili štampač.

4.3. Opišite multiprogramirane batch sisteme.

Diskovi su omogućili da se poslovi smjeste na uređaj koji ima direktni pristup. To je dovelo do sistema kod kojih se također pripremi grupa zadataka, ali se oni sada ne moraju izvršavati istim redoslijedom. Kako je memorija porasla, u nju je sada postalo moguće smjestiti više poslova istovremeno, pa se ti sistemi zovu multiprogramirani batch sistemi. Ukoliko neki od poslova pređe u stanje čekanja, operativni sistem može pokrenuti izvršavanje drugog posla. Time se na neki način izvršava više poslova paralelno; npr. dok jedan čeka na završetak ulazno/izlazne operacije, procesor prelazi na izvršavanje drugog.

4.4. Opišite time sharing sisteme.

- Daljnjim razvojem multiprogramiranih batch sistema nastali su sistemi u dijeljenom vremenu. Oni omogućavaju da više korisnika istovremeno koriste jedan računar preko terminala (Multics, Unix, VAX/VMS). U memoriji se nalazi više programa, baš kao i kod multiprogramiranih batch sistema. Ali programi se sada ne izvršavaju proizvoljno dugo. Tehnika dijeljenja vremena radi tako da se svaki program u memoriji izvršava u malim intervalima. CPU povremeno prebacuje izvršavanje sa programa na program. To prebacivanje se događa toliko često da korisnik ima osjećaj kontinualnog izvršavanja programa i utiče na njegov rad tokom izvršenja.

4.5. Klasifikacija operativnih sistema prema broju korisnika, procesa i načinu obrade.

Po broju korisnika koji ih istovremeno koriste, OS se dijele na jednokorisničke i višekorisničke. Jednokorisnički OS se obično koristi na stonim računarima, ručnim sistemima i konzolama za igru. Računar je predviđen za samo jednog korisnika, pa su problemi zaštite i dijeljenja resursa jednostavniji. Višekorisnički računari omogućavaju pristup više korisnika istovremeno (preko posebnih terminala ili softvera za terminalski pristup. U nekim slučajevima računarski sistem ima samo jednu tastaturu i ekran, pa ti korisnici ipak ne mogu raditi istovremeno, ali postoji mogućnost prebacivanja između korisnika posebnim komandama. Prema broju procesa koji se mogu istovremeno izvršavati, operativni sistemi se dijele na jednoprocenke i višeprocenke. Jednoprocesni sistemi dopuštaju da se u memoriji smjesti samo jedan program, dok višeprocenki sistemi dopuštaju više programa u memoriji. Potkategorija jednoprocenih operativnih sistema su jednoprogramski. To su sistemi koji ne samo što u memoriji čuvaju samo jedan program, nego je to i jedini program ikada napisan za njih, što se sreće npr. u

neprogramabilnim kalkulatorima ili sistemima koji pokreću ulične semafore. Po broju CPU (procesora) operativne sisteme dijelimo na sisteme za jednoprocesorske i višeprocesorske računare. OS za višeprocesorske računare predviđeni su za sisteme sa više procesora smještenih u istom kućištu. Procesori komuniciraju putem dijeljene memorije. Postoje dva osnovna principa višeprocesorskih operativnih sistema. Prvi je simetrični, koji sve procesore vidi ravnopravno i svaki procesor izvršava i korisničke programe i operativni sistem. Drugi je asimetrično multiprocesiranje, gdje jedan master procesor izvršava dijelove operativnog sistema i dodjeljuje zadatke ostalim procesorima.

4.6 Klasifikacija operativnih sistema prema strukturi jezgra.

Zavisno od toga koliki procenat funkcionalnosti operativnog sistema se izvršava u najvećem nivou privilegija jezgra, podjela operativnih sistema se može napraviti u tri osnovne grupe, koje se dalje mogu dijeliti. Prva grupa su monolitni sistemi. U monolitnim sistemima većina funkcionalnosti jezgra ili čak cijelog operativnog sistema se izvršava u režimu najvećih privilegija. Monolitne možemo podijeliti na potpuno monolitne, sisteme monolitnog jezgra, modularne, slojevite, prstenaste i unikernel. U drugu kategoriju spadaju sistemi sa mikro jezgrom. Kod ovih sistema se nastoji smjestiti u režim najveće privilegije što je moguće manje programskog koda. Njih dijelimo na mikro jezgro sa serverima, OS sa virtualnim mašinama, nano jezgra i egzokernel. Treću grupu predstavljaju jezgra koja su između ove dvije krajnosti, hibridna jezgra.

4.7 Šta je to sistemski poziv?

Najčešća primjena softverskih prekida je sistemski poziv. Korisnički procesi normalno ne mogu pristupiti jezgru, ali program često treba čitati datoteku, slati podatke nekom uređaju, ili možda pokrenuti neki drugi program. Za sve ovo je potrebna akcija jezgra operativnog sistema. Sistemski pozivi su definirane ulazne tačke u jezgru koji obezbjeđuju usluge korisničkim procesima. Realizacija sistemskih poziva koristi mehanizam softverskog prekida koji omogućuje procesu da se prebaci na dobro definirane tačke u jezgru.

4.8 Šta je to API poziv?

Mada postoje načini da programer poziva sistemske pozive, češće se koriste viši nivoi apstrakcije. Problem sa sistemskim pozivima je što su često loše dokumentovani i nekompatibilni između verzija operativnih sistema. Direktno pozivanje sistemskog poziva u Linux-u koristeći INT 80h će raditi samo na određenom procesoru, ili čak samo na određenoj verziji jezgra Linux-a za taj procesor. Umjesto sistemskih poziva, prilikom programiranja, preporučuje se da programer koristi samo API (application programming interface) operativnog sistema, ili ako želi postići i manju ovisnost o operativnom sistemu, standardnu biblioteku programskog jezika. Sistemi pružaju biblioteke ili API koji se nalazi između korisničkog programa i operativnog sistema. Na sistemima sličnim Unix, taj API je obično dio standardne biblioteke jezika (libc). Jedan dio funkcija u toj biblioteci je omot za sistemske pozive, skup funkcija koje se zovu kao sistemski pozivi koje pozivaju, ali prosljeđuju parametre na način koji je pogodniji za jezik C nego za asemblerski jezik. Na Windows NT, API koji omotava sistemske pozive se zove Native API, smješten u ntdll.dll biblioteci. Native API je slabo dokumentirani API koga koriste funkcije regularnog Windows API i neki sistemski programi za Windows. Poziv funkcije omotač-biblioteke predstavlja običan poziv potprograma, koji sam po sebi ne prelazi u režim jezgra, nego prepakuje parametre i pozove odgovarajući softverski prekid. Na ovaj način, biblioteka koja postoji između jezgra OS i aplikacija, povećava prenosivost programa.

4.9 Koje vrste prekida postoje?

Postoje tri vrste prekida: softverski prekidi, hardverski prekidi i izuzeci.

Softverski prekid se pokreće kada korisnički program izvrši specijalnu instrukciju. Ona prisiljava program da pređe u sistemski režim rada i nastavi izvršavanje od dobro poznate memorijske lokacije. Lokacija se određuje na osnovu broja prekida, koji je obično parametar ove instrukcije. Program koji se izvršava na toj lokaciji je pod kontrolom jezgra. Primjeri softverskog prekida su instrukcije `int` ili `sysenter` na Intel Pentium procesorima; `syscall` na AMD procesorima ili `SWI` na ARM procesorima.

Hardverski prekid nastaje kada vanjski događaj pošalje prekidni signal procesoru. Događaji mogu biti pritisak na taster, završetak direktnog prijenosa podataka sa diska u memoriju, periodični otkucaj sata itd. Hardverski prekidi mogu nastati u bilo koje vrijeme. Za razliku od softverskih prekida i izuzetaka, oni nisu vezani za instrukciju koja se trenutno izvršava.

Izuzeci se dešavaju ako korisnički program pokuša izvršiti instrukciju koja se smije izvršiti samo u režimu jezgra, ili pokušava pristupiti području memorije koje njemu ne pripada. Procesori imaju svoj skup izuzetaka koje poznaju. Izuzeci mogu biti kršenje pristupa memoriji, dijeljenje nulom, pokušaj izvršenja nepostojeće instrukcije, ili pristup nepostojećoj lokaciji memorije. U obradi izuzetaka jezgro može simulirati željeno ponašanje (npr. izvršenje instrukcije za aritmetiku sa realnim brojevima na sistemima koji nemaju instrukcije za to) ili sankcionisati neželjeno (prekinuti program koji pokušava pristupiti memoriji dodijeljenoj drugom procesu).

4.10. Šta je Monolitni kernel?

Daleko najčešći način organizacije jezgra nema posebnu strukturu, kao što je slučaj sa MSDOS ili UNIX. OS je napisan kao skup potprograma, od kojih svaki može pozvati bilo koji drugi potprogram kad god je to potrebno. Ako se koristi ova tehnika, svaki potprogram u sistemu ima dobro definirano značenje parametara i rezultata, a svaki potprogram je ima pravo pozvati bilo koji drugi. Cijelo jezgro se izvršava kao jedinstveni proces i svi potprogrami jezgra imaju pune privilegije. Monolitna jezgra se odlikuju dobrim performansama, jer je potrebno manje prelazaka između različitih nivoa privilegije. Ako npr. dijelu za upravljanje memorijom zatreba privremeno snimanje sadržaja memorije na disk, dovoljno je pozvati potprogram za snimanje na disk. Glavne mane monolitnog jezgra su:

Greška, čak i u nebitnom dijelu jezgra, npr. nekoj rijetko korištenoj mrežnoj funkciji, može dovesti do prestanka rada cijelog operativnog sistema.

Razumijevanje principa rada jezgra je daleko teže zbog ogromnog broja međuzavisnosti potprograma koje je teško pratiti.

Dodavanje nove funkcionalnosti (npr. podrška za veće diskove) zahtijeva pravljenje novog jezgra.

Nepotrebne funkcionalnosti (npr. podrška za mrežnu karticu koju nemamo) zauzimaju memorijski prostor, a u nekim slučajevima troše i procesorsko vrijeme.

4.11. Šta je Mikrokernel?

Neke nedostatke monolitnih jezgara možemo zaobići tako što će se ukloniti što je više moguće funkcionalnosti sa privilegijama jezgra, ostavljajući minimalno mikro jezgro (engl. microkernel). Većina operativnog sistema u tom slučaju se implementira u korisničkim procesima. Kada traži uslugu, npr. čitanje datoteke, korisnički proces poziva serverski proces, koji odradi svoj posao i vrati odgovor. U OS sa mikro jezgrom, sve što jezgro radi je realizacija komunikacije između korisničkih procesa i serverskih procesa. OS je razbijen na dijelove, od kojih je svaki samo upravlja jednim aspektom sistema, kao što su usluge datoteka, raspoređivanje procesa, pristup terminalu, ili upravljanje memorijom. Time svaki dio postaje mali i upravljiv. Pošto se svi serverski procesi izvršavaju u korisničkom režimu, a ne u režimu jezgra, oni ne pristupaju direktno hardveru. Posljedica je da npr. greška na

serveru datotečnog sistema neće oboriti cijeli operativni sistem, nego samo taj podsistem. Podsystemi su u odvojenim područjima memorije, pa ne komuniciraju međusobno niti pozivom potprograma niti dijeljenom memorijom. Komunikacija između podsystema obavlja se razmjenom poruka koje raspoređuje mikro-jezgro. Mana mikro jezgra je u slabijim performansama, pošto se prelazak iz sistemskog u korisnički režim rada dešava prilično često. Najpoznatiji operativni sistemi sa mikro jezgrom su Mach, Minix, Symbian i Fuchsia. U jezgru se nalaze samo dva procesa, sistemski i proces sata. Sve ostalo, uključujući i drajvere perifernih uređaja i datotečni sistem se nalazi u korisničkom režimu rada, u formi odvojenih servera. Postoji samo pet sistemskih poziva, koji prelaze u režim jezgra, uglavnom vezanih za slanje i prijem poruka. Nije moguće direktno upisivati iz servera na memorijske apsolutne lokacije, nego se zahtjev za tim obrađuje u jezgru. Minix 3 povećava svoju pouzdanost koristeći reinkarnacijski server, koji poziva poznate podsysteme i u slučaju pada nekog od njih pokreće iz početka podsystem koji se ne odaziva, ako je to moguće. Koriste se i termini nano jezgro i piko jezgro (nanokernel, picokernel). Kod ove vrste jezgra ne samo što je u režimu privilegija jezgra minimalna funkcionalnost, nego je ona realizovana sa što je moguće manje programskih instrukcija u usporedbi sa nekim operativnim sistemima klasifikovanim kao sistemima sa mikro jezgrom.

4.12. Šta je Slojeviti operativni sistem

U cilju povećanja razumijevanja rada jezgra, jedan način strukturiranja OS je urediti operativni sistem kroz više slojeva, tako da je svaki sloj postavljen logički jedan iznad drugog. Jedan operativni sistem strukture slojeva realizovan je na Technische Hogeschool Eindhoven u Nizozemskoj, autor Edgser Dijkstra. Sistem je imao 6 slojeva. Sloj iznad hardvera se bavio dodjelom procesora, prebacivanjem između procesa kada je došlo do prekida ili isteklo predviđeno vrijeme procesa. Svaki sloj predstavlja grupu funkcija operativnog sistema. Potprogrami koji pripadaju jednom sloju smiju pozivati samo potprogramme iz istog ili nižih slojeva. Time se smanjuje mogućnost da promjena jednog dijela operativnog sistema drastično utiče na njegov ostatak. Koncept slojeva ima i svoje nedostatke. Neke funkcionalnosti je teško uklopiti u postojeće slojeve. Neka, na primjer, operativnom sistemu treba dodati mrežne mogućnosti, koje treba uključiti u novi sloj. Ako se taj sloj postavi ispod sloja za rad sa memorijom i datotekama sistema, neće biti moguće čitanje datoteka za konfiguraciju mreže. Ako se mrežni sloj postavi iznad sloja za datoteke i memoriju, tada neće biti moguće dijeliti datoteke na mrežnim diskovima.

4.13. Šta je Operativni sistem zasnovan na virtuelnim mašinama.

U sistemu IBM VM/370 uveden je operativni sistem zasnovan na virtualnim mašinama. Umjesto jezgra, najbliže hardveru nalazi se monitor virtualnih mašina, koji realizuje virtualne mašine za ostale dijelove operativnog sistema. Za razliku od ostalih OS, te virtualne mašine ne predstavljaju adresni prostor jednog procesa sa datotekama, sistemskim pozivima i povećanim virtualnim područjima memorije. Ove virtualne mašine su potpuna kopija hardvera, i uključuju režime privilegija, ulazno/izlazne uređaje, prekide itd. Kako je ovakva virtualna mašina savršena simulacija pravog hardvera, ona može izvršiti bilo koji operativni sistem koji bi radio nad samim hardverom. Tako, dok klasično, monolitno jezgro izvršava jedan operativni sistem, ovim pristupom je moguće pokrenuti više operativnih sistema istovremeno na istom računaru. Na primjer neke virtualne mašine izvršavaju OS/360 za batch obradu ili obradu transakcija, dok ostale izvršavaju jednokorisnički operativni sisteme CMS. Od 2010. godine ovaj koncept operativnog sistema postao je moguć na personalnim računarima. Procesori Intel Core 2 imaju režim za virtualizaciju koji generiše izuzetke kada se izvršavaju privilegovane instrukcija. Pri obradi izuzetaka se simulira ponašanje prave mašine na virtualnoj mašini. Time postaje moguće realizovati monitor virtualnih mašina (zovu ga hipervizor tipa 1), čiji su primjeri Oracle VM i Microsoft Hyper V. Hipervizor tipa 1 se izvršava direktno nad hardverom. Za razliku od njega, hipervizor tipa 2, se izvršava kao korisnički program (npr. Qemu, VMWare) u već aktivnom operativnom sistemu i unutar njega izvršava drugi operativni sistem. OS preko virtualnih mašina je posebno koristan za serverske operativne sisteme. Firmama je često potrebno da imaju servere za

elektronsku poštu, baze podataka, mrežno posredništvo i dijeljenje datoteka. Iz razloga pouzdanosti dobro bi bilo da je svaki od njih poseban računar, pa ako padne operativni sistem na računaru za elektronsku poštu, podaci iz baze su i dalje dostupni. Ali to znači i veću potrošnju električne energije, prostora i manju iskorištenost hardvera. Stavljanjem četiri virtualizovana operativna sistema nad hipervizorom u istoj fizičkoj mašini postiže se zaštita od rušenja jednog od operativnih sistema, a dovoljan je samo jedan računar.

4.14. Kako je organizovana memorija u MSDOS?

DOS cijeli memorijski prostor vidi kao područje od jednog megabajta memorije, pri čemu se na nižim adresama od adrese 0 nalaze interrupt vektori, zatim određeni prostor zauzima jezgro sistema, nakon koga slijedi prostor za korisnički program. Od 640-g kilobajta smješteni su video memorija, nakon nje ROM video kartice, mali prostor za proširenje RAM-a i na vrhu memorije ROM BIOS. Memorijskim lokacijama se ne pristupa linearno, nego koristeći segmentni i ofsetni dio adrese, ali su u ovoj tabeli date apsolutne adrese. No, kako ovaj operativni sistem većinu vremena provodi u realnom režimu rada, apsolutna adresa se dobiva zbirom ofsetnog dijela adrese i segmentnog dijela pomnoženog s 16.

4.15. Koja je uloga tri najvažnije datoteke u MSDOS?

- **IO.SYS**, koji sadrži osnovne drajvere perifernih uređaja i učitava u memoriju glavni dio DOS-a MSDOS.SYS. U PC DOS on se zove IBMBIO.COM
- **MSDOS.SYS**, je jezgro operativnog sistema i sadrži implementaciju sistemskih poziva MSDOS-a, kroz softverske trap-ove između INT 20h i INT 2Fh, od kojih je posebno značajan INT 21h. U PC DOS on se zove IBMDOS.COM
- **COMMAND.COM** je interpreter komandne linije, koji od korisnika očekuje unos komande s parametrima i izvršava komandu ugrađenu u njega ili eksterni program. Ova datoteka se ne čuva cijelo vrijeme u RAM-u u potpunosti, ali je jedan njen dio uvijek prisutan. On ima mali skriptni jezik koji omogućava pokretanje programa s ekstenzijom .BAT. Jedan od njih AUTOEXEC.BAT se pokreće pri svakom podizanju MSDOS.

4.16. Koje su osnovne razvojne linije Windows operativnih sistema

Windows se razvijao u 4 različite razvojne linije: **šesnaestbitne verzije** (koje su samo produžetak MS-DOS-a grafičkim sučeljem), **verzije 9x** (32-bitni sa ugrađenim MS-DOS), **Windows NT verzije** (sa sigurnim 32 bitnim ili 64 bitnim jezgrom) i **verzije CE** (jezgro u ROM-u za džepne uređaje)

4.17. Koji režimi rada su u 16 bitnim Windows i koja im je glavna mana?

U **realnom** režimu (potreban 8086 procesor i 640 K RAM) Windows je samo GUI za MS-DOS, bez ikakve zaštite. **Standardni** režim (procesor 80286 u zaštićenom režimu uz 1 M do 16 M memorije) koristi segmentaciju sa zaštitom. U **proširenom** režimu (bar 80386 sa 2 M RAM) koristi se segmentacija sa straničenjem, teoretsko adresiranje do 4 G (u praksi do 512 M) i preemptivno raspoređivanje MS-DOS programa. **Ali, u sva tri režima se Windows programi raspoređuju kooperativno, što i predstavlja najveću manu ove serije. Krah jedne aplikacije izaziva krah cijelog sistema,** a česta je i potreba za nekim MS DOS drajverima.

4.18. Koja je uloga KERNEL, USER i GDI u Windowsima?

KERNEL.EXE **KRNL386.EXE** Alocira i prati sistemske resurse, koordinira U/I zahtjeve prema hardveru, upravljanje memorijom i pokretanje programa.

USER.EXE Obrađuje U/I zahtjeve, poruke tastaturu, miša, zvuk, korisničko sučelje

GDI.EXE Graphic Driver Interface, odgovoran za crtanje i štampanje

4.19. Kako Windows 9x obrađuje 16 bitne a kako 32 bitne drajvere?

VMM može kreirati virtualne mašine (VM), područja memorije u kojima se aplikacije izvršavaju. Upotreba više virtualnih mašina omogućava MSDOS programima da se izvršavaju u vlastitom memorijskom prostoru, poboljšavajući performanse operativnog sistema. Kada se radi u GUI režimu, DOS pozivi (npr. INT 21h) se preusmjere na odgovarajuće drajvere unutar VMM.

4.20. Šta je to Registry?

Kako su INI datoteke postale mnogobrojne, zamijenjene su **Registry bazom**. **To je centralizovana konfiguracijska baza za softverske postavke i hardverske postavke**. Svi konfiguracijski parametri su smješteni u datoteke User.dat i System.dat, pri čemu operativni sistem pravi njihove kopije kao User.da0 i System.da0. Registry se čita alatom regedit.exe.

4.21. Nabrojte verzije Windows sa NT jezgrom.

Windows NT 3.1, Windows NT 3.5, Windows NT 3.51, Windows 95, Windows ME. Windows NT 4.0, Windows 2000, Windows XP, Window Server 2003, Windows Vista, Windows Server 2008, Windows 7, Windows 8, Windows 10.

4.22. Čemu služi HAL.DLL?

U dizajnu jezgra donesena je jedna, za to vrijeme neobična odluka. Kako je OS zamišljen za različite platforme, uveden je **još jedan sloj između hardvera i jezgra**. HAL (Hardware Abstraction Layer) je NT-ov interfejs prema samom procesoru. Kako je NT dizajniran da bude prenosiv između različitih računarskih arhitektura. **Dijelovi koda specifični za pojedini procesor pišu se u ovom modulu (datoteka HAL.DLL). HAL eksportuje standardni procesor i driver-i se pišu za njega**. Čak i na pločama sa istim procesorom može biti razlika, recimo ako su jednoprocesorske i višeprocorske. Tako se **prave različite verzije HAL.DLL-a, npr. jednoprocesorska, višeprocorska i debug verzija**. Primjer funkcije koju HAL eksportuje prema jezgru je WRITE_PORT_UCHAR, koja na periferni port šalje osmootnu vrijednost. Prebacivanjem ove akcije u potprogram jezgro i driver-i su rasterećeni brige o tome da li je periferni uređaj memorijski ili U/I mapiran. Slično tome, funkcija HalEnableInterrupt za uključanje prekida smanjuje potrebu pisanja dijelova jezgra u asemblerskom jeziku.

4.23. Koje su dvije osnovne uloge NT kernela?

Obavlja dvije funkcije: **raspoređivanje niti i procesa, te upravljanje prekidima**.

4.24. Kako Object Manager u Windows Executive vidi objekte a kako procesi?

Object Manager obavlja zadatke upravljanja objektima koji uključuju identifikaciju i brojanje referenci. Kada aplikacija otvori resurs, OM locira objekt ili kreira novi. Objekti se označavaju identifikatorima koji se zovu handle. Oni su jedinstveni na nivou aplikacije, ali ne i između aplikacija. Brojač referenci prati koliko je procesa pristupilo resursu. Identifikacija objekta se nalazi u NT-jevom prostoru za imena. Svi djeljivi resursi imaju imena, kao što su imena datoteka, registry ključeva ili semafora. Svi drugi podsistemi u Executive koriste Object Manager da definišu objekte i upravljaju objektima koji predstavljaju resurse. Na primjer, kroz Object manager, Process manager definiše objekt "proces" kojim prati aktivne procese

Process Manager radi sa Kernelom da definiše objekte procesa i tredova. Process Manager analizira Kernelov objekt Process i dodaje mu identifikator procesa PID, pristupni token, adresnu mapu i tabelu handle-ova. Opis svakog procesa drži se u strukturi EPROCESS, koja pruža pogled na proces iz Executive.

4.25 Na koji način Security Reference Monitor (SRM) određuje koji objekt se može dodijeliti?

Security Reference Monitor je vezan s Object Manager-om. Object Manager poziva Security Reference Monitor, koji provjerava prava pristupa prije nego dopusti aplikaciji da otvori objekt. Object manager također zove Security Reference Monitor prije nego dopusti procesu da obavlja druge operacije nad objektima, kao što su čitanje i pisanje. SRM implementira sigurnosni model baziran na sigurnosnim identifikatorima (SID) i Diskrecionim listama za kontrolu pristupa (DACL). Svaki proces u NT-u ima pridruženi token pristupa koji sadrži SID korisnika, a koji posjeduje proces i SID-ove grupa kojima on pripada. Objekti koje Object Manager evidentira pokazuju na sigurnosni deskriptor, koji sadrži SID vlasnika objekta, njegovu primarnu grupu i pokazivač na diskrecionu listu.

4.26. Koje načine alokacije memorije procesima koristi Virtual Memory Manager u Windows Executive ?

Virtual Memory Manager transformiše virtualne adrese procesa u fizičke adrese i kontroliše alokaciju fizičke memorije. Na IA-32 Windows sistemima, virtualna memorija je velika 4G. Aplikacije mogu pristupiti prva 2G (user mode polovina, koja se mijenja kako se pokreću novi procesi). Druga polovina (2-4G) je Kernel polovina i ona se ne mijenja. Virtual Memory Manager implementira virtualnu memoriju koristeći straničenje. Novopokrenuti proces se mapira (njegov izvršni program na disku) u fizičku memoriju. Ako procesi zahtijevaju više memorije nego što je fizički ima, koristi se zamjenska datoteka za virtualnu memoriju. Može ih biti i više na različitim diskovima.

4.27. Čemu unutar Windows Executive služi I/O Manager?

I/O manager integriše dodatne drajvere za uređaje sa NT-om. Drajver uređaja prevodi komande koje NT i aplikacije šalju uređaju i proslijeđuju zahtjeve hardveru. Ako Microsoft nije isporučio drajver, proizvođač hardvera jeste.

4.28. Čemu unutar Windows Executive služi Cache manager?

Cache manager sarađuje sa VMM i drajverima za datotečni sistem. Cache manager održava NT globalni (dijeljen od svih datotečnih sistema) keš za datoteke. NT keš je datotečno orijentiran (za razliku od Windows 95 koji je blokovno orijentiran). Područje predviđeno za cache datoteka alocira se u virtualnom području procesa, tako da ako nema dovoljno fizičke memorije cache će se smanjivati postojećim algoritmima straničenja.

4.29. Čemu unutar Windows Executive služi Process Manager?

Process Manager radi sa Kernelom da definiše objekte procesa i tredova. Process Manager analizira Kernelov objekt Process i dodaje mu identifikator procesa PID, pristupni token, adresnu mapu i tabelu handle-ova. Opis svakog procesa drži se u strukturi EPROCESS, koja pruža pogled na proces iz Executive. Ona ima istu funkciju kao kontrolni blok procesa.

4.30. Čemu unutar Windows Executive služe PnP Manager i Power Manager?

Plug and Play manager (od Windows 2000) određuje koji su drajveri potrebni za podršku odgovarajućem uređaju i učitava te drajvere. Preuzima zahtjeve svakog uređaja za resursima.

Power manager (od Windows 2000) upravlja promjenama statusa napajanja za sve uređaje koji podržavaju te promjene. To se često radi kroz složeni niz uređaja za kontrolu drugih uređaja. Drajveri treba da reaguju na sljedeće informacije: Nivo aktivnosti sistema, nivo baterije itd.

4.31. Čemu unutar Windows služi dispečer sistemskih servisa?

Dispečer sistemskih servisa prema rednom broju sistemskog poziva pogleda u tabeli adresu gdje se u Kernelu ili Executive nalazi potprogram koji implementira i pozove poziv.

4.32. Šta su prirodni API i NT.DLL?

Sistemski pozivi koje prima dispečer sistemskih servisa i dalje proslijede ka Executive treba da budu realizovani i sa druge strane, iz korisničkih procesa. Prije svega, svakom od tih sistemskih poziva treba poslati parametre u formatu kakav očekuju programi u jeziku C i omogućiti njihovo pozivanje iz aplikacija. Ovi potprogrami se zovu **NT-jev prirodni API**, smješten u datoteci NTDLL.DLL. Ovaj API ima oko 2500 funkcija koji se pozivaju koristeći softverske izuzetke. Softverski izuzetak je hardverski potpomognuti način da se prelazi između korisničkog režima rada i režima jezgra.

4.33. Koji serveri okruženja su postojali ili postoje u Windows NT seriji?

Skup API funkcija koje su nastale od nekog operativnog sistema, a koji se veže na prirodni API da bi pozvao usluge jezgra zove se okruženje. Do verzije Windows 2000 bila su tri takva okruženja **Win32, POSIX i OS/2**. Ali od Windows 10 (build juni 2017) uveden je novi Windows Subsystem for Linux, koji omogućava pokretanje Linux programa pod Windows-om.

4.34. Kako se pozivaju funkcije Win32 API na 32 bitnim platformama?

U 32 bitnim verzijama parametri funkcija se smještaju na stack prije poziva, a sama funkcija ih nakon završetka obriše sa steka. Parametri su najčešće 32 bitne vrijednosti, čak i kada po prirodi podatka zauzimaju manje prostora (npr. logičke vrijednosti).

4.35. Zašto je uveden COM i koje su tehnike bazirane na njemu?

U sistemima sa grafikom i prozorima, kao što je Windows, pojavila se potreba za većom komunikacijom između procesa. Rješenje su pronašli u načinu pisanja svakog specijalizovanog programa, tako da on pruža više ulaznih tačaka.

Uprkos tome, osnovni Win32 API, dizajniran za programe u jeziku C nije bio dovoljan za takve aplikacije, pa je zbog toga uveden pristup komponenti. Komponenta je skup srodnih funkcija spojenih u zajedničku datoteku (DLL ili EXE) koje se mogu pozivati i ugrađivati u druge programe. Ove komponente se opisuju binarnim formatom.

Već od Windows NT 3.1 definisan je standard za takve komponente koji se zove Component Object Model (**COM**). Za dobro napisane komponente, COM dozvoljava ponovnu upotrebu objekata bez znanja o njihovoj unutrašnjoj realizaciji.

Zahvaljujući COM-u omogućene su tehnike poput međuprocenog ili čak međukompjuterskog programiranja (posljednje koristi podršku DCOM-a).

4.36. Šta su Windows servisi?

Windows servisi su računarski programi koji rade u pozadini, koji nemaju interakciju sa korisnikom, nego isključivo sa datotekama, mrežom ili Registry-jem. Oni su osnovna komponenta operativnog sistema Microsoft Windows i omogućavaju stvaranje i upravljanje dugotrajnih procesa.

4.37. Čemu služe NTVDM, WOW i WOW64?

NTVDM je specijalni program koji se pokreće kada se pokrene program namijenjen za MS DOS i Windows 3.1. NTVDM može da poziva odgovarajuće rutine za simuliranje željenog ponašanja. NTVDM se pokreće i kada se želi na 32 bitnoj verziji Windows-a pokrenuti 16 bitni Windows program. Uz njega se aktivira Windows on Windows (**WoW**). Podsystem **WOW** operativnog sistema preusmjerava stare 16-bitne pozive u nove 32bitne ekvivalente kako bi pružili podršku za 16-bitne pokazivače, modele memorije i adresni prostor.

NTVDM i WoW ne postoje u 64 bitnoj verziji Windows, jer procesor nema u 64 bitnom režimu podršku za 16 bitne programe. Podsystem **WoW64** to omogućava. On sadrži sloj komponenti za kompatibilnost koji ima slične interfejsa na svim 64-bitnim verzijama Windowsa. Njegov cilj je da stvori 32-bitno okruženje potrebno za pokretanje neizmijenjenih 32-bitnih Windows aplikacija na 64-bitnom sistemu.

4.39. Šta je Windows runtime?

Windows runtime je objektno orijentisani API, sa klasama koje su smještene u windows.*.dll datotekama. Prvi put je predstavljen u Windows-u 8. Može se koristiti i sa aplikacijama u C++ (prirodni kôd) i sa .NET aplikacijama (kontrolisani kôd).

4.40. Šta je to Windows CE?

Windows CE je operativni sistem dijeljenog izvornog koda, gdje proizvođači uređaja licenciraju izvorni kôd. Na raspolaganju je nekoliko arhitektura. U Windows CE sistemima cijeli operativni sistem je u ROM-u kao i aplikacije koje dolaze sa sistemom. Nije kompatibilan s WIN32.

4.41. Kako su nastali Unix, Minix i Linux?

Inženjeri Bell Labs-a Denis Richie i Ken Thompson 1969. su kreirali prvi UNICS, kasnije **UNIX** sistem, nakon što se firma povukla iz MULTICS projekta zbog prevelikih hardverskih zahtjeva. On je prvenstveno napisan u asemblerskom jeziku. Kasnije je čitav sistem ponovo napisan u programskom jeziku C (koga su također razvili), čime je znatno olakšano prenošenje UNIX operativnog sistema na druge vrste računara.

Paralelno su nastala još dva jezgra pisana od početka, a sa stanovišta sistemskih poziva slična Unix jezgru.

Prvo je **Minix**, prije svega namijenjen u obrazovne svrhe, objavljen u okviru knjige Andrew Tanenbaum-a, "Dizajn operativnih sistema". Iz razloga da bi to jezgro ostalo dovoljno malo, kako bi bilo upotrebljivo za nastavu, autor nije dopuštao dodavanje novih osobina u Minix.

Drugo jezgro je **Linux**. Linus Torvalds je napisao novo jezgro koje je bilo kompatibilno sa UNIX jezgrom, ali sa dostupnim izvornim kodom i liberalnom politikom oko izmjena tog izvornog koda.

4.42. Šta je to POSIX i od kojih dijelova se sastoji?

Najvažniji standard koji omogućava međusobnu kompatibilnost operativnih sistema, na nivou izvornog koda aplikacija, nastalih iz Unix-a je **POSIX**. POSIX, skraćenica od Portable Operating System Interface je zajedničko ime za porodicu povezanih standarda koje definiše Institut inženjera elektrotehnike i elektronike (IEEE).

Trenutna verzija ovog standarda podijeljena je u 4 dijela:

- Dio 1: Osnovne definicije – terminološki rječnik, varijable okruženja, preporučena struktura direktorija, terminalski interfejs, preporučeni način pisanja alata, C biblioteke.

- Dio 2: Sistemski interfejsi – opisuje funkcije pozivane iz biblioteke za C za pristup sistemskim pozivima, kao i standardnu C biblioteku.

- Dio 3: Školjka i alati – Interne komande školjke i sintaksa alata.

- Dio 4: Razlozi - napomene o poglavljima prethodna tri dijela o motivaciji za standard.

4.43. Objasnite pojam kernel modula u Linux sistemima?

Kernel moduli su dijelovi koda koji se na zahtjev mogu učitati i istovariti u kernel (osnovno jezgro operativnog sistema). Oni proširuju funkcionalnost kernela bez potrebe za ponovnim pokretanjem sustava. Obično koriste za dodavanje podrške za novi hardver (kao upravljački programi uređaja) i / ili datotečni sistem ili za dodavanje sistemskih poziva.

4.44. Šta je Linux distribucija?

Linux distribucija je operativni sistem napravljen iz kolekcije softvera koja se temelji na Linux kernelu i često sistemu upravljanja paketima. Korisnici Linuxa obično dobivaju svoj operativni sistem preuzimanjem jedne od Linux distribucija. Sastoji se od jezgra koje se nalazi u glavnoj datoteci i modulima, sistema za pokretanje (boot loader, inicijalizacijski program i inicijalni RAM disk), skupa biblioteka od kojih je najvažnija libc, pozadinskih programa, školjke, alata iz komandne linije, grafičke infrastrukture oko X.org servera i alata u grafičkom režimu.

4.45. Šta su deskriptori procesa u Linux-u?

Procesi u jezgri se evidentiraju kroz deskriptore procesa. Deskriptor procesa ili niti je sastavljen od strukture `task_struct` i svih objekata na koje ona pokazuje. O svakom procesu su sačuvane informacije o stanju, adresnom prostoru, otvorenim datotekama, signalima, resursima i pokazivačima na druge strukture. Svaki `task_struct` objekt je stavljen u ulančanu listu koju koristi raspoređivač, a koja počinje objektom `init_task`.

4.46. U kojim stanjima može biti proces u Linux-u?

Proces u Linuxu može biti u sljedećim stanjima:

- Running**(trenutno se izvršava ili čeka u redu spremih procesa)
- Interruptiblewaiting**(u stanju čekanja, ali ga POSIX signal može reaktivirati)
- Uninterruptiblewaiting**((u stanju čekanja, signali ga ne mogu reaktivirati)
- Stopped** (zaustavljen signalom SIGSTOP)
- Traced** (u trenutku sesije sa debugger-om)
- Zombie** (završio, ali njegov deskriptor se još čuva)
- Dead** (kernel briše proces).

4.47. Koje metode međuprocenjske komunikacije postoje u Linux-u?

Linux u jezgru podržava klasične UNIX komunikacijske **mehanizme, signale i cijevi**. Pored ovoga podržani su **semafori, poruke i dijeljena memorija**.

Signali su najjednostavniji komunikacijski mehanizam prema korisničkim procesima, ali ih jezgro ne koristi. Koriste se za informaciju procesima o izuzecima.

Linux realizuje cijevi preko dva datotečna objekta. Oni dijele isti privremeni indeksni čvor. Indeksni čvor vezan za cijev ne predstavlja datoteku, nego se koristi specijalni, za korisnika nevidljivi, datotečni sistem pipefs. Taj datotečni sistem je u memoriji, u području jezgra. Dva datotečna objekta imaju različite zadatke, jedan je samo za čitanje a drugi samo za pisanje prema cijevi.

Postoje dvije vrste semafora, semafori za među-procesnu komunikaciju i semafori za sinhronizaciju jezgra. Semafori za sinhronizaciju korisničkih procesa. se čuvaju u jedinstvenom nizu `sem_array`. Proces koji želi obaviti semaforne operacije poziva sistemski poziv `semop`.

Procesi mogu komunicirati i redovima poruka. Proces koji želi komunicirati sa drugim preko poruka poziva `msgget` i šalje odnosno čita poruke pozivima `msgsnd` i `msgrcv`. Poruke u redu čekanja su u objektima `msg_queue` koji sadrže dva reda čekanja, za pošiljaoca i primaoca. Red čekanja poruka je ograničene veličine. Ako pošiljalac ne može dalje slati poruke jer je red pun, on se blokira.

4.48. Kako je organizovana memorija procesa u Linux-u?

Svaki proces ima virtualni i fizički pogled na memorijsko područje. Novi virtualni prostori se kreiraju pri pozivima `fork` i `exec`. Linux jezgro za svaki proces ima svoje, mašinski ovisno područje u memoriji, podijeljeno na pokazivače na sve dostupne stranice i nerezervisani dio. Procesima je dostupno samo njihovo vlastito područje, kojeg čine: *Kernel područje*(nevidljivo za program), *stek područje*, *.bss sekcija*(neinicijalizirani podaci), *data sekcija*(podaci), *text sekcija*(programski kod) i *zabranjeno područje*.

4.49. Koja je uloga biblioteka `libc` i `libm` u Linux-u?

-**Libc** je najvažnija biblioteka u Linux sistemima. Ona pruža funkcionalnost standardne C biblioteke zajedno sa sistemskim pozivima jezgra. To uključuje funkcije koje direktno pozivaju sistemski poziv poput `fork`, `open`, `read`, `write`, `brk`, `execv`, `exit`, kao i funkcije iz standardne C biblioteke.

-**Libm** ima ulogu matematičke unkcije iz standardne C biblioteke (`sin`, `cos`, `atan`, `sqrt`,...)

4.50. Koja je uloga pozadinskog procesa `crond`, `cupsd`, `sendmail` i `httpd` u Posix sistemima?

-Za raspoređivanje procesa u određenim trenucima (npr. automatski backup) koriste se pozadinski procesi **crond** i atd. **Crond** je namijenjen za periodične zadatke (npr. preuzimanje nove verzije softvera svakog četvrtka).

-Printer spooler je realizovan procesima **cupsd** i **lpd**. Mogućnosti **cupsd** su znatno veće od **lpd**(korišten za stari stil štampanja), on po potrebi pokreće konverziju programe za štampu, a može mu se pristupiti i preko web interfejsa.

- Slanje elektronske pošte obavlja se pozadinskim procesom **sendmail** koji prihvata poruke od korisnika i šalje ih prema drugim računarima na Internetu.

- Ako računar služi za čuvanje web stranica, na njemu se pokreće odgovarajući **httpd** World Wide Web server. Najpopularnijim se smatra Apache httpd. Serveru se može pristupiti koristeći web browser.

4.51. Koja je najpopularnija školjka u Linuxu i šta je Coreutils?

-Najpopularnija školjka u Linuxu je **bash**. Kolekcija najvažnijih komandi iz komandne linije koji se koriste u Unix, a tako i Linux sistemima zove se **Coreutils**. To uključuje oko 100 komandi za rad sa datotekama i direktorijima, prikaz datoteka i neke sistemske alate (chgrp, chown, chmod, cp, dd, df, dir, ln, ls, mkdir, mkfifo, mknod, mv, rm, rmdir, cat,...).

4.52. Na kojim operativnim sistemima su bazirana jezgra Androida, iOS, Windows Phone i Simbian?

Android operativni sistem zasnovan je na Linux jezgru. **iOS** je zasnovan na Mac OS X jezgro. **Windows Phone** operativni sistemi su prošli kroz dvije vrste arhitektura. Verzije Windows Mobile i Windows Phone do verzije 7 bile su bazirane na Windows CE jezgru, dok je Windows Phone baziran na Windows NT jezgru. **Simbian** koristi operativni sistem Psion EPOC.

4.53. Koja je uloga procesa zygote u Androidu?

Pozadinski proces **zygote**, pokreće Java procese višeg nivoa. On je vezan za sistemske biblioteke i Java biblioteke. Mobilnim uređajima potrebna je velika brzina pokretanja procesa, a Java platforma nema tu osobinu. Proces **zygote** rješava taj problem. **Zygote** je odgovoran za podizanje i inicijalizaciju Dalvik-a do tačke gdje je spreman za pokretanje sistema ili aplikacijskog koda napisanog u Java.

Prvi proces koji **zygote** uvijek pokreće, zove se **system_server** (sistemski server), koji sadrži sve osnovne servise operativnog sistema. Ključni dijelovi su menadžer napajanja, menadžer paketa, menadžer prozora i menadžer aktivnosti. Iz **zygote** će se po potrebi kreirati ostali procesi. Neki od njih će se izvršavati svo vrijeme, kao što je telefonski proces. Dodatni procesi aplikacija će se kreirati i zaustaviti po potrebi dok je sistem pokrenut.

4.54. Šta je Dalvik?

- **Dalvik** je virtualna mašina koja je zadužena za pokretanje aplikacija višeg nivoa napisanih u Java programskom jeziku. Java kôd u Android aplikacijama je dostupan u Dalvik-ovom formatu. Ovaj format simulira mašinu baziranu na registrima umjesto Java tradicionalnog stack baziranog formata. Format Dalvikovog bytecode omogućava brže izvršavanje, dok i dalje podržava JIT (Just-in-Time) kompilaciju. Dalvikov bytecode je također efikasniji u pogledu prostora diska i RAM-a.

4.55. Koje vrste ulaznih tačaka postoje u Android aplikaciji?

Android aplikacije nemaju jednu glavnu ulaznu tačku koja se izvršava kada ih korisnik pokrene. Umjesto toga, objavljuju pod `manifest<application>` oznakom razne ulazne tačke koje opisuju različite stvari koje aplikacija može

učiniti. Postoje 4 vrste ulaznih tačaka, i svaka od njih definiše osnovne tipove ponašanja koje aplikacije mogu pružiti, a to su: aktivnost, prijemnik, servis i pružalac sadržaja. Aktivnost je ulazna tačka za interakciju sa korisnikom. Ona predstavlja jedan ekran sa korisničkim interfejsom. Usluga/Servis je opšta ulazna tačka održavanje aplikacije da radi svo vrijeme pozadini. To je komponenta koja radi u pozadini za obavljanje dugotrajnih operacija ili za obavljanje posla za daljinske procese. Usluga ne pruža korisnički interfejs. Prijemnik je komponenta koja omogućava sistemu da isporučuje događaje u aplikaciju izvan redovnog protokola korisnika, omogućavajući aplikaciji da odgovara nasistemske programe. Pružalac sadržaja upravlja zajedničkim skupom podataka koje aplikacija može spremiti u datotečnom sistemu, u bazi podataka SQLite, na webu ili na bilo kojoj drugoj stalnoj lokaciji za skladištenje kojoj aplikacija može pristupiti.

4.56. Koji dijelovi LEKOS jezgra su napisani u asemblerskom jeziku a koji u C?

U asemblerskom jeziku je napisan dio programa koji se zove **boot sector**, a svi ostali dijelovi su napisani u jeziku C.